

운영체제 7

남궁명수

[1-1] 파일과 디렉터리	3
[1] 파일과 디렉터리	4
[2] 파일 시스템 인터페이스	6
[3] 파일의 생성	7
[4] 파일의 읽기와 쓰기	9
[5] 비순차적 읽기와 쓰기	12
[6] fsync()를 이용한 즉시 기록	14
[7] 파일 이름 변경	17
[8] 파일 정보 추출	19
[9] 파일 삭제	21
[10] 디렉터리 생성	23
[11] 디렉터리 읽기	25
[12] 디렉터리 삭제하기	27
[13] 하드 링크	28
[14] 심볼릭 링크	31
[15] 하드 링크 vs 심볼릭 링크	33
[16] 파일 시스템 생성과 마운트	34
[1-2] 파일 시스템 구현	37
[1] 생각하는 방법	38
[2] 전체 구성	39
[3] 파일 구성: 아이노드	42
[4] 디렉터리 구조	47
[5] 빈 공간의 관리	50
[6] 실행 흐름: 읽기와 쓰기	50
[7] 캐싱과 버퍼링	54
[1-3] 지역성과 Fast File System	58
[1] 문제: 낮은 성능	59
[2] FFS: 디스크에 대한 이해가 해답이다.	60
[3] 파일 시스템 구조: 실린더 그룹	60
[4] 파일과 디렉터리 할당 정책	62
[5] 파일 접근의 지역성 측정	63
[6] 대용량 파일 예외 상황	65
[7] FFS 에 대한 기타 사항	67
[1-4] 크래시 일관성: FSCK 와 저널링	71
[1] 예제 1	72

[2] 해법 1: 파일 시스템 검사기(fsck)	77
[3] 해법 2: 저널링(또는 Write-Ahead Logging)	81
[1-5] 로그 기반 파일 시스템	86
[1] 디스크에 순차적으로 쓰기	88
[2] 순차적이면서 효율적으로 쓰기	89
[3] 적절한 버퍼의 크기는?	90
[4] 문제 1: 아이노드 찾기	92
[5] 간접 계층을 이용한 해법: 아이노드 맵	93
[6] 최종 완성: 체크포인트 영역	94
[7] 디스크에서 읽기: 요약	95
[8] 디렉터리 관리 방법은?	96
[9] 새로운 문제: 가비지 컬렉션	97
[10] 블록의 최신 여부 판단	98
[11] 정책: 어떤 블록을 언제 정리하는가	99
[12] 크래시로부터의 복구와 로그	100
[1-6] 데이터 무결성과 보호	102
[1] 디스크 오류 모델	102
[2] 숨어있는 섹터 에러(Latent Sector Error)	105
[3] 손상 검출: 체크섬	107
[4] 체크섬의 활용	110

[1-1] 파일과 디렉터리

핵심 질문: 어떻게 영속 장치를 관리하는가

- 운영체제가 영속 장치를 어떻게 관리해야 할까?
- API 들은 어떤 역할이 있는가?
- 구현의 중요한 측면은 무엇인가?

지금까지 우리는 운영체제를 구성한 두 가지 핵심 개념을 살펴보았다.

하나는 CPU 를 가상화한 “프로세스”,

다른 하나는 메모리를 가상화한 “주소 공간”이다.

이 두 개념은 서로 협력하면서, 응용 프로그램들이 마치 서로 완전히 독립된 환경에서 실행되는 것처럼 만들어준다.

각 프로그램은 자신만의 CPU 를 가진 것처럼 보이고, 자신만의 메모리를 가진 것처럼 느껴진다.

이러한 “환상”은 프로그램 개발을 훨씬 쉽게 만들어준다.

이 개념은 데스크탑과 서버뿐 아니라, 스마트폰을 포함한 거의 모든 프로그래밍 가능한 플랫폼에서 널리 사용된다.

(1) 새로운 핵심 개념: 영속 저장 장치

이번 장에서는 또 하나의 중요한 가상화 퍼즐 조각을 추가한다.

그것이 바로 영속 저장 장치(persistent storage)이다.

하드 디스크 드라이브(HDD)나 더 최근의 솔리드 스테이트 드라이브(SSD) 같은 장치들은 데이터를 영구적으로 저장한다.

또는 최소한 매우 오랜 시간 동안 데이터를 유지한다.

이것은 전원이 꺼지면 데이터가 사라지는 메모리(RAM)와는 다르다.

영속 저장 장치는 전원이 꺼져도 데이터를 그대로 보존한다.

그래서 운영체제는 이런 장치를 훨씬 더 신중하게 다뤄야 한다.

왜냐하면 여기에 저장된 데이터는 사용자에게 매우 중요한 정보일 가능성이 높기 때문이다.

[1] 파일과 디렉터리

저장 장치를 가상화하기 위해 만들어진 두 가지 핵심 개념이 있다.

(1) 첫 번째 개념: 파일(file)

파일은 단순히 읽거나 쓸 수 있는 바이트의 순차적인 배열이다.

각 파일은 저수준의 이름(low-level name)을 가지며, 보통 숫자로 표현된다.

하지만 사용자는 이 이름을 직접 알지 못한다(이 부분은 앞으로 더 자세히 다룬다).

역사적인 이유로 이 저수준 이름은 아이노드 번호(inode number)라고 불린다.

앞으로의 장에서 inode에 대해 더 깊게 설명하겠지만, 지금은 “각 파일은 inode 번호와 연결되어 있다”라고 이해하면 된다.

대부분의 시스템에서 운영체제는 파일의 “의미”를 알지 못한다.

즉, 어떤 파일이 이미지인지, 문서인지, C 코드인지 전혀 모른다.

파일 시스템의 역할은 단순하다.

- 데이터를 디스크에 안전하게 저장하고, 요청이 오면 그대로 다시 돌려주는 것

하지만 이 작업은 생각보다 간단하지 않다.

(2) 두 번째 개념: 디렉터리(directory)

두 번째 핵심 개념은 디렉터리(directory)이다.

파일과 마찬가지로 디렉터리도 저수준 이름(예: inode 번호)을 가진다.

하지만 파일과 달리 디렉터리는 내부 구조가 정해져 있다.

디렉터리는 다음과 같은 형태의 목록을 가진다.

- <사용자가 읽을 수 있는 이름, 저수준 이름>

예를 들어, 어떤 파일이 inode 번호 10을 가지고 있고,

사용자에게는 “foo”라는 이름으로 보인다고 하자.

그러면 해당 디렉터리에는 다음과 같은 항목이 존재한다.

- (“foo”, “10”)

즉, 사용자가 이해할 수 있는 이름과 실제 시스템 내부 이름을 연결해주는 역할을 한다.

디렉터리의 구조와 트리

디렉터리 항목은 파일을 가리킬 수도 있고, 다른 디렉터리를 가리킬 수도 있다.

디렉터리 안에 또 다른 디렉터리를 포함할 수 있기 때문에,

사용자는 디렉터리 트리(directory tree) 또는 디렉터리 계층(directory hierarchy) 구조를 만들 수 있다.

루트 디렉터리와 경로

디렉터리 계층은 항상 루트 디렉터리(root directory)에서 시작한다.

Unix 계열 시스템에서는 루트 디렉터를 /로 표현한다.

원하는 파일이나 디렉터리는 구분자(/)를 사용해 경로로 표현한다.

예를 들어,

- /foo 라는 디렉터를 루트 아래에 만들고
- 그 안에 bar.txt 파일을 만든다면
- 이 파일의 절대 경로는: /foo/bar.txt

더 복잡한 예

좀 더 복잡한 디렉터리 구조를 생각해보자.

이 구조에서 유효한 디렉터리는 다음과 같다.

- /
- /foo
- /bar
- /bar/bar
- /bar/foo

유효한 파일

- /foo/bar.txt
- /bar/foo/bar.txt

여기서 중요한 점은 서로 다른 위치에 있으면 같은 이름의 파일도 존재할 수 있다.

예를 들어, “bar.txt”라는 이름이 두 번 등장할 수 있다.

하지만 경로가 다르기 때문에 서로 다른 파일이다.

- /foo/bar.txt
- /bar/foo/bar.txt

파일 이름 구조

이 예제를 보면 파일 이름은 보통 두 부분으로 구성되어 있다.

예: main.c

- main: 실제 이름
- .c: 파일 종류를 나타내는 관례적 확장자

예를 들어:

- .c : c 소스 코드
- .jpg : 이미지
- .mp3: 음악

하지만 중요한 점은 확장자는 단지 관례일 뿐, 반드시 내용과 일치해야 하는 것은 아니다.
즉, main.c 라고 해서 반드시 C 코드일 필요는 없다.

이 구조에서 중요한 기능은 바로 파일을 효율적으로 “이름 붙이는 기능”이다.
왜냐하면 어떤 자원이든 접근하려면 가장 먼저 필요한 것은 “이름”이기 때문이다.

Unix 파일 시스템은 이러한 이름 체계를 통해
디스크, USB, CD-ROM 등 다양한 장치에 있는 파일들을 하나의 통합된 방식으로 접근할 수 있게 한다.
즉, 모든 파일은 하나의 디렉터리 트리 안에서 통합적으로 관리된다.

[2] 파일 시스템 인터페이스

이번에는 파일 시스템 인터페이스를 좀 더 자세히 살펴보자.
파일을 생성하고, 접근하고, 삭제하는 기본 동작들부터 시작한다.

이러한 동작들은 겉보기에는 매우 단순해 보일 수 있다.
하지만 실제로는 그렇지 않으며, 특히 파일 삭제를 담당하는 unlink()라는 시스템 콜은 다소 혼란스러울 수 있다.

이 장의 마지막에 이르면, 이러한 개념들이 더 이상 혼란스럽지 않게 이해될 것이다.

[3] 파일의 생성

가장 기본적인 파일 시스템 동작 중 하나는 파일 생성이다.
먼저 이 기본적인 연산부터 살펴보자.

파일은 open 시스템 콜을 사용해서 생성할 수 있다.

open() 호출 시 O_CREAT 플래그를 전달하면 새로운 파일을 만들 수 있다.

다음은 현재 디렉터리에 “foo”라는 이름의 파일을 생성하는 코드이다.

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC
```

(1) open()의 플래그 의미

open()은 여러 개의 플래그를 동시에 받을 수 있다.

이 예제를 기준으로 각각의 의미는 다음과 같다.

- O_CREAT: 파일이 없으면 새로 생성한다.
- O_WRONLY: 파일을 쓰기 전용으로 연다(읽기는 불가능)
- O_TRUNC: 파일이 이미 존재할 경우, 내용을 모두 삭제하고 크기를 0으로 만든다.

즉, 이 조합은 “파일이 없으면 만들고, 있으면 내용을 초기화 한 뒤, 쓰기 전용으로 연다”는 의미이다.

(2) 파일 디스크립터(file descriptor)

open()의 중요한 반환값은 파일 디스크립터이다.

파일 디스크립터는 각 프로세스마다 존재하는 정수값이며,

Unix 시스템에서 열린 파일을 접근하는 데 사용된다.

즉, 파일을 읽거나 쓰기 위해 항상 이 값을 사용한다.

(3) 파일 디스크립터의 의미

파일 디스크립터를 이해하는 방법은 여러 가지가 있다.

(3-1) capability(권한 객체)

파일 디스크립터는 단순한 숫자가 아니라. 특정 파일에 대해 읽고/쓸 수 있는 권한을 가진 “자격증”처럼 볼 수 있다.

이 관점에서 file descriptor 는 capability 이다.

- 어떤 파일에 접근할 수 있는 권한 자체를 의미
- 그 권한이 있어야만 read/write 가능

(3-2) 객체를 가리키는 포인터처럼 이해

또 다른 관점에서 파일 디스크립터는 “파일 객체를 가리키는 포인터”처럼 볼 수도 있다.
이 파일 객체를 통해 이후에는

```
- read()
- write()
```

같은 작업을 수행하게 된다.

즉, open() -> 파일 객체 생성 -> file descriptor 반환 -> 이후 read/write 로 접근

여담: creat() 시스템 콜

파일을 생성하는 방법은 creat()를 다음과 같이 호출하는 것이다.

```
int fd = creat("foo");
```

creat()는 O_CREAT | O_WRONLY | O_TRUNC 플래그를 사용하는 open()이라고 생각할 수 있다.

open()이 파일을 생성할 수 있기 때문에 creat()의 사용이 인기를 잃어가고 있다.

하지만 이것은 Unix 에서 역사적으로 특별한 위치를 갖고 있다.

Ken Thompson 에게 Unix 에서 재설계를 하고 싶은 부분이 있다면 어떤 것이 있느냐라는 질문을 했을 때 그는 이렇게 답했다.

“creat 의 철자에 e 를 더하겠다.”

[4] 파일의 읽기와 쓰기

파일이 존재한다면, 당연히 그 파일을 읽거나 쓰고 싶어진다.
먼저 이미 존재하는 파일을 읽는 과정부터 살펴보자.

커맨드 라인 환경에서는 cat 이라는 프로그램을 사용하면 파일 내용을 화면에 출력할 수 있다.

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

여기서 echo hello > foo 는 echo 의 출력을 파일 foo 로 리다이렉션(redirect)하여 문자열 "hello"를 파일에 저장하는 것이다.
그 다음 cat foo 를 실행하면 파일 내용을 화면에 출력한다.

(1) cat 은 어떻게 파일에 접근하는가?

그렇다면 cat 은 내부적으로 어떻게 파일 foo 에 접근할까?

이 과정을 이해하기 위해서는 프로그램이 실제로 어떤 시스템 콜을 호출하는지 추적해야 한다.

Linux 에서 strace 라는 도구를 사용하면 이를 확인할 수 있다. 다른 시스템에도 비슷한 도구가 있다.

```
- macOS: dtruss
- 일부 Unix 계열
```

이 도구들은 프로그램 실행 중 발생하는 모든 시스템 콜을 추적해서 보여준다.

(2) strace 로 본 cat 의 동작

다음은 cat foo 를 strace 로 분석한 결과이다.

```
prompt> strace cat foo
...
open("foo", O_RDONLY | O_LARGEFILE) = 3
read(3, "hello\n", 4096) = 6
write(1, "hello\n", 6) = 6
hello
read(3, "", 4096) = 0
close(3) = 0
...
prompt>
```

(2-1) open()

가장 먼저 cat 은 파일을 열기 위해 open()을 호출한다.

여기서 중요한 점은

- O_RDONLY : 읽기 전용으로 파일을 연다.(쓰기 불가)
- O_LARGEFILE: 64 비트 파일 오프셋을 사용하도록 설정

그리고 open()의 반환값은 3 이다.

보통 파일 디스크립터는 0, 1, 2 가 먼저 떠오르지만, 이 경우는 왜 3 일까?

이 이유는 이미 프로세스가 기본적으로 3 개의 파일을 열고 있기 때문이다.

- 0 : 표준 입력(STDIN)
- 1: 표준 출력(STDOUT)
- 2: 표준 에러(STDERR)

그래서 새로운 파일은 그 다음 번호인 3 을 받는다.

(2-2) read()

파일이 열리면 cat 은 read()를 반복적으로 호출하여 파일 내용을 읽는다.

read(3, buffer, 4096)

여기서 의미는

- 3: 파일 디스크립터(어떤 파일을 읽을지 지정)
- buffer: 읽을 데이터를 저장할 메모리 위치
- 4096: 최대 읽을 크기(4KB)

starce 결과에서 “helloWn”가 buffer 에 저장된 것으로 표시된다.

그리고 read()는 6 을 반환하는데, 이는

- “hello”(5 글자)
- 줄바꿈 문자Wn(1 개)

총 6 바이트이기 때문이다.

(2-3) write()

그 다음 cat 은 화면에 출력하기 위해 write()를 호출한다.

write(1, “helloWn”, 6)

여기서 중요한 점

- 파일 디스크립터 1: 표준 출력(화면)

즉, cat 은 읽은 내용을 stdout(화면)으로 출력하는 것이다.

cat 이 write 를 직접 호출할까?

가능성은 두 가지다.

- 최적화된 경우: cat 이 직접 write() 호출
- 일반적인 경우: printf() 같은 라이브러리 함수 사용

하지만 내부적으로 printf()도 결국 write() 시스템 콜로 연결된다.
즉, 최종 출력은 항상 write()를 통해 이루어진다.

(2-4) read()가 0 을 반환

파일을 끝까지 읽으면 더 이상 데이터가 없기 때문에:

```
read(3, ..., 4096) = 0
```

이 0 은 파일 끝(EOF, End Of File)을 의미한다.

(2-5) close()

마지막으로 cat 은 파일을 닫는다.

```
close(3)
```

이것으로 파일 디스크립터가 해제되고 작업이 종료된다.

(3) 파일 쓰기도 비슷한 구조

파일 쓰기 과정도 거의 동일하다.

1. open()으로 파일 열기
2. write()를 반복적으로 호출
3. 작업 완료 후 close()

파일이 크면 write()는 여러 번 호출될 수 있다.

(4) 예시: dd 명령어

파일 쓰기 과정을 확인하는 또 다른 방법:

```
dd if=foo of=bar
```

- if: input file
- of: output file

이것도 내부적으로 open->read->write->close 흐름을 따른다.

[5] 비순차적 읽기와 쓰기

지금까지 우리는 파일을 읽고 쓰는 과정을 살펴보았는데, 모든 접근은 순차적이었다. 즉, 파일의 처음부터 끝까지 차례대로 읽고, 쓸 때도 처음부터 끝까지 기록하는 방식이다.

하지만 경우에 따라서는 파일의 특정 위치(오프셋)에서부터 읽거나 쓰는 것이 더 유용할 때가 있다.

예를 들어, 문서의 인덱스를 만들거나, 특정 단어를 찾는 경우를 생각해보자.

이런 상황에서는 파일의 임의 위치에서 바로 읽기를 수행해야 한다.

(1) lseek() 시스템 콜

이러한 작업을 위해 lseek()라는 시스템 콜이 사용된다.

함수의 형태는 다음과 같다.

```
off_t lseek(int fd, off_t offset, int whence);
```

각 인자의 의미는 다음과 같다.

- 첫 번째 인자: 파일 디스크립터(어떤 파일을 대상으로 할지)
- 두 번째 인자: 이동할 위치(offset)
- 세 번째 인자: 기준 위치(whence)

(2) whence 의 의미

whence 는 기준점을 의미하며, 다음과 같은 값들을 가진다.

- SEEK_SET: 파일의 시작 위치를 기준으로 offset 만큼 이동
- SEEK_CUR: 현재 위치를 기준으로 offset 만큼 이동
- SEEK_END: 파일의 끝을 기준으로 offset 만큼 이동

(3) 파일 오프셋의 개념

운영체제는 open()으로 열린 각 파일에 대해 현재 위치(current offset)를 따로 관리한다.

이 오프셋은 “다음에 읽거나 쓸 위치”를 의미한다.

즉, 열린 파일이라는 개념에는 단순히 파일 정보뿐 아니라 현재 오프셋 상태도 포함되어 있다.

(4) 오프셋이 갱신되는 방식

오프셋은 두 가지 방법으로 변경된다.

read/write 에 의한 자동 갱신

파일에서 N바이트를 읽거나 쓰면: 현재 오프셋 + N이 되어 자동으로 다음 위치로 이동한다.
즉, 순차적 접근은 자동으로 오프셋이 증가하면서 이루어진다.

lseek()에 의한 명시적 변경

lseek()를 사용하면 프로그래머가 직접 오프셋을 원하는 위치로 변경할 수 있다.
즉, “다음 읽기/쓰기 위치를 강제로 이동시키는 것”

중요한 주의점

lseek()라는 이름 때문에 혼동하기 쉬운 점이 있다.
여기서 “seek”는 디스크의 물리적인 움직임(디스크 헤드 이동)을 의미하지 않는다.

즉, lseek()는 실제 디스크를 움직이는 것이 아니라 커널 내부에서 관리하는 “오프셋 변수”값을 바꾸는 것뿐이다.

(5) 실제 디스크 동작과의 관계

실제 디스크에서 헤드가 움직이는지는 별도의 문제이다.

I/O가 발생할 때:

- | |
|--|
| <ul style="list-style-type: none">- 현재 디스크 헤드 위치에 따라 이동이 필요할 수도 있고- 아닐 수도 있다. |
|--|

하지만 lseek() 자체는 단순히 “다음에 어디서 읽을지”만 바꾸는 논리적 작업이다.

[6] fsync()를 이용한 즉시 기록

write() 시스템 콜의 목적은 보통, 데이터를 가까운 미래에 영속 저장 장치(디스크)에 기록해달라고 요청하는 것이다.

하지만 성능상의 이유로 파일 시스템은 즉시 디스크에 기록하지 않는다.

대신 일정 시간 동안(예: 5 초, 30 초) 데이터를 메모리에 모아두는 버퍼링(buffering)을 수행한다.

그리고 일정 시간이 지나면, 모아둔 쓰기 요청들을 한 번에 디스크로 전달한다.

(1) write()의 한계

응용 프로그램 입장에서는 write()가 호출되면 마치 데이터가 즉시 디스크에 기록된 것처럼 보인다.

하지만 실제로는 그렇지 않을 수 있다.

예를 들어:

- write() 호출 완료
- 아직 디스크에는 기록되지 않음
- 그 사이에 시스템 크래시 발생

이 경우, 데이터가 유실될 수 있다.

(2) 더 강한 보장이 필요한 경우

일부 프로그램은 이러한 불확실성을 허용할 수 없다.

예를 들어:

- 데이터베이스(DBMS)의 복구 모듈

이러한 시스템은 write()가 끝났으면 반드시 디스크에 기록되어 있어야 한다는 강한 보장을 필요로 한다.

(3) fsync() 시스템 콜

이러한 요구를 위해 파일 시스템은 추가적인 제어 API 를 제공한다.

Unix에서는 fsync(int fd)가 그 역할을 한다.

fsync()를 호출하면:

- 해당 파일의 모든 dirty 데이터(수정되었지만 아직 디스크에 반영되지 않은 데이터)를 강제로 디스크에 기록한다.

그리고 모든 쓰기가 완료된 후에야 fsync()는 리턴한다.

사용 예제

다음은 fsync()를 사용하는 간단한 코드이다.

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
assert(fd > 1);

int rc = write(fd, buffer, size);
assert(rc == size);

rc = fsync(fd);
assert(rc == 0);
```

이 코드의 흐름은 다음과 같다.

1. 파일을 열고
2. 데이터를 쓰고
3. fsync()를 호출하여 즉시 디스크에 기록

fsync()가 성공적으로 리턴되면 데이터가 실제로 디스크에 안전하게 저장되었다고 보장받는다. 따라서 이후 작업을 안전하게 진행할 수 있다.

중요한 주의사항

하지만 이 코드만으로는 완벽히 보장이 되지 않을 수 있다. 특히, 파일이 새로 생성된 경우에는 추가적인 문제가 있다.

파일 자체뿐만 아니라:

- 그 파일이 속한 디렉터리 fsync()해야 한다.

왜냐하면:

- 파일 내용은 디스크에 기록되었더라도
- 디렉터리 정보(파일 이름 <-> inode 연결)가 아직 반영되지 않을 수 있기 때문이다.

(3) 디렉터리 fsync 의 중요성

따라서 안전하게 보장하려면

- 파일 -> fsync()
- 디렉터리 -> fsync()

두 가지를 모두 수행해야 한다.

이 과정을 통해 파일 디렉터리와 디렉터리 구조 모두 디스크에 확실히 기록된다.

(4) 현실적인 문제

이러한 세부 사항은 매우 중요하지만, 실제로는 자주 간과된다.

그 결과:

- 데이터 유실
- 파일 시스템 불일치

와 같은 응용 프로그램 수준의 버그가 발생하기도 한다.

[7] 파일 이름 변경

때로는 파일의 이름을 변경하는 작업이 매우 유용하다.

명령행에서는 mv 명령어를 사용하여 파일 이름을 바꿀 수 있다.

예를 들어:

```
prompt> mv foo bar
```

이 명령은 foo 라는 파일의 이름을 bar 로 변경한다.

(1) rename() 시스템 콜

이 동작을 내부적으로 살펴보면, mv 는 다음과 같은 시스템 콜을 사용한다.

```
rename(char *old, char *new);
```

- old: 기존 파일 이름

- new: 새로운 파일 이름

즉, 파일의 이름을 바꾸는 핵심은 rename()이다.

(2) rename()의 중요한 특징: 원자성(atomicity)

rename()의 가장 중요한 특징은 원자적(atomic)으로 동작한다는 것이다.

이 의미는 다음과 같다.

- 이름 변경 도중 시스템이 크래시가 발생하더라도 결판내 반드시 “이전 이름” 또는 “새 이름” 둘 중 하나만 남는다.

즉, 중간 상태(예: 파일이 사라지거나 이름이 깨진 상태)는 발생하지 않는다.

이러한 특성은 매우 중요하다.

특히 파일 상태를 안정적으로 갱신해야 하는 프로그램에서 필수적이다.

(3) 실제 활용 예: 파일 편집기

예를 들어, 텍스트 편집기(예: emacs)가 파일 중간에 내용을 추가한다고 가정해보자.

원래 파일 이름은 foo.txt 이다.

이 파일을 안전하게 수정하는 방법은 다음과 같다.

```
int fd = open("foo.txt.tmp", O_WRONLY | O_CREAT | O_TRUNC);
write(fd, buffer, size); // 새로운 내용 작성
fsync(fd);
close(fd);
rename("foo.txt.tmp", "foo.txt");
```

동작 과정

이 과정은 다음과 같은 순서로 진행된다.

1. 임시 파일(foo.txt.tmp)을 생성한다.
2. 수정된 내용을 임시 파일에 모두 작성한다.
3. fsync()를 호출하여 디스크에 확실히 기록한다.
4. 파일을 닫는다.
5. 마지막으로 rename()을 사용해 기존 파일을 새 파일로 교체한다.

이 방식의 목적은 파일이 항상 정상적인 상태를 유지하도록 보장하기 위함이다.

즉, 파일이 부분적으로만 수정된 상태로 남는 것을 방지한다.

[8] 파일 정보 추출

파일 시스템은 각 파일에 대한 다양한 정보를 함께 저장한다.
이러한 정보를 메타데이터(metadata)라고 부른다.

파일의 메타데이터를 확인하려면 stat() 또는 fstat() 시스템 콜을 사용한다.

- stat() : 파일 경로를 기반으로 정보 조회
- fstat(): 파일 디스크립터를 기반으로 정보 조회

(1) stat 구조체

stat()이 반환하는 정보는 다음과 같은 구조체 형태로 제공된다.

```
struct stat {  
    dev_t    st_dev;    /* 파일이 저장된 장치 ID */  
    ino_t    st_ino;    /* inode 번호 */  
    mode_t    st_mode;  /* 접근 권한 */  
    nlink_t   st_nlink; /* 하드 링크 수 */  
    uid_t    st_uid;    /* 소유자 사용자 ID */  
    gid_t    st_gid;    /* 소유자 그룹 ID */  
    dev_t    st_rdev;   /* 특수 파일일 경우 장치 ID */  
    off_t     st_size;  /* 파일 크기 (바이트 단위) */  
    blksize_t st_blksize; /* I/O 블록 크기 */  
    blkcnt_t  st_blocks; /* 할당된 블록 수 */  
    time_t    st_atime; /* 마지막 접근 시간 */  
    time_t    st_mtime; /* 마지막 수정 시간 */  
    time_t    st_ctime; /* 상태 변경 시간 */  
};
```

(2) 어떤 정보들이 있는가?

이 구조를 보면, 파일에 대해 매우 많은 정보가 저장되어 있다는 것을 알 수 있다.
대표적으로:

- 파일 크기(bytes)
- inode 번호(저수준 이름)
- 접근 권한
- 소유자 정보(user, group)
- 하드 링크 수
- 마지막 접근/수정/상태 변경 시간

(3) stat 명령어 예시

실제로는 stat 명령어를 통해 쉽게 확인할 수 있다.

```
prompt> echo hello > file  
prompt> stat file
```

출력 결과에는 다음과 같은 정보들이 포함된다.

- 파일 크기: 6bytes("helloWn")
- inode 번호
- 접근 권한(예: rw-r-----)
- 사용자 / 그룹 정보
- 마지막 접근 / 수정 시간

[9] 파일 삭제

지금까지 우리는 파일의 생성과 접근 방법을 살펴보았다.
그렇다면 파일은 어떻게 삭제할까?

Unix 를 사용해 본 사람이라면 rm 명령어를 떠올릴 것이다.
하지만 이 rm 은 내부적으로 어떤 시스템 콜을 사용할까?

(1) strace 로 확인

이 질문을 확인하기 위해 strace 를 사용해보자.

```
prompt> strace rm foo
...
unlink("foo") = 0
...
```

불필용한 출력은 제거하고 보면, 핵심은 다음과 같다.

- unlink("foo")

즉, 파일 삭제는 unlink() 시스템 콜을 통해 이루어진다.

(2) unlink()의 동작

unlink()는 삭제할 파일의 이름을 인자로 받고, 성공하면 0 을 반환한다.

그런데 왜 이름이 unlink 일까?

여기서 중요한 의문이 생긴다.

- 왜 “delete”나 “remove”가 아니라 “unlink(연결을 끊다)”일까?

이 질문을 이해하려면 파일 자체뿐 아니라, 디렉터리의 역할을 함께 이해해야 한다.

(3) 핵심 개념

앞에서 배운 것처럼:

- 파일은 inode 로 관리된다.
- 디렉터리는 “이름 <-> inode 번호”를 연결하는 구조이다.

즉, 파일 이름은 실제 파일이 아니라 inode 를 가리키는 “링크(link)”일 뿐이다.

(4) unlink 의 의미

따라서 unlink()가 하는 일은:

- 디렉터리에서 “파일 이름과 inode 를 연결하는 항목”을 제거하는 것

즉,

- “foo”라는 이름과 inode 사이의 연결을 끊는다.
- 그래서 “unlink”라는 이름이 붙은

(5) 중요한 포인트

파일이 바로 삭제되는 것이 아니라,

1. 디렉터리에서 이름이 제거됨
2. inode 의 링크 수(nlink)가 감소
3. 링크 수가 0 이 되면 -> 실제 데이터 블록이 해제됨(진짜 삭제)

[10] 디렉터리 생성

디렉터리와 관련된 시스템 콜들은 디렉터리를 생성, 읽기, 삭제하는 역할을 한다.
하지만 중요한 점이 있다.

– 디렉터리는 직접 수정(write)할 수 없다.

디렉터리는 파일 시스템의 메타데이터로 취급되기 때문에, 항상 간접적으로만 변경된다.

예를 들어,

- 파일 생성
- 디렉터리 생성
- 파일 삭제

이런 작업을 통해서만 디렉터리 내용이 변경된다.

이러한 방식으로 파일 시스템은 디렉터리의 내용이 항상 일관성 있게 유지되도록 보장한다.

(1) mkdir() 시스템 콜

디렉터리를 생성하기 위한 시스템 콜은 mkdir()이다.

명령행에서 mkdir 프로그램을 실행하면 내부적으로 이 시스템 콜이 호출된다.

예를 들어:

```
prompt> strace mkdir foo
...
mkdir("foo", 0777) = 0
...
prompt>
```

– “foo”: 생성할 디렉터리 이름

– 0777: 디렉터리 접근 권한(읽기/쓰기/실행)

(2) 디렉터리는 완전히 빈가?

디렉터리를 처음 생성하면 겉보기에는 비어 있는 것처럼 보인다.

하지만 실제로는 기본적으로 두 개의 항목이 존재한다.

기본 항목: “.” 과 “..”

모든 디렉터리에는 다음 두 가지 항목이 포함된다.

- “.”(dot): 자기 자신을 가리킴
- “..”(dot-dot): 부모 디렉터리를 가리킴

즉, 디렉터리는 항상 자신과 부모를 참조하는 구조를 가진다.

확인 방법

이 항목들은 기본적으로 숨겨져 있기 때문에, 일반 `ls` 로는 보이지 않는다.
다음과 같이 `-a` 옵션을 사용하면 확인할 수 있다.

```
prompt> ls -a
```

```
...
```

또는 자세한 정보까지 보려면

```
prompt> ls -al
```

이 명령을 통해

- 현재 디렉터리(.)
- 부모 디렉터리(..)

를 확인할 수 있다.

[11] 디렉터리 읽기

이제 디렉터리를 생성했으니, 디렉터리의 내용을 읽어보자.
사실 우리가 자주 사용하는 ls 프로그램이 바로 이 작업을 수행한다.

여기서는 ls 와 비슷한 프로그램을 직접 만들어, 디렉터리 읽기가 어떻게 이루어지는지 살펴본다.

(1) 디렉터리 읽기용 인터페이스

디렉터리를 여는 작업은 일반 파일을 여는 open()과는 다르며,
다음과 같은 전용 함수들을 사용한다.

- opendir(): 디렉터리 열기
- readdir(): 디렉터리 항목 하나씩 읽기
- closedir(): 디렉터리 닫기

(2) 예제 코드

다음은 디렉터리의 내용을 출력하는 간단한 프로그램이다.

```
int main(int argc, char *argv[]) {
    DIR *dp = opendir(".");
    assert(dp != NULL);

    struct dirent *d;
    while ((d = readdir(dp)) != NULL) {
        printf("%d %s\n", (int)d->d_ino, d->d_name);
    }

    closedir(dp);
    return 0;
}
```

코드 흐름 설명

1. opendir(".") : 현재 디렉터리를 연다.
2. readdir() 반복 호출: 디렉터리 안의 항목을 하나씩 읽는다.
3. 각 항목에 대해
 - inode 번호(d_ino)
 - 파일 이름(d_name)을 출력한다.
4. 모든 항목을 읽으면 closedir()로 종료

(3) dirent 구조체

readdir()는 struct dirent 형태의 데이터를 반환한다.

```
struct dirent {
    char d_name[256]; /* 파일 이름 */
    ino_t d_ino;      /* inode 번호 */
    off_t d_off;      /* 다음 항목 위치 */
    unsigned short d_reclen; /* 레코드 길이 */
    unsigned char d_type; /* 파일 타입 */
};
```

(4) 디렉터리에 저장된 정보

디렉터리는 생각보다 많은 정보를 담고 있지 않다.

핵심은 “파일 이름 <-> inode 번호”의 매핑
그 외에는 몇 가지 부가 정보만 포함된다.

추가 정보는 어떻게 얻는가?

파일 크기, 권한 같은 자세한 정보는 디렉터리에 저장되어 있지 않다.
따라서 프로그램은 각 파일에 대해 stat()을 호출해서
추가적인 메타데이터를 가져와야 한다.

(5) ls -l의 동작

예를 들어,

- ls: 단순히 이름 목록만 출력(readdir 만 사용)
- ls -l: 상세 정보 출력
 - 각 파일마다 stat() 호출

확인 방법

이 차이는 strace 로 확인할 수 있다.

- ls 실행 -> readdir() 중심
- ls -l 실행 -> stat() 호출이 추가 됨

[12] 디렉터리 삭제하기

마지막으로 디렉터리를 삭제하는 방법을 살펴보자.

디렉터리는 `rmdir()` 시스템 콜을 사용하여 삭제할 수 있다.

명령행에서 사용하는 `rmdir` 프로그램도 내부적으로 이 시스템 콜을 호출한다.

(1) 파일 삭제와의 차이

디렉터리 삭제는 파일 삭제와는 다른 점이 있다.

– 디렉터리는 하나의 명령으로 매우 많은 데이터를 포함할 수 있기 때문에 더 위험한 작업이다.

즉, 디렉터리 하나를 지우면 그 안에 있는 모든 파일과 하위 디렉터리까지 영향을 줄 수 있다.

(2) `rmdir()`의 조건

이러한 위험성을 줄이기 위해 `rmdir()`에는 중요한 조건이 있다.

– 삭제하려는 디렉터리는 반드시 “비어 있어야 한다”

여기서 “비어 있다”는 의미는:

- “.”(자기 자신)
- “..”(부모 디렉터리)

이 두 항목만 존재해야 한다는 뜻이다.

(3) 실패 조건

만약 디렉터리 안에:

- 파일이 있거나
- 다른 디렉터리가 포함되어 있다면

`rmdir()` 호출은 실패한다.

[13] 하드 링크

파일 삭제 시 왜 `unlink()`라는 이름을 사용하는지 이해하기 위해,
파일 시스템 트리에 항목을 추가하는 또 다른 시스템 콜인 `link()`를 살펴보자.

(1) `link()` 시스템 콜

`link()`는 두 개의 인자를 받는다.

- 원래 파일의 경로명
- 새로 만들 이름(경로명)

이 시스템 콜은 기존 파일에 새로운 이름을 “연결(link)”하여,
같은 파일에 접근할 수 있는 또 다른 경로를 만들어준다.

명령행에서는 `ln` 프로그램이 이 역할을 수행한다.

(2) 예제

```
prompt> echo hello > file  
prompt> cat file  
hello
```

```
prompt> ln file file2  
prompt> cat file2  
hello
```

- “file”이라는 파일을 생성하고
 - `ln file file2`로 새로운 이름을 만든다.
- 이제 “file”과 “file2” 둘 다 같은 내용을 보여준다.

(3) 실제로 일어나는 일

중요한 점은 다음이다.

- 파일이 복사되는 것이 아니다.

`link()`는 단순히

- 디렉터리에 새로운 항목을 추가하고
- 그 항목이 기존 파일과 같은 inode 번호를 가리키도록 한다.

즉, 하나의 파일(inode)에 대해 여러 개의 이름이 존재하게 된다.

(4) inode 확인

이를 확인하기 위해 `ls -i` 를 사용하면:

```
prompt> ls -i file file2
67158084 file
67158084 file2
```

두 파일이 동일한 inode 번호를 가진다는 것을 알 수 있다.

(5) unlink()의 의미 다시 보기

이제 `unlink()`라는 이름의 의미가 분명해진다.

파일을 생성할 때는 사실 두 가지 일이 일어난다.

1. inode 생성(파일 정보와 데이터 관리)
2. 디렉터리에 이름을 추가하여 inode 와 연결

(6) 하드 링크 이후 상태

하드 링크를 만들면

- “file”과 “file2”는 서로 다른 파일이 아니다.
- 둘 다 동일한 inode 를 가리키는 “이름”일 뿐이다.

즉

- 파일 이름 ≠ 파일 자체
- 파일 이름 = inode 를 가리키는 링크

(7) unlink()의 동작

파일을 삭제할 때 `unlink()`가 수행하는 일은

1. 디렉터리에서 이름을 제거
2. inode 의 참조 횟수(link count)감소

중요한 예제

```
prompt> rm file
prompt> cat file2
hello
```

file 을 삭제했지만 file2 는 여전히 정상적으로 동작한다.

이유는 inode 는 아직 다른 이름(file2)에 의해 참조되고 있기 때문이다.

(8) 참조 횟수(link count)

각 inode 는 참조 횟수(link count)를 가진다.

이 값은 해당 inode 를 가리키는 이름의 개수를 의미한다.

(9) 삭제의 진짜 의미

unlink()가 호출되면 참조 횟수가 1 감소한다.

그리고

- 참조횟수가 0 이 되면 inode 와 데이터 블록이 해제된다.
- 즉 진짜 삭제 발생

stat 으로 확인

```
prompt> stat file  
Inode: 67158084 Links: 1
```

```
prompt> ln file file2  
prompt> stat file  
Inode: 67158084 Links: 2
```

```
prompt> ln file2 file3  
prompt> stat file  
Inode: 67158084 Links: 3
```

링크가 추가될수록 Links 값이 증가한다.

[14] 심볼릭 링크

또 다른 매우 유용한 링크의 종류가 있는데, 이를 심볼릭 링크(symbolic link) 또는 소프트 링크(soft link)라고 부른다.

(1) 왜 심볼릭 링크가 필요한가?

하드 링크에는 몇 가지 중요한 제한이 있다.

- 디렉터리에는 하드 링크를 만들 수 없다.
 - 디렉터리 트리에 순환 구조가 생길 수 있기 때문
- 다른 파일 시스템(디스크 파티션)에 있는 파일에는 링크를 만들 수 없다.
 - inode 번호는 하나의 파일 시스템 내에서만 유효하기 때문

이러한 한계를 해결하기 위해 심볼릭 링크가 등장하였다.

(2) 심볼릭 링크 생성

심볼릭 링크도 ln 명령어로 만들 수 있으며, -s 옵션을 사용한다.

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
```

이제 file 뿐 아니라 file2 라는 이름으로도 파일에 접근할 수 있다.

(3) 하드 링크와의 차이

겉보기에는 하드 링크와 비슷하지만, 내부적으로는 완전히 다르다.

가장 중요한 차이는 심볼릭 링크는 “독립된 파일”이다.

즉,

- 하드 링크: 같은 inode 를 공유
- 심볼릭 링크: 다른 inode 를 가진 별도의 파일

(4) 파일 타입 확인

stat 으로 확인하면 차이를 알 수 있다.

```
prompt> stat file
... regular file ...

prompt> stat file2
... symbolic link ...
```

또는 ls -l 로 보면:

- -: 일반 파일
- d: 디렉터리
- l: 심볼릭 링크

(5) 심볼릭 링크의 내용

심볼릭 링크는 실제 데이터가 아니라 “연결 대상의 경로명”을 저장하는 파일이다.

예:

```
lrwxrwxrwx ... file2 -> file
```

여기서 file2 는 “file”이라는 경로 문자열을 저장하고 있다.

(6) 크기의 의미

심볼릭 링크의 크기는 연결된 파일의 크기가 아니라 경로 문자열의 길이이다.

```
file2 -> file (길이 4 bytes)
```

더 긴 이름이면 크기도 커진다.

```
ln -s longerfilename file3
```

“longerfilename” 길이만큼 크기가 증가

(7) 중요한 문제: dangling reference

심볼릭 링크는 경로를 저장하기 때문에 문제가 발생할 수 있다.

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
```

```
prompt> rm file
prompt> cat file2
cat: file2: No such file or directory
```

- 원본 파일(file)이 삭제됨
- 하지만 file2 는 여전히 존재

하지만 가리키는 대상이 없어서 사용할 수 없음
이를 dangling referece(끊어진 참조)라고 한다.

[15] 하드 링크 vs 심볼릭 링크

(1) 구조적 차이

- 하드 링크: 같은 inode 공유(완전히 같은 파일)
- 심볼릭 링크: 다른 inode, 경로만 저장(파일을 가리키는 링크)

(2) 핵심 비교

구분	하드 링크	심볼릭 링크
inode	동일	다름
본질	파일 자체	파일을 가리키는 경로
원본 삭제	영향 없음	링크 깨짐
파일 시스템	같은 곳만 가능	다른 곳도 가능
디렉토리	거의 불가	가능

(3) 예시

하드 링크

- A.txt, B.txt 가 있으면
 - 둘 다 완전히 동일한 파일
 - 하나 지워도 다른 하나는 그대로

심볼릭 링크

- shortcut.txt -> A.txt 를 가리킴
 - A.txt 삭제하면
 - shortcut.txt 는 고장난 링크

[16] 파일 시스템 생성과 마운트

지금까지 파일, 디렉터리, 그리고 다양한 링크들을 다루는 기본 인터페이스를 살펴보았다. 이제 마지막으로 중요한 질문이 남아있다.

– 여러 개의 파일 시스템이 있을 때, 이를 어떻게 하나의 디렉터리 구조로 통합할까?

(1) 여러 파일 시스템 → 하나의 트리

유닉스(또는 리눅스) 시스템에서는:

– 여러 개의 디스크 파티션(파일 시스템)이 모여 하나의 큰 디렉터리 트리를 구성한다.

이것은 두 단계로 이루어진다.

1. 파일 시스템 생성
2. 파일 시스템 마운트

파일 시스템 생성: mkfs

대부분의 시스템에서는 mkfs(make filesystem)라는 도구를 제공한다.

사용 방식

- 장치 이름(예: /dev/sda1)
- 파일 시스템 타입(예: EXT3)

을 전달하면, 해당 파티션에 새로운 파일 시스템을 생성한다.

생성 결과

파일 시스템이 생성되면:

- 자체적인 디렉터리 구조를 가짐
- 초기 상태에서는 비어 있음
- 루트 디렉터리(/)만 존재

즉, 독립적인 “작은 파일 시스템 세계”가 하나 만들어진다.

(2) 마운트(mount)

이제 생성된 파일 시스템을 실제로 사용하려면 기존 디렉터리 트리에 연결해야 한다.

이 과정을 마운트(mount)라고 한다.

마운트의 개념

마운트는 다음과 같이 동작한다.

1. 기존 디렉터리 중 하나를 선택(마운트 지점)
2. 새로운 파일 시스템을 그 위치에 “붙인다”

예제

```
prompt> mount -t ext3 /dev/sda1 /home/users
```

- /dev/sda1 : 마운트할 파일 시스템
- /home/users: 마운트 지점

마운트 이후 변화

마운트가 완료되면

- /home/users 는 더 이상 기존 디렉터리가 아니라
- 새 파일 시스템의 루트를 가리키게 된다.

접근 방식

예를 들어, 새 파일 시스템 내부 구조가

```
/
|
|—— a
|   |—— foo
|—— b
|   |—— foo
```

이러면

- /home/users/a
- /home/users/b
- /home/users/a/foo
- /home/users/b/foo

와 같은 경로로 접근할 수 있다.

(3) mount 명령으로 확인

현재 시스템의 마운트 상태는 다음과 같이 확인할 수 있다.

```
prompt> mount
```

예시 출력:

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
tmpfs on /dev/shm type tmpfs (rw)
```

다양한 파일 시스템

- ext3: 일반 디스크 파일 시스템
- proc: 프로세스 정보 제공
- tmpfs: 메모리 기반 파일 시스템
- AFS: 분산 파일 시스템

(4) 최종 구조

여러 개의 파일 시스템이 하나의 루트(/)아래에 통합되어 동작한다.

즉 정리하자면,

```
prompt> mount -t ext3 /dev/sda1 /home/users
```

/dev/sda1

- 물리 디스크(HDD/SDD)의 1 번 파티션
- 리눅스에서는 이걸 파일처럼 표현한 것
- 즉, 하드웨어를 다루기 위한 창구

그래서 정확히 말하면 “물리 위치”라기보다는 “장치를 나타내는 특수 파일”이다.

/home/users

- 그냥 디렉터리(폴더)
- 마운트하면 그 위치에서 /dev/sda1 의 파일 내용이 보인다.

즉, 사용자가 접근하는 “논리적 경로”이다.

[1-2] 파일 시스템 구현

핵심 질문: 어떻게 간단한 파일 시스템을 만들 것인가

간단한 파일 시스템을 어떻게 만들 수 있을까?

1. 디스크 위에는 어떤 자료구조가 필요한가?
2. 그러한 자료 구조는 어떤 정보를 추적해야 하는가?
3. 그 자료 구조들을 어떻게 접근되어야 하는가?

이번 장에서는 vsfs(Very Simple File System)라고 하는 간단한 파일 시스템 구현에 대해 소개하겠다.

이 파일 시스템은 Unix 파일 시스템을 단순화한 것으로, 디스크 자료 구조(on-disk structure)와 접근 방법, 그리고 다양한 파일 시스템들의 정책들을 소개하기 위해 제작되었다.

파일 시스템은 순수한 소프트웨어이다.

이점이 이 책의 앞부분에서 다룬 CPU 가상화, 메모리 가상화 부분과 다른점이다.

CPU 가상화나 메모리 가상화에서는 하드웨어가 필요하다.

특권 모드로 변환을 위한 명령어, 페이징을 위한 명령어 등이 그 예이다.

파일 시스템에서는 성능 개선을 위해 하드웨어를 추가하지 않을 것이다.

(물론, 잘 동작하는 파일 시스템을 만들려면 장치 특성을 반영해야 한다.)

파일 시스템을 구현하는 다양한 방법이 존재하기 때문에 문자 그대로 AFS(Andrew 파일 시스템)부터 시작하여 ZFS(Sun의 Zettabyte(10^{21})파일 시스템)까지 매우 다양한 파일 시스템들이 개발되었다.

이 모든 파일 시스템들은 서로 다른 자료 구조를 갖고 있으며 각각은 장단점이 있다.

사례 연구를 통해 파일 시스템을 학습할 것이다.

먼저 이번 장에서 간단한 파일 시스템(vsfs)을 사용하여 개념을 소개하고, 파일 시스템에 대한 실제 사례들을 다루면서 현실에서는 어떻게 동작하는지 이해해 보도록 하겠다.

[1] 생각하는 방법

파일 시스템에 대해 학습할 때, 두 가지 측면에서 접근할 것을 권장한다.

그 두 측면을 다 이해하게 되면 파일 시스템이 기본적으로 어떻게 동작하는 지 이해하게 될 것이다.

첫 번째는 파일 시스템의 자료 구조이다.

즉, 파일 시스템이 자신의 데이터와 메타데이터를 관리하기 위해 디스크 상에 어떤 종류의 자료 구조가 있어야 하겠는가?

보게될 첫 파일 시스템(vsfs 를 포함하여)은 블록과 다른 객체들을 배열과 같은 간단한 자료구조로 표현하지만, SGI 의 XFS 와 같은 파일 시스템들은 좀 더 복잡한 트리 기반의 자료 구조를 사용한다.

두 번째는 접근 방법(access method)이다.

- 프로세스가 호출하는 open(), read(), write() 등의 명령들은 파일 시스템의 자료 구조와 어떤 관련이 있는가?
- 특정 시스템 콜을 실행할 때 어떤 자료 구조들이 읽히는가?
- 어떤 것들이 쓰이는가?
- 이 모든 과정이 얼마나 효율적으로 동작하는가?

파일 시스템의 자료 구조와 접근 방법을 이해하였다면, 실제 동작 방식에 대한 개념 모델을 제대로 정립할 수 있다.

그것이 시스템적 사고 방식의 핵심이다.

우리가 파일 시스템을 처음으로 구현하려 할 때, 개념적 모델을 잘 만들려고 노력하기 바란다.

여담: 파일 시스템의 개념적 모델

이전에도 언급했듯이 시스템을 배우려고 할 때 실제로 하는 일은 모델을 잘 만드는 것이다. 파일 시스템을 이해하기 위한 개념적 모델에는 다음과 같은 질문들이 자연스럽게 포함된다.

- 디스크 상의 어떤 자료 구조가 파일 시스템의 데이터와 메타데이터를 저장하는가?
- 프로세스가 파일을 열면 무슨 일이 일어나는가?
- 읽기 또는 쓰기를 할 때에는 디스크 상의 어떤 자료 구조가 접근되는가?

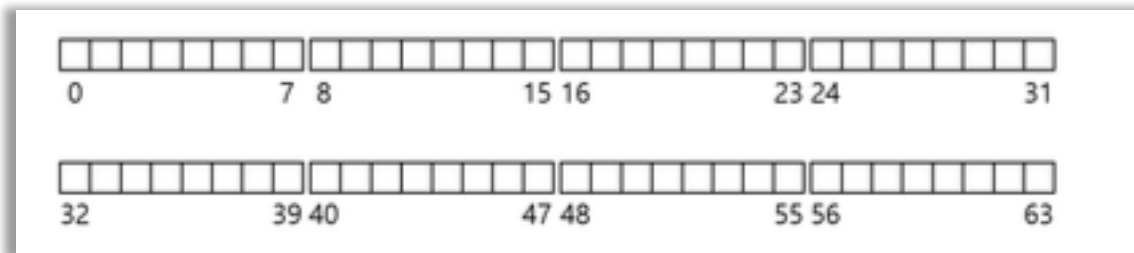
파일 시스템의 코드를 읽어서 구체적인 부분을 이해하려고 노력하는 대신(물론, 이 또한 유용하기는 하지만), 개념적 모델을 확장시키려고 노력하면 일어나는 현상에 대해 더 쉽게 이해할 수 있다.

[2] 전체 구성

이제 vsfs 파일 시스템의 자료 구조에 대해 디스크 상의 전체적인 구성을 개발해보자.
가장 먼저 해야 할 일은 디스크를 블록(block)들로 나누는 일이다.
간단한 파일 시스템은 단일 블록 크기만 사용하기 때문에 여기서도 그렇게 하겠다.
일반적으로 사용되는 크기인 4KB 를 사용하겠다.

우리가 파일 시스템을 만들려는 디스크 파티션 구조는 단순하다.
4KB 블록들로 나열되어 있다.

N 개의 4KB 블록의 크기를 갖는 파티션에서 블록은 0 부터 N-1 까지의 주소를 갖는다.
블록이 64 개 있는 작은 디스크를 가정하자.



파일 시스템을 생성하기 위해 이들 블록에 무엇을 저장할지 생각해 보자.

가장 먼저 떠오르는 것은 사용자 데이터이다.

사실 파일 시스템의 대부분의 공간은 사용자 데이터로 이루어져 있다.(또한 그래야 한다.)

사용자 데이터가 있는 디스크 공간을 데이터 영역(data region)이라고 하자.

간단하게 64 개의 디스크 블록 중 마지막 56 개의 블록처럼 디스크의 일정 부분을 데이터 영역으로 확보하자.



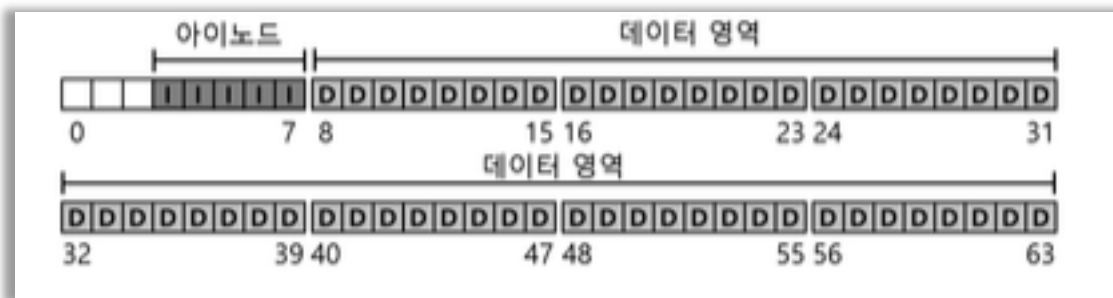
앞에서 배운 것처럼 파일 시스템은 각 파일에 대한 정보를 관리한다.
 그 정보가 메타데이터(metadata)의 핵심이다.
 메타데이터에는 다음이 포함된다.

- 파일을 구성하는 데이터 블록(데이터 영역 내)
- 파일 크기
- 소유자
- 접근 권한
- 접근 / 변경 시간

파일 시스템은 이 정보를 보통 아이노드(inode)라는 자료 구조에 저장한다.

아이노드들의 저장을 위해 디스크 공간이 필요하다.
 이 영역을 아이노드 테이블(inode table)이라고 한다.
 아이노드 테이블에는 아이노드들이 배열 형태로 저장된다.

- 전체 64 개 블록 중 5 개 블록을 아이노드 저장용으로 할당



아이노드는 일반적으로 128~256bytes 정도이다.
 아이노드 크기를 256bytes 로 가정하면

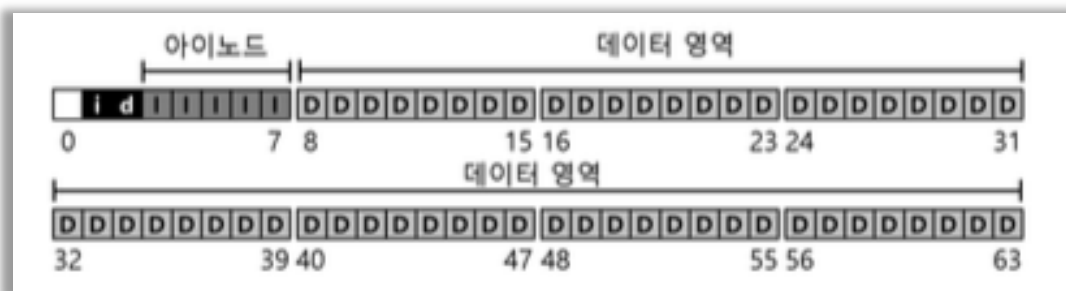
- 1 블록(4KB) = 16 개 아이노드
- 총 5 블록 → 80 개 아이노드

즉, 이 파일 시스템에서는 최대 80 개의 파일 생성 가능하다.
 (디스크가 커지면 아이노드 테이블도 커져 생성 가능한 파일 수가 증가한다.)

우리의 예제 파일 시스템에는 다음이 존재한다.

- 데이터 블록(D)
- 아이노드 블록(I)

하지만 아직 완전하지 않다.
 필요한 것은 사용 여부를 관리하는 구조이다.



할당 구조(Allocation Structure)

각 블록이 사용 중인지 아닌지를 표현하는 구조가 필요하다.

방법 예시:

- 프리 리스트(free list) → 사용하지 않는 블록들을 연결 리스트로 관리
- 비트맵(bitmap) → vsfs 에서 사용

vsfs 에서는 다음을 사용한다.

- 데이터 비트맵(data bitmap) → 데이터 블록 사용 여부 관리
- 아이노드 비트맵(inode bitmap) → 아이노드 사용 여부 관리

비트맵은 비트 배열이며:

- 1 → 사용 중
- 0 → 사용 안함

우리의 작은 파일 시스템에서는 4KB 비트맵은 과하지만, 이를 위해 그대로 사용한다.

- 데이터 블록: 56 개
- 아이노드: 80 개

(실제로는 136 비트면 충분하지만 단순화를 위해 4KB 사용)

슈퍼블록(Superblock)

남은 1 개 블록은 슈퍼블록(superblock)이다.(s)

슈퍼블록은 파일 시스템 전체 정보를 담는다.

- 아이노드 개수(80 개)
- 데이터 블록 개수(56 개)
- 아이노드 테이블 시작 위치(예: 블록 3)
- 파일 시스템 식별 정보(매직 넘버, vsfs)

(파일 시스템이 깨지는 경우 → 슈퍼블록 손상)

그래서 보통 복사본을 여러 개 저장한다.



파일 시스템 마운트 과정

파일 시스템을 마운트할 때:

1. 운영체제가 슈퍼블록을 먼저 읽음
2. 파일 시스템 구조 초기화
3. 파일 시스템 트리에 연결(mount)

이 과정을 통해

- 디스크 볼륨 내 파일 접근 가능
- 필요한 자료 구조 위치 파악 가능

정리를 하자면

- 블록 단위(4KB)
- 구성 요소:
 - Superblock(S)
 - Inode bitmap
 - Data bitmap
 - Inode table(I)
 - Data region(D)

[3] 파일 구성: 아이노드

파일 시스템의 디스크 자료 구조 중 가장 중요한 것은 아이노드(inode)이다.

거의 모든 파일 시스템들이 비슷한 구조를 갖고 있다.

아이노드는 인덱스 노드(index node)의 줄임말로써 역사적으로는 Unix 의 창시자인 Ken Thompson 이 처음 사용하였다.

이 노드들은 원래 배열로 되어 있는데, 각 배열은 특정 아이노드를 접근하기 위해 탐색된다.

여담: 자료 구조 - 아이노드

파일 메타데이터를 저장하기 위한 자료 구조로서 많은 파일 시스템들이 아이노드(inode)라는 이름을 사용한다.

이 자료 구조는 다음 정보를 포함한다.

- 파일 크기
- 접근 권한
- 데이터 블록의 위치 정보

이 이름은 최소한 Unix 초기 시절부터 사용되었으며, 인덱스 노드(index node)의 약어이다. 디스크 상의 아이노드 배열에서 인덱스(번호)로 접근하던 것에서 유래하였다.

아이노드 설계는 파일 시스템 설계의 핵심이다. 대부분의 현대 시스템들은 모든 파일에 대해 유사한 구조를 사용한다. 다만 이름은 다를 수 있다.(dnode, fnode 등)

계산 공식

$$\text{blk} = (\text{inumber} * \text{sizeof}(\text{inode_t})) / \text{blockSize}$$
$$\text{sector} = ((\text{blk} * \text{blockSize}) + \text{inodeStartAddr}) / \text{sectorSize}$$

아이노드가 담는 정보(메타데이터)

아이노드는 파일에 대한 모든 정보를 담고 있다.

- 파일 종류(일반 파일, 디렉터리 등)
- 파일 크기
- 할당된 블록 수
- 보호 정보(소유자, 접근 권한 등)
- 시간 정보
- 데이터 블록 위치

이러한 정보를 메타데이터(metadata)라고 한다.(사용자 데이터가 아닌 모든 정보)

데이터 블록 위치 표현

아이노드 설계에서 가장 중요한 결정 중 하나는
데이터 블록의 위치를 어떻게 표현할 것인가이다.

(1) 직접 포인터(Direct Pointer)

- 아이노드 내부에 여러 개 존재
- 각 포인터 → 데이터 블록 1 개

한계:

$$\text{파일 크기} = \text{포인터 개수} \times \text{블록 크기}$$

파일 크기에 제한이 생김

멀티 레벨 인덱스

큰 파일을 지원하기 위해서 파일 시스템 개발자들은 아이노드 내에 다른 자료 구조를
추가해야 했다.

일반적으로 사용되는 방법 중 하나는 간접 포인터(indirect pointer)라는 특수한 포인터를
사용하는 것이다.

(2) 간접 포인터(Indirect Pointer)

- 데이터 블록이 아니라 → 포인터들을 담은 블록을 가리킴

구성:

- 직접 포인터: 12 개
- 간접 포인터: 1 개

블록 = 4KB, 포인터 = 4B → 1024 개 포인터 가능

$$\text{최대 파일 크기} = (12 + 1024) \times 4\text{KB} = 4144\text{KB}$$

4144KB는 사실 그리 큰 파일이 아니다.

더 큰 파일을 저장하고 싶을 수도 있다.

그 경우 아이노드에 이중 간접 포인터(double indirect pointer)를 추가한다.

아이노드

├─ 직접 포인터 12 개 → 데이터 블록

└─ 간접 포인터 1 개 → [포인터 1024 개]

└─ 데이터 블록

└─ 데이터 블록

└─ ...

(3) 이중 간접 포인터(Double Indirect)

- 간접 포인터들을 가리킴

$4KB \times 1024 \times 1024 \approx 4GB$

(4) 삼중 간접 포인터(Triple Indirect)

- 더 큰 파일 지원 가능

이제까지 설명한 것을 종합하면, 디스크 블록들은 일종의 트리 형태로 구성되어 하나의 파일을 이룬다.

약간 한쪽으로 치우쳐진 형태의 트리다.

이러한 구성 방식을 멀티 레벨 인덱스 기법이라 한다.

열두 개의 직접 포인터와 하나의 간접 블록과 이중 간접 블록을 사용하는 예를 살펴보자.

블록 크기가 4KB 이고 각 포인터의 크기가 4 바이트라고 하자.

이 구조에서 파일은 4GB 가 조금 넘는 크기를 가질 수 있다.

팁: 익스텐트 기반의 접근법

포인터를 사용하는 대신 익스텐트(extent) 방식을 사용할 수도 있다.

익스텐트는 단순히 디스크 포인터와 길이(블록 단위)로 이루어진다.

파일의 모든 블록에 대해서 포인터를 써야 하는 대신, 디스크 상의 한 파일의 위치를 가리키기 위해서 하나의 포인터와 길이만 표현하면 된다.

디스크 상에 충분히 크고 연속적인 비어 있는 공간을 찾는 것이 어려울 수 있기 때문에 하나의 익스텐트만을 갖는 것은 제한적일 수 있다.

그렇기 때문에 익스텐트 기반의 파일 시스템은 하나 이상의 익스텐트를 갖는 것을 허용하여 파일을 할당할 때 좀 더 자유도가 높아질 수 있도록 한다.

두 기법을 비교하면 포인터 기반의 접근법은 자유도가 높지만, 각 파일당 많은 양의 메타데이터를 사용한다(특히 큰 파일들의 경우)

익스텐트 기반의 접근법은 자유도가 낮은 대신 좀 더 집약되어 있다.

특히 디스크에 여유 공간이 충분히 있고 파일들을 연속적으로 배치할 수 있을 때 잘 동작한다.

일반적으로 사용되는 파일 시스템인 Linux 의 ext2 와 ext3, NetApp 의 WAFL 을 포함하여 많은 파일 시스템들이 멀티 레벨 인덱스를 사용한다.

SGI 의 XFS 와 Linux 의 ext4 와 같은 파일 시스템들은 간단한 포인터를 사용하는 대신 익스텐트를 사용한다.

즉, 멀티레벨 인덱스 방식은 포인터를 여러 단계로 나눠서 파일 크기를 확장하는 방식이다.

재미 있는 사실은 트리의 형태가 매우 편한적이라는 것이다.

파일의 시작 부분을 이루는 블록들은 한 번의 포인터로 접근이 가능하다.

큰 파일의 경우, 파일의 끝부분에 있는 블록들은 포인터를 세 번 따라가야 실제 블록을 읽을 수 있다.

왜 이렇게 했을까? 실수일까? 실수가 아니다.

오랜 고민과 연구의 결과이다.

많은 연구자들이 파일 시스템 사용 형태를 분석해서 발견한 아주 중요한 사실이 있다.

대부분의 파일 크기는 작다는 것이다.

비대칭적 트리 형태를 채용한 이유는 이러한 특성을 반영한 것이다.

대부분의 파일들이 작다면, 작은 파일을 빠르게 읽고 쓸 수 있도록 파일 구조를 설계해야 한다.

vsfs 의 아이노드는 첫 12 개의 블록들을 빠르게 읽을 수 있도록 직접 포인터(대체적으로 12 개)를 갖고 있다.

큰 파일들을 위해서 하나(또는 그 이상)의 간접 블록이 필요하다.

파일은 위에서 다룬 방법 외에도 다양한 방식으로 구성할 수 있다.

아이노드는 하나의 자료 구조 일뿐이다.

데이터와 관련 내용을 효율적으로 읽고 쓸 수 있다면 어떤 방식도 사요 가능하다.

파일 시스템 소프트웨어는 손쉽게 변경이 가능하다.

워크로드 특성의 변화에 따라 새로운 파일 구성 방식을 연구,개발할 자세가 되어 있어야 한다.

[4] 디렉터리 구조

vsfs의 디렉터리는 (다른 많은 파일 시스템과 마찬가지로) 매우 간단한 구조를 가진다. 디렉터리는 기본적으로 (이름, 아이노드 번호) 쌍의 배열로 구성되어 있다.

디렉터리의 데이터 블록에는 다음과 같은 정보가 저장된다.

- 문자열(name)
- 아이노드 번호(inum)
- 문자열 길이(strlen)
- 레코드 길이(reclen)

즉, 문자열과 숫자가 쌍으로 존재하는 구조이며, 가변 길이 이름을 지원하기 위해 길이 정보도 함께 저장된다.

예를 들어, dir이라는 디렉터리(아이노드 번호 5)에는 foo, bar와 foobar라는 3개의 파일이 있고 각각의 아이노드 번호는 12, 13 그리고 24라고 하자.

dir의 데이터 블록은 아래와 같은 내용을 가지고 있을 것이다.

inum	reclen	strlen	name
5	4	2	.
2	4	3	..
12	4	4	foo
13	4	4	bar
24	8	7	foobar

각 항목은 아이노드 번호와 레코드 길이(이름에 사용된 총 바이트와 남은 공간의 합), 문자열 길이(실제 이름의 길이), 그리고 마지막으로 항목의 이름을 갖고 있다.

각 디렉터리는 .(dot)과 ..(dot-dot)이라는 두 개의 추가 항목이 있으며, dot 디렉터리는 현대 디렉터리(이 예제에서는 dir)를 가리키며, dot-dot은 부모 디렉터리(이 경우에는 루트)를 가리킨다.

파일이 삭제되면(예: unlink() 호출) 디렉터리 중간에 빈 공간이 발생한다.

영역이 비어 있다는 것을 표시할 방법이 필요하다.

(예: 0번 아이노드는 삭제된 파일을 나타내기로 약속한 후 아이노드 번호를 0으로 쓰는 식)

항목의 길이를 명시하는 이유 중 하나가 중간에 빈 공간이 생기기 때문이다.

새로운 디렉터리 항목을 생성할 때, 기존 항목이 삭제되어 생긴 빈 공간에 새로 생성된 항목을 위치시킬 수도 있기 때문이다.

디렉터리들은 정확히 어디에 저장될까?

대부분의 파일 시스템에서 디렉터리는 특수한 종류의 파일로 간주된다.

디렉터리는 자신의 아이노드를 가지며, 이 아이노드는 아이노드 테이블에 존재한다(아이노드의 type 필드에 “일반 파일” 대신 “디렉터리”라고 명시되어 있다.)
디렉터리는 자신의 데이터 블록을 갖고 있으며, 이들 블록의 위치는 일반 파일과 마찬가지로 아이노드에 명시되어 있다.
이 데이터 블록들은 데이터 블록 영역에 존재한다.

이 예제에서 디렉터리는 가변 길이 레코드로 구성된 배열이다.
이 외에도 많은 방식이 존재할 수 있다.
앞에서도 언급했듯이 제대로 동작하기만 한다면 어떤 자료 구조라도 상관없다.

예를 들면, XFS 는 디렉터리를 B-tree 형식으로 구성한다.
파일 생성 시 현재 디렉터리에 동일한 이름의 파일이 있는지를 먼저 검사해야 한다.
디렉터리 항목들이 선형 배열로 구성되어 있는 경우, 항목들을 모두 검색해야 한다.

B-tree 와 같은 검색 전용 자료 구조를 사용할 경우 검색 시간이 단축된다.
파일 생성을 좀 더 빠르게 할 수 있다.(단 이전에 동일한 이름으로 파일이 생성된 적이 없어야 한다는 조건이 있다.)

여담: 링크 기반의 파일 구조

아이노드를 설계하는 데 사용되는 또 다른 간단한 방법은 연결리스트를 사용하는 것이다.
아이노드 안에서 다수의 포인터를 두지 않고 파일의 첫 번째 블록을 가리키는 포인터만 하나 둔다. 큰 파일의 경우, 파일의 마지막 블록을 가리키는 포인터를 추가한다.

예상했을 수도 있겠지만, 링크 기반의 파일 블록 구성은 특정 워크로드에 대해서 성능이 좋지 않을 수 있다. 예를 들면 가장 마지막 블록을 읽어야 하는 경우나 임의의 위치를 읽어야 하는 경우이다.

링크드 리스트 기반 구성의 성능을 개선하기 위해, 데이터 블록 내에 다음 블록의 위치를 저장하지 않고 링크드 리스트의 포인터 정보들만 테이블로 관리하기도 한다.
그러면 이 테이블을 메모리에 상주시킬 수 있다.

다음 블록의 주소를 각 블록에 저장할 경우, 10 번째 블록을 읽기 위해서는 첫 번째 블록부터 열 번째 블록까지를 차례로 읽어야 한다.

하지만 각 블록의 위치를 테이블에 모아둘 경우, 이 테이블이 메모리에 존재하면 각 블록을 순차적으로 읽는 연산은 발생하지 않는다.

테이블 항목은 테이블 블록 D 의 주소로 인덱스된다.
각 항목의 내용은 D 의 다음 블록을 가리키는 포인터, 즉 파일에서 D 다음에 오는 블록의 주소를 저장한다.

- NULL 값을 가질 수도 있으며(파일의 끝을 나타냄)
- 또는 특정 표식을 사용하여 블록이 비어 있음을 나타낼 수도 있다.

다음 포인터를 저장하는 테이블을 사용하면 연결식 할당 방법에서도 임의의 파일 접근을 효율적으로 할 수 있게 된다.

먼저(메모리 내의) 테이블을 스캔하여 원하는 블록을 찾은 후, 그 다음(디스크 내의) 해당 위치를 직접 접근하면 된다.

이러한 테이블이 익숙하게 들리는가?

방금 설명한 내용이 바로 파일 할당 테이블(File Allocation Table, FAT) 파일 시스템의 기본 구조이다.

이는 과거 Windows 에서 사용되던 파일 시스템으로, 간단한 연결 방식의 할당 방법을 사용하였다.

표준 Unix 파일 시스템들과의 차이점도 있다.

- 아이노드가 존재하지 않는다.
 - 파일의 메타데이터는 디렉터리 항목에 저장된다.
- 따라서 하드 링크를 만드는 것은 불가능하다.

그다지 세련된 방식은 아니지만, 매우 널리 사용되었던 파일 시스템이다.

여담: 빈 공간의 관리

빈 공간을 관리하는 여러 가지 방법이 있는 데 그 중 하나가 비트맵(bitmap)이다.

초기의 파일 시스템들은 프리 리스트(free list)를 사용하여 슈퍼블록 안의 한 포인터가 첫 번째 프리 블록을 가리키도록 하였다.

그리고 그 블록은 다른 프리 블록을 가리키는 포인터를 갖는 식으로 시스템 내의 프리 블록들의 리스트를 만든다.

블록이 하나 필요하면 리스트의 헤드를 사용하고, 슈퍼블록은 그 다음 블록이 프리 리스트의 헤드가 되도록 한다.

현대의 파일 시스템은 좀 더 정교한 자료 구조들을 사용한다.

예를 들어, SGI 의 XFS 의 경우, B-tree 의 일종을 사용하여 디스크의 어느 공간이 비어 있는지를 작은 크기로 표현한다.

[5] 빈 공간의 관리

파일 시스템은 아이노드와 데이터 블록 사용 여부를 관리해야 한다.

그래야 새로운 파일이나 디렉터리를 할당할 공간을 찾을 수 있다.

빈 공간 관리(free space management)는 모든 파일 시스템에서 중요하다.

vsfs에서는 두 개의 비트맵을 사용한다.

파일 생성 시 아이노드를 할당해야 한다.

파일 시스템은 아이노드 비트맵을 탐색하여 비어 있는 아이노드를 찾아 파일에 할당한다.

파일 시스템은 해당 아이노드를 사용 중으로 표기하고(1로 표기) 디스크 비트맵도 적절히 갱신한다.

이와 유사한 동작이 데이터 블록을 할당할 때에도 일어난다.

데이터 블록 할당 시 고려해야 할 사항이 또 있다.

예를 들면, ext2, ext3와 같은 Linux 파일 시스템의 경우 데이터 블록 할당 시 가능하면 여러 개의 블록들이 연속적으로 비어 있는 공간을(예를 들면, 여덟개)찾아서 할당한다.

추후에 할당 요청이 발생하면 기존에 할당된 공간에 이어서 블록을 할당하기 위해서이다.

연속적으로 여러 개의 블록들이 비어 있는 공간을 할당함으로써 해당 파일에 대한 출력 성능을 개선한다.

이러한 선할당(pre-allocation) 정책은 데이터 블록 할당 시 자주 사용된다.

[6] 실행 흐름: 읽기와 쓰기

지금까지 우리는 파일과 디렉터리가 디스크에 어떻게 저장되는지에 대해 학습했다.

이제는 파일을 실제로 읽고 쓰는 과정의 세부 흐름을 이해할 준비가 되었다. 이 단계까지 이해해야만 한다.

이러한 실행 과정(access path)을 이해하는 것은, 파일 시스템의 동작을 완전히 이해하기 위해 필요한 매우 중요한 두 번째 요소이다.

다음 예제에서는 파일 시스템이 이미 마운트(mount)되어 있고, 슈퍼블록(superblock)은 메모리 위에 존재한다고 가정한다.

하지만 다른 모든 것(예: inode, 디렉터리 등)은 아직 디스크에 존재한다. 즉, 이들은 아직 메모리에 올라오지 않은 상태이다.

디스크에서 파일 읽기(VSF 흐름 정리)

파일 /foo/bar 를 열고(open) -> 읽고(read) -> 닫는(close) 과정을 단계별로 보자.
이 파일은 크기가 4KB 라서 데이터 블록 1 개짜리 파일이라고 가정한다.

(1) open("/foo/bar") - 파일 읽기

파일을 열기 위해 파일 시스템은 먼저 bar 파일의 아이노드(inode)를 찾아야 한다.
왜냐하면 아이노드에는 파일의 권한, 크기, 데이터 위치 정보가 들어 있기 때문이다.

하지만 우리는 현재 경로(/foo/bar)만 알고 있고, 아이노드 번호는 모른다.
그래서 파일 시스템은 경로를 하나씩 따라가며 아이노드를 찾아간다.

(2) 루트 디렉터리부터 시작

모든 경로 탐색은 항상 루트 디렉터리 /에서 시작한다.

루트는 특별해서

- 부모 디렉터리가 없음
- 아이노드 번호가 미리 정해져 있음(보통 2 번)

그래서 파일 시스템은 먼저 아이노드 2 번이 들어 있는 블록을 디스크에서 읽는다.

(3) 루트 디렉터리에서 "foo"찾기

루트 아이노드를 읽으면, 그 안에는 루트 디렉터리 데이터 블록을 가리키는 포인터가 있다.
그 포인터를 따라가면 디렉터리 내용이 있다(파일 이름 + 아이노드 번호)

- "foo"라는 이름을 찾는다.
- 찾으면 -> "foo"의 아이노드 번호를 얻는다(예: 44 번)

(4) "foo" 디렉터리에서 "bar"찾기

이제 "foo"의 아이노드를 읽는다.

그 다음

- "foo" 디렉터리의 데이터 블록을 읽어서
- "bar"라는 항목을 찾는다.

그러면

- "bar"의 아이노드 번호를 얻는다.

(5) “bar” 아이노드 읽기

이제 목표에 도착하였다.

- “bar”의 아이노드를 디스크에서 읽음
- 파일크기, 권한, 데이터 블록 위치 등을 확인
- 접근 권한 체크
- 파일 디스크립터 할당

open() 완료

(6) read() - 파일 읽기

이제 실제 데이터를 읽는다.

- 처음 읽기라서 오프셋(offset) = 0
- 아이노드에 있는 포인터를 통해 데이터 블록 위치를 확인
- 해당 블록을 디스크에서 읽음(4KB)

읽고 나면

- 파일의 마지막 접근 시간(atime) 갱신
- 파일 오프셋 증가(다음 읽기 준비)

(7) close() - 파일 닫기

파일을 다 읽고 나면 닫는다.

- 파일 디스크립터 해제
- 특별한 디스크 I/O 는 없음

디스크에 파일 쓰기(VSFS 흐름 정리)

파일 /foo/bar 를 생성하고 (write) -> 데이터 쓰고 -> 닫는 과정을 보자

(1) open() - 파일 생성

먼저 open("/foo/bar", O_CREATE | O_WRONLY) 같은 호출이 들어온다.

이 단계에서 하는 일은

경로 탐색

- / -> foo 까지 찾아감(읽기 때랑 동일)

새로운 파일 생성

이제 "bar"가 없으니까 새로 만들어야 한다.

- 빈 inode 하나 할당 - 디렉터리(foo)에 "bar" 항목 추가

여기서 발생하는 I/O(파일 하나 생성하는데도 꽤 많다)

- inode bitmap 읽기(빈 inode 찾기) - inode bitmap 쓰기(사용 중 표시) - 새 inode 쓰기(초기화) - 디렉터리 데이터 블록 쓰기(bar 추가) - 디렉터리 inode 읽기 + 쓰기 (크기 변경 등)
--

write() 1 번당 발생하는 I/O 는 5 번이다.

(2) write() - 실제 데이터 쓰기

이제 파일에 데이터를 쓴다.

여기서는 블록 3 개(각 4KB)를 쓴다고 가정한다.

기존 블록이 아니라 새로운 블록을 할당하면서 쓰는 경우를 allocating write 라고 한다.

총 15 번의 I/O 가 발생한다.

(3) close() - 파일 닫기

- 파일 디스크립터 해제
- 추가 디스크 I/O 없음

여담: 읽기는 할당 자료 구조를 접근하지 않는다.

파일을 읽기(read)만 할 때는 비트맵과 같은 할당 자료 구조를 전혀 볼 필요가 없다. 많은 사람들이 “디스크에서 뭔가를 읽으니까 비트맵도 확인해야 하지 않나”라고 생각하는데, 실제로는 그렇지 않다.

비트맵의 역할은 단 하나다.

어떤 블록이 비어 있는지(사용 가능한지) 관리하는 것이다. 즉, 새로운 공간을 할당할 때만 필요하다.

반대로 읽기 작업은 이미 만들어진 파일을 대상으로 한다.

이 경우 파일에 필요한 정보는 전부 아이노드 안에 들어 있다.

아이노드는 해당 파일이 어떤 데이터 블록들을 사용하는지 정확히 알고 있고, 그 위치를 포인터로 가지고 있다.

따라서 읽기 과정은 다음처럼 진행된다.

아이노드를 읽고 -> 그 안에 있는 포인터를 따라가서 -> 데이터 블록을 읽는다.

이 과정에서 “이 블록이 사용 중인지”를 다시 확인할 필요는 없다. 이미 아이노드가 그 블록을 사용한다고 명시하고 있기 때문이다.

정리하면, 비트맵은 “앞으로 쓸 공간을 찾기 위한 도구”이고, 아이노드는 “이미 사용 중인 데이터의 위치를 알려주는 지도”이다.

읽기 작업은 지도를 따라가는 일이기 때문에, 빈 공간을 찾는 비트맵은 전혀 필요하지 않다.

[7] 캐싱과 버퍼링

파일 시스템에서 파일을 읽고 쓰는 작업은 디스크 I/O 를 많이 발생시키기 때문에 전체 성능에 큰 영향을 준다. 이를 해결하기 위해 대부분의 파일 시스템은 자주 사용되는 디스크 블록을 메모리(DRAM)에 저장해 두는 방식, 즉 캐싱을 사용한다.

먼저 파일을 여는 과정을 보면 캐시가 없는 경우에 경로를 따라가면서 각 디렉터리마다 아이노드와 데이터 블록을 계속 디스크에서 읽어야 한다.

예를 들어 /1/2/3/.../file.txt 처럼 경로가 길어지면 각 단계마다 디렉터리 아이노드와 디렉터리 내용을 디스크에서 읽어야 하므로 파일을 여는 것만으로도 수십에서 수백 번의 디스크 읽기가 발생할 수 있다.

하지만 캐시를 사용하면 이전에 읽었던 아이노드나 디렉터리 블록이 메모리에 남아 있기 때문에 같은 경로를 다시 탐색할 때 디스크 접근 없이 메모리에서 바로 처리할 수 있다. 그래서 반복적인 파일 접근에서는 I/O 가 크게 줄어든다.

캐시에 저장된 데이터가 계속 유지되는지 LRU 같은 교체 정책이 결정하난.

초기 시스템에서는 전체 메모리의 일정 비율(예: 10%)을 캐시로 고정해서 사용했다. 그러나 이 방식은 비효율적이 경우가 많았다. 어떤 시점에는 캐시가 너무 많이 남아 낭비되기도 하고, 반대로 부족하면 성능이 떨어지기 때문이다.

이를 해결하기 위해 현대 운영체제는 통합 페이지 캐시(unified page cache) 구조를 사용한다. 이는 파일 시스템 캐시와 가상 메모리 캐시를 하나로 통합한 구조로, 메모리를 파일과 메모리 모두에 유동적으로 사용할 수 있게 한다.

즉, 특정 순간에 파일 I/O 가 많으면 캐시에 더 많은 메모리를 할당하고, 그렇지 않으면 다른 용도로 사용할 수 있다.

캐시가 있는 경우 파일을 처음 열 때는 디렉터리 탐색 과정 때문에 여러 디스크 I/O 가 발생하지만, 이후 같은 파일이나 같은 디렉터리 내 파일을 다시 열면 대부분 캐시에 존재하기 때문에 추가적인 디스크 I/O 없이 빠르게 처리된다.

쓰기의 경우는 읽기와 다르게 동작한다.

읽기는 필요한 데이터를 가져오기만 하면 되지만, 쓰기는 파일 시스템 구조를 변경할 수 있기 때문에 더 복잡하다. 특히 새로운 블록이 필요할 경우에는 디스크 비트맵을 수정하고 아이노드를 갱신해야 한다.

하지만 모든 쓰기를 즉시 디스크에 반영하면 성능이 매우 나빠진다.

그래서 파일 시스템은 쓰기 작업을 바로 수행하지 않고 메모리에 잠시 저장해 두는 방식인 쓰기 버퍼링(write buffering)을 사용한다.

쓰기 버퍼링을 사용하면 여러 가지 이점이 있다.

첫째, 여러 번의 쓰기 요청을 모아서 한 번에 디스크에 기록할 수 있어 I/O 횟수를 줄일 수 있다.

둘째, 비슷한 시점에 발생한 쓰기 작업들을 합쳐서 처리할 수 있다.

예를 들어, 파일 생성이 연속적으로 발생하면 아이노드 비트맵 갱신 작업을 한 번으로 합칠 수 있다.

셋째, 경우에 따라서는 쓰기 자체를 하지 않아도 되는 상황이 발생한다.

예를 들어, 파일을 생성했다가 바로 삭제하는 경우 디스크에 기록할 필요가 없어진다.

이러한 이유로 대부분의 시스템은 쓰기 작업을 일정 시간 동안 메모리에 유지한 후 디스크에 반영한다. 보통 몇 초에서 수십 초 정도 지연시킨다.

하지만 이 방식에는 위험도 존재한다. 만약 디스크에 기록되기 전에 시스템이 크래시되면 해당 데이터는 유실될 수 있다.

이 문제를 해결하기 위해 중요한 데이터를 즉시 디스크에 기록해야 하는 경우 `fsync()`를 사용한다. 이를 호출하면 메모리에 있는 변경 사항이 강제로 디스크에 반영된다.

또한 일부 응용 프로그램은 캐시를 사용하지 않는 direct I/O 나 raw disk 접근 방식을 사용하기도 한다.

결국 대부분의 일반적인 프로그램은 성능을 위해 파일 시스템의 캐싱과 버퍼링을 사용하지만, 데이터의 중요도에 따라 이를 제어할 수 있는 방법도 함께 제공된다.

팁: 정적 대 동적 파티션 이해하기

컴퓨터 시스템에서 메모리 디스크 대역폭 같은 자원을 여러 사용자나 작업에 나눠줄 때는 크게 두가지 방식이 있다.

정적 파티션(static partitioning)과 동적 파티션(dynamic partitioning)이다.

정적 파티션은 자원을 처음에 한 번 고정된 크기로 나누고, 이후에는 바꾸지 않는 방식이다.

예를 들어, 메모리를 두 사용자가 나눠쓴다고 하면, 처음부터 각각 일정 비율을 정해준다.

한 사용자는 50%, 다른 사용자는 50%처럼 미리 정해진 영역만 사용할 수 있다.

이 방식의 장점은 구조가 단순하고 예측 가능한 성능을 얻을 수 있다는 점이다.

각 사용자는 항상 자신에게 할당된 만큼의 자원을 보장받기 때문에 시스템이 안정적이다.

대신 문제가 있어도 유동적으로 대응 할 수 없다.

반면 동적 파티션은 상황에 따라 자원 배분이 계속 바뀌는 방식이다.

예를 들어, 사용자가 특정시간 동안 디스크를 많이 사용하고 있다면, 시스템은 그 사용자에게 더 많은 디스크 대역폭을 줄 수 있다.

이후 다른 사용자가 더 필요해지면 다시 자원을 돌려줄 수도 있다. 이 방식은 자원을 효율적으로 사용할 수 있다는 장점이 있다.

놓고 있는 자원을 다른 곳에서 활용할 수 있기 때문이다.

하지만 동적 방식은 단점도 있다. 자원을 빌려줬다가 다시 회수해야 할 수도 있는데, 그 과정에서 기존 사용자의 성능이 갑자기 나빠질 수 있다.

또한, 전체 시스템이 어떤 방식으로 자원을 재분배할지 계속 판단해야 하기 때문에 구현이 복잡해진다.

결국 정적 파티션은 단순하고 예측 가능하지만 유연성이 부족하고, 동적 파티션은 효율적이지만 복잡하고 성능이 변동될 수 있다.

그래서 실제 시스템에서는 상황에 따라 어떤 방식이 더 적절한지를 판단해서 사용해야 한다.

팁: 영속성과 성능 간의 절충

저장 시스템에서 가장 중요한 두 가지 목표는 데이터의 영속성(persistence)과 성능(performance)이다.

그런데 이 두가지는 동시에 최대로 만족시키기 어렵기 때문에 항상 서로 균형을 맞추는 절충(trade-off)이 필요하다.

만약 데이터가 즉시 안전하게 저장되기를 원한다면, 모든 쓰기 작업은 반드시 디스크에 바로 기록되어야 한다. 즉, 메모리에 잠시 저장하지 않고 즉시 커밋해야 한다.

이렇게 하면 시스템이 너제 크래시가 나더라도 데이터는 안전하게 유지된다.

하지만 문제는 성능이다.

디스크는 메모리보다 훨씬 느리기 때문에 모든 쓰기를 즉시 디스크에 반영하면 전체 시스템 속도가 크게 떨어진다.

반대로 성능을 더 중요하게 생각하면 다른 방식이 사용된다.

쓰기 작업을 바로 디스크에 기록하지 않고 메모리에 먼저 저장한 뒤 일정 시간 동안 모아둔다. 이후 백그라운드에서 한꺼번에 디스크에 기록한다.

이 방식은 사용자 입장에서는 쓰기 작업이 매우 빠르게 끝난 것처럼 보이기 때문에 체감 성능이 좋아진다.

하지만 이 경우에는 위험이 있다. 시스템이 갑자기 크래시되면 아직 디스크에 기록되지 않은 데이터는 모두 사라질 수 있다.

결국 어떤 방식이 옳은지는 절대적으로 정해져 있지 않다. 중요한 것은 응용 프로그램이 무엇을 요구하는지를 이해하고 그에 맞는 방식을 선택하는 것이다.

예를 들어, 웹 브라우저가 다운로드한 이미지 몇 개는 손실되어도 큰 문제가 되지 않는다.

하지만 은행 거래나 데이터베이스 트랜잭션처럼 중요한 데이터는 단 하나라도 손실되면 심각한 문제가 발생한다.

따라서 시스템 설계에서는 항상 “속도와 안전성 중 무엇을 더 우선할 것인가”를 상황에 맞게 결정해야 한다.

[1-3] 지역성과 Fast File System

처음 Unix 가 등장했을 때, Unix 의 창시자 Ken Thompson 장신이 첫 번째 파일 시스템을 개발하였다. 그 파일 시스템은 정말 단순해서 “오래된 Unix 파일 시스템”이라 불린다. 디스크 상의 구조는 다음과 같다.



파일 시스템 전체를 관리하는 가장 중요한 구조는 슈퍼블록이다. 슈퍼블록은 디스크의 가장 앞부분에 위치하며, 파일 시스템 전체에 대한 핵심 정보를 가지고 있다.

예를 들어, 전체 디스크의 크기, 아이노드의 개수, 그리고 빈 블록을 관리하기 위한 정보(프리 블록 리스트의 시작 위치 등)가 여기에 저장된다.

슈퍼블록 뒤에는 아이노드들이 저장되는 영역이 있다.

이 영역에는 모든 파일과 디렉터리에 대한 아이노드가 모여 있으며, 각 아이노드는 해당 파일의 메타데이터와 데이터 블록 위치 정보를 가지고 있다.

즉, 파일의 이름을 제외한 거의 모든 중요한 정보는 아이노드에 저장된다.

그 다음에는 데이터 영역이 존재한다.

이 영역은 실제 파일의 내용이 저장되는 공간으로, 대부분의 디스크 공간이 여기에 해당한다. 파일이 커지면 이 데이터 영역에 블록이 할당되고, 이 아이노드는 이 블록들을 포인터로 가리키게 된다.

이 구조의 가장 큰 장점은 단순함이다. 파일 시스템은 기본적으로 파일과 디렉터리라는 두 가지 개념만으로 구성되어 있으며, 복잡한 구조 없이도 동작할 수 있다.

이러한 단순성 덕분에 구현이 쉽고 이해하기도 편리하다.

하지만 단순하다고 해서 가치가 낮은 것은 아니다. 오히려 이런 구조는 기존의 더 단순한 저장 방식, 예를 들어 코드 기반이나 평면적인 저장 구조에 비해 큰 발전이었다.

특히 유닉스 시스템에서 도입된 계층적 디렉터리 구조는 이전처럼 모든 파일이 한 곳에 나열되는 단일 디렉터리 방식과 비교하면 매우 중요한 변화였다.

즉, 이 파일 시스템은 단순하지만 그 안에서 파일을 계층적으로 관리할 수 있게 만들었고, 이는 이후 현대 파일 시스템의 기본 구조가 되는 중요한 발전이었다.

[1] 문제: 낮은 성능

구형 파일 시스템의 가장 큰 문제는 성능이 매우 낮다는 점이다.

연구 결과에 따르면 시간이 지날수록 성능이 계속 떨어져서, 결국에는 디스크의 전체 대역폭 중 약 2% 정도 밖에 사용하지 못하는 수준까지 떨어질 수 있었다.

이 문제가 발생한 핵심 이유는 파일 시스템이 디스크를 저장 장치가 아니라 마치 RAM 처럼 취급했다는 점이다.

즉, 디스크라는 장치는 물리적으로 헤드가 이동해야 하는데, 이를 고려하지 않고 데이터를 아무 위치에나 저장했다. 그 결과 디스크 접근 시 헤드 이동 시간이 매우 발생하게 된다.

예를 들어, 파일을 읽는 과정을 보면, 먼저 아이노드를 읽고 그 다음 데이터 블록을 읽는다. 그런데 아이노드와 데이터 블록이 디스크에서 서로 멀리 떨어져 있는 경우가 많다.

이 경우 아이노드를 읽은 뒤 다시 데이터 블록 위치로 헤드를 이동해야 하므로 추가 시간이 많이 발생한다.

이런 “아이노드 접근 -> 데이터 접근” 패턴은 매우 자주 발생하기 때문에 전체 성능에 큰 영향을 준다.

문제를 더 악화시키는 것은 파일 시스템의 공간 관리 방식이다.

구형 파일 시스템은 빈 공간을 효율적으로 관리하지 못하고, 단순히 “다음 빈 블록”을 리스트에서 꺼내서 할당하는 방식이었다.

이로 인해 시간이 지나면서 디스크 전체에 빈 공간이 흩어지는 단편화가 발생한다.

예를 들어, 디스크에 파일 A, B, C, D 가 있고 각각이 연속된 공간에 있지 않고 여기저기 흩어져 있다고 가정해보자. 이때 B와 D가 삭제되면 그 자리에 빈 공간이 생기지만, 이 공간들은 연속된 하나의 큰 공간이 아니라 작은 조각들로 나뉘어 남게 된다.

이 상태에서 새로운 파일 E를 저장하려고 하면, E를 구성하는 블록들은 연속된 공간에 들어가지 못하고 디스크 전역에 흩어져 저장된다.

따라서 E를 읽거나 쓸 때에도 블록들은 순차적으로 읽을 수 없고, 매번 디스크 헤드를 이동해야 한다. 즉, 논리적으로는 순차 접근이지만 실제 물리적으로는 랜덤 접근이 되어 성능이 크게 떨어진다.

이 문제는 오래된 유닉스 파일 시스템에서도 흔하게 발생했다. 이를 해결하기 위해 등장한 것이 조각 모음(defragmentation) 도구이다. 이 도구는 파일 블록들을 물리적으로 이동시켜 연속된 위치에 배치하고, 그것에 맞는 아이노드 정보를 수정한다.

또 다른 문제는 블록 크기 자체가 너무 작았다는 점이다. 예를 들어 512바이트 같은 작은 블록을 사용하면 내부 단편화는 줄어들지만, 디스크 입장에서 보면 한 번의 I/O로 가져오는 데이터 양이 너무 적다. 디스크는 물리적으로 데이터를 읽는 데 시간이 오래 걸리기 때문에, 작은 블록을 여러 번 읽는 것은 매우 비효율적이다.

결국 문제를 정리하면 두 가지로 볼 수 있다. 하나는 디스크의 물리적 특성을 고려하지 않은 공간 배치로 인한 헤드 이동 비용 증가이고, 다른 하나는 작은 블록 크기로 인한 비효율적인 데이터 전송이다.

핵심 질문: 성능 개선을 위해 어떻게 디스크 상의 데이터를 구성해야 할까

- 성능을 위해 파일 시스템의 자료 구조를 어떻게 구성해야 할까?
- 이러한 자료 구조에는 어떤 종류의 할당 정책이 필요할까?
- 어떻게 파일 시스템에 디스크 특성을 반영할까?

[2] FFS: 디스크에 대한 이해가 해답이다.

Berkeley 의 한 그룹이 좀 빠르고 좋은 파일 시스템을 만들기로 결정하였다.

그리고 독창적으로 Fast File System, FFS 라고 명명하였다.

파일 시스템의 자료 구조와 할당 정책을 “디스크에 적합”하게 설계하여 성능을 개선하는 것이 핵심이었다.

실제로 그들은 목적을 달성하였다. FFS 는 파일 시스템 연구의 새로운 장을 열었다.

파일 시스템 인터페이스는 그대로 유지하면서(open(), read(), write(), close() 등),

내부 구현 방식을 변경하였다.

이 연구를 통해 파일 시스템 개발에 새로운 지평을 열었고, 그 길은 지금까지도 사용되고 있다. 거의 모든 현대의 파일 시스템은 기존의 인터페이스를 그대로 사용하면서(응용 프로그램과의 호환성을 유지하기 위해) 성능과 신뢰성 또는 기타 다른 목적으로 내부를 최적화하고 있다.

[3] 파일 시스템 구조: 실린더 그룹

구형 파일 시스템의 가장 큰 문제는 디스크 전체에 데이터가 흩어지면서 탐색 시간이 계속 늘어난다는 점이었다. 이를 해결하기 위해 등장한 개념이 FFS 이며, 그 핵심 구조가 실린더 그룹(cylinder group)이다. 현대 시스템에서는 이를 블록 그룹(block group)이라고 부르기도 한다.

실린더 그룹의 기본 아이디어는 디스크 전체를 하나로 보지 않고, 여러 개의 작은 단위로 나누는 것이다. 디스크를 여러 개의 구역으로 나눈 뒤, 각 구역을 독립적인 작은 파일 시스템처럼 다루는 방식이다.

이 구조의 핵심 목표는 “관련 있는 데이터는 최대한 가까운 위치에 배치한다”는 것이다.

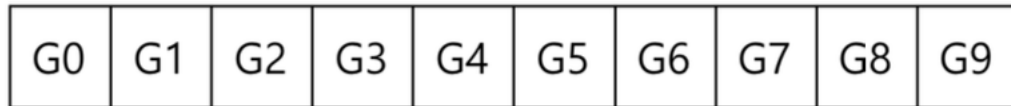
예를 들어, 하나의 파일과 그 파일이 들어 있는 디렉터리를 같은 그룹 안에 배치하면, 파일을 열거나 다른 파일로 이동할 때 디스크 전체를 헤드가 이동할 필요가 줄어든다.

즉, 물리적인 탐색 비용을 줄이는 것이 목적이다.

각 실린더 그룹은 독립적인 구조를 가진다. 그룹 안에는 파일 시스템의 핵심 구성 요소들이 모두 포함된다.

먼저 전체 파일 시스템의 상태를 나타내는 슈퍼블록의 복사본이 각 그룹에 존재한다.

이는 디스크 일부가 손상되더라도 다른 복사본을 이용해 파일 시스템을 복구하거나 마운트할 수 있게 하기 위함이다.



또한 각 그룹에는 아이노드와 데이터 블록의 사용 상태를 관리하기 위한 비트맵이 존재한다. 아이노드 비트맵과 데이터 비트맵은 각각 어떤 아이노드와 데이터 블록이 사용 중인지, 혹은 비어 있는지를 표시한다.

이러한 비트맵 구조 덕분에 파일 시스템은 빈 공간을 빠르게 찾을 수 있고, 연속된 공간을 확보하기도 쉬워진다.



이 방식은 기존의 단순한 프리 리스트 방식보다 훨씬 효율적이다.

프리 리스트 방식에서는 빈 블록이 디스크 전체에 흩어지기 쉬워 단편화가 심해졌지만, 비트맵 기반 구조에서는 상대적으로 큰 연속 공간을 찾고 할당하기가 더 쉽다.

각 그룹 내부에는 아이노드 영역과 데이터 블록 영역도 존재한다.

아이노드는 파일의 메타데이터를 저장하고, 데이터 블록은 실제 파일 내용을 저장한다.

결국 각 실린더 그룹은 작은 독립적인 파일 시스템처럼 동작하며, 대부분의 데이터는 이 그룹 내부에 분산되어 저장된다.

정리하면

- 디스크 전체를 여러 그룹으로 나눔(실린더 그룹)
- 관련 데이터는 같은 그룹에 배치
- 디스크 헤드 이동 최소화(성능 개선)
- 각 그룹이 독립적인 구조(슈퍼블록, inode bitmap, data bitmap 포함)

여담: FFS 에서 파일 생성

파일이 생성될 때 어떠한 자료 구조들이 변경되어야 하는지 생각해보자.

이 예제에서는 사용자가 /foo/bar.txt 라는 파일을 생성하고 블록 하나 크기(4KB)라고 하자. 새로운 파일이기 때문에 새로운 아이노드가 필요하다.

아이노드 비트맵과 새로 할당받은 아이노드가 디스크에 기록된다.

파일에는 데이터도 있기 때문에 데이터 비트맵과 데이터 블록도 자연스럽게 디스크에 기록되어야 한다.

그렇게 하여 최고의 네 번의 쓰기가 현재의 실린더 그룹에서 발생한다(이 쓰기들은 실제 기록이 되기 전에 버퍼에 잠시동안 있을 수 있다.)

하지만 이게 전부가 아니다.

파일 생성을 구체적으로 살펴보면 파일을 파일 시스템 계층에 저장하여야 하므로 디렉터리도 같이 갱신되어야 한다.

여기서는 bar.txt 에 대한 항목을 추가하기 위해 부모 디렉터리인 foo 가 갱신되어야 한다.

이 갱신은 foo 가 있는 데이터 블록에 같이 기록될 수도 있고 또는 새로운 블록을 할당받아야 할 수도 있다.(해당 하는 데이터 비트맵과 함께)

변경된 디렉터리의 크기와 갱신 시간 필드(마지막 변경 시간과 같은)를 반영하기 위해 foo 의 아이노드 역시 갱신되어야 한다.

전체적으로 단순한 파일 하나를 생성하기 위해 많은 작업이 필요하다.

다음에 당신이 파일을 만들 때는 좀 더 감사해야 할 것이다. 최소한 모든 것이 다 잘 동작하는 것에 대해 놀라야 할 것이다.

[4] 파일과 디렉터리 할당 정책

FFS 에서는 실린더 그룹이라는 구조를 만든 후, 이제 파일과 디렉터리, 그리고 관련된 메타데이터를 디스크 어디에 배치할지를 결정해야 한다.

이 단계에서 중요한 원칙은 단순하다.

서로 관련 있는 데이터는 가능한 가까운 위치에 저장하고, 관련이 없는 데이터는 떨어뜨려 저장한다는 것이다.

이 원칙을 적용하려면 먼저 “무엇이 관련 있는 데이터인지”를 판단해야 한다.

하지만 파일 시스템은 파일의 의미를 이해할 수 없기 때문에 완벽하게 판단할 수는 없다.

따라서 FFS 는 실제 사용 패턴에 기반한 경험적인 규칙, 즉 휴리스틱을 사용한다.

먼저 디렉터리 배치부터 보면, FFS 는 특정 조건을 기준으로 디렉터를 실린더 그룹에 배치한다. 한 그룹 내에 디렉터리가 너무 많이 몰리지 않도록 균형을 고려하고, 동시에 해당 그룹에 충분한 빈 아이노드가 존재하는지를 확인한다. 이런 조건을 만족하는 그룹을 선택해서 디렉터리와 그에 관련된 아이노드를 저장한다.

즉, 디렉터리는 특정 기준에 따라 적절히 분산 배치된다.

파일의 경우에는 조금 더 구체적인 규칙이 적용된다.

일반적으로 FFS 는 파일의 아이노드와 데이터 블록을 같은 실린더 그룹에 배치하려고 한다. 이렇게 하면 아이노드에서 데이터 블록으로 접근할 때 디스크 헤드 이동이 줄어들기 때문에 성능이 좋아진다.

또한, 동일한 디렉터리 안에 있는 파일들은 가능한 한 같은 실린더 그룹에 함께 배치된다. 예를 들어, /dir1/1.txt, /dir1/2.txt, /dir1/3.txt 처럼 같은 디렉터리에 있는 파일들은 같은 그룹에 저장된다.

하지만 모든 경우가 그렇게 이상적으로 동작하는 것은 아니다.

예를 들어, /dir99/4.txt 처럼 다른 디렉터리에 속한 파일은 다른 실린더 그룹에 배치될 수 있다. 이는 파일 시스템이 디렉터리 단위로 지역성을 가정하기 때문이다.

이러한 정책의 핵심 가정은 간단하다. 같은 디렉터리 안의 파일들은 함께 사용될 가능성이 높다는 것이다.

예를 들어, 여러 소스 파일을 컴파일하거나 하나의 프로그램을 구성할 때, 같은 디렉터리 내 파일들이 함께 접근되는 경우가 많다.

따라서 이들을 가까운 위치에 두면 디스크 탐색 시간이 줄어들고 전체 성능이 개선된다.

결국 FFS 의 배치 정책은 정밀한 분석 결과라기보다는 실제 사용 경험에 나온 상식에 기반한 것이다. 완벽한 예측은 불가능하지만, 일반적인 사용 패턴을 활용하면 충분히 성능을 개선할 수 있다는 접근이다.

[5] 파일 접근의 지역성 측정

이 장에서는 FFS 가 사용한 핵심 가정, 즉 “파일 시스템에서는 지역성(locality)이 존재한다”는 가정이 실제로 타당한지를 확인하려고 한다.

여기서 말하는 지역성이란, 파일들이 디렉터리 구조 내에서 서로 가까운 위치에 존재하고, 실제 접근도 그 근처에서 반복적으로 일어난다는 의미이다.

이 가정을 검증하기 위해 SEER 트레이스 데이터를 사용한다. 이 데이터는 여러 워크스테이션에서 발생한 실제 파일 접근 기록을 기반으로 한다.

분석 방법은 파일 접근 간의 “거리”를 정의하고, 그 거리가 얼마나 자주 짧게 나타나는지를 측정하는 것이다.

여기서 거리라는 개념은 디렉터리 트리 구조에서 두 파일이 얼마나 떨어져 있는지를 의미한다. 예를 들어, 같은 파일을 연속해서 두 번 열었다면 그 사이의 거리는 0 이다. 같은 디렉터리 안에 있는 다른 파일들, 예를 들어, dir/f 와 dir/g 를 연속으로 접근하면 거리는 1 이다.

이 거리 값은 두 파일의 경로를 비교해서 공통 조상 디렉터리까지 얼마나 올라가야 하는지를 기준으로 계산된다.

즉, 트리 구조에서 가까울수록 거리는 작고, 멀수록 거리는 커진다.

실제 SEER 트레이스를 분석한 결과를 보면, 약 7%의 경우는 직전에 열었던 파일을 다시 열고 있었다. 그리고 약 40% 정도의 접근은 같은 파일이거나 같은 디렉터리 안의 파일이었다. 즉, 거리로 보면 0 또는 1에 해당하는 경우가 상당히 많았다. 이는 FFS가 가정한 “같은 디렉터리나 가까운 파일링에 자주 접근한다”는 전체가 어느 정도 맞다는 것을 보여준다.

하지만 흥미로운 점도 있다. 약 25% 정도의 접근은 거리 2에 해당하는 경우였다. 이는 단순히 같은 디렉터리가 아니라, 서로 관련된 디렉터리 구조를 나눠서 사용하는 경우이다. 예를 들어, 소스 코드 디렉터리(src)와 컴파일 결과 디렉터리(obj)를 분리해서 사용하는 방식이 대표적이다.

이 경우 두 디렉터리는 같은 상위 디렉터리(proj) 아래에 존재하기 때문에 구조적으로는 가깝지만, 직접적인 디렉터리는 다르다.

따라서 접근 거리로 보면 2가 된다. 하지만 FFS는 이런 형태의 지역성까지는 완전히 고려하지 못하기 때문에, 실제로는 추가적인 디스크 탐색이 발생할 수 있다.

비교를 위해 랜덤 트레이스도 생성해서 분석했다.

이는 실제 접근 순서를 무작위로 섞은 경우이다. 이 경우에는 예상대로 지역성이 거의 나타나지 않았다. 다만 모든 파일이 결국 루트 디렉터리 아래에 존재하기 때문에 아주 약한 수준의 구조적 지역성은 남아 있었다.

결론적으로 실제 파일 시스템 접근 패턴은 완전히 무작위가 아니라 어느 정도 지역성을 가지고 있으며, 특히 같은 디렉터리 내 접근이 매우 자주 발생한다. 하지만 일부 작업 패턴에서는 디렉터리 간의 구조적 관계까지 고려해야 하는 경우도 존재한다.

[6] 대용량 파일 예외 상황

FFS에서 앞서 설명한 파일 배치 정책은 기본적으로 “관련 있는 파일들을 같은 실린더 그룹에 모아라”는 원칙을 따른다. 이 방식은 작은 파일이나 디렉터리 중심의 접근에서는 매우 효과적이다. 하지만 이 원칙을 그대로 적용하면 문제가 발생하는 경우가 있다. 바로 매우 큰 파일이 존재하는 상황이다.

큰 파일이 하나의 실린더 그룹에 계속 저장된다고 가정해 보자. 그러면 그 파일이 사용하는 블록 수가 너무 많아져서 결국 해당 그룹의 공간 대부분을 차지하게 된다. 이렇게 되면 그 그룹에 저장되려고 했던 다른 파일들, 특히 같은 디렉터리에 속한 “관련 있는 파일들”이 들어갈 공간이 부족해진다. 결과적으로 FFS가 중요하게 생각하는 지역성, 즉 관련 파일들을 가까이 두는 전략이 깨지게 된다.

이 문제를 해결하기 위해 FFS는 큰 파일을 특별하게 처리한다. 작은 파일처럼 한 그룹에 몰아서 저장하지 않고, 파일을 여러 개의 “청크”로 나누어 저장한다.

구체적으로 보면 다음과 같다. 먼저 파일의 앞부분, 예를 들어, 아이노드의 직접 포인터가 가리키는 정도의 작은 블록들(약 12개 정도)은 첫 번째 실린더 그룹에 저장된다. 이 부분은 파일 접근 시 가장 먼저 사용되는 영역이기 때문에 아이노드와 가까운 위치에 두는 것이 효율적이다.

그 이후부터는 파일을 더 이상 같은 그룹에 계속해서 저장하지 않는다. 대신 파일을 일정 크기의 덩어리, 즉 청크 단위로 나눈 뒤 각 청크를 서로 다른 실린더 그룹에 분산해서 저장한다.

예를 들어, 첫 번째 청크는 A 그룹, 두 번째 청크는 B 그룹, 세 번째 청크는 C 그룹과 같은 방식이다.

이 방식은 직관적으로 보면 문제가 있어 보인다. 파일이 한 곳에 모여 있지 않고 디스크 전역에 퍼지기 때문에, 순차적으로 읽을 때마다 디스크 헤드가 계속 이동해야 한다. 예를 들어, 0부터 9까지의 블록을 순서대로 읽는 상황에서도 각 블록이 서로 다른 그룹에 있다면 매번 탐색이 발생한다.

즉, 순차 파일 접근에서는 성능이 떨어질 수밖에 없다. 이 점은 FFS도 알고 있는 문제다.

하지만 FFS는 이 방식이 전체 시스템 관점에서는 더 낫다고 판단했다. 이유는 큰 파일 하나의 최적화보다, 작은 파일과 디렉터리들의 지역성을 유지하는 것이 더 중요하기 때문이다. 만약 큰 파일이 한 그룹을 독점하면, 그 이후 생성되는 관련 파일들이 같은 그룹에 들어가지 못하고 전체 구조가 망가진다.

왜 이런 손해를 감수하나

이 설계에는 중요한 전제가 있다. 실제 시스템에서는 파일 접근이 완전히 순차적인 경우보다, 다양한 파일을 오가며 접근하는 경우가 더 많다는 것이다.

즉, “전체 파일을 처음부터 끝까지 계속 읽는 경우” 보다 “여러 파일을 섞어서 사용하는 경우”가 더 일반적이다.

그래서 FFS는 큰 파일의 순차 성능 일부를 희생하더라도, 전체 디렉터리 구조의 지역성을 유지하는 쪽을 선택한다.

성능 문제를 줄이는 방법: 청크 크기

물론 이렇게 파일을 나누면 성능이 무조건 나빠지는 것은 아니다. 핵심은 “청크 크기”이다.

청크 크기가 너무 작으면:

- 파일이 너무 잘게 쪼개진다.
- 디스크 헤드 이동이 매우 자주 발생한다.
- 순차 접근 성능이 급격히 떨어진다.

청크가 충분히 크면

- 한 번 헤드 이동 후 많은 데이터를 읽을 수 있다.
- 탐색 비용이 상대적으로 희석된다.
- 전체 성능이 유지된다.

이 개념을 설명할 때 사용하는 것이 “분할 상환(amortization)”이다. 즉, 한 번 발생하는 탐색 비용을 더 많은 데이터 전송에 나누어 부담시키는 방식이다.

왜 409.6KB 같은 계산이 나오나

디스크 성능은 두 부분으로 나뉜다.

- 탐색 및 회전 시간: 한 번 발생하면 고정적으로 10ms 정도 소모
- 데이터 전송 속도: 초당 40MB 정도로 매우 빠름

만약 전체 성능의 절반을 탐색에 쓰고 절반을 데이터 전송에 쓰고 싶다면, 탐색 10ms 동안 전송해야 할 데이터 양이 결정된다. 계산하면 약 409.6KB 정도가 된다.

이 의미는 단순하다.

탐색 비용을 낭비하지 않으려면, 한 번 위치를 잡을 때 충분히 많은 데이터를 읽어야 한다는 것이다.

$$\frac{40 \text{ MB}}{\text{sec}} \cdot \frac{1024 \text{ KB}}{1 \text{ MB}} \cdot \frac{1 \text{ sec}}{1000 \text{ msec}} \cdot 10 \text{ msec} = 409.6 \text{ KB}$$

FFS 의 실제 구현 방식

이런 이론적 계산을 기반으로 FFS 가 실제로 구현한 방식은 더 단순하다.

복잡한 최적 계산을 하지 않고, inode 구조를 이용해서 규칙을 정했다.

- 직접 포인터가 가리키는 첫 12 개 블록은 같은 그룹에 둔다.
- 이후 간접 포인터가 가리키는 블록들은 다른 그룹에 둔다.

예를 들어, 블록 크기가 4KB 이고 포인터 구조를 고려하면

- 처음 48KB 정도만 같은 그룹
- 이후 약 4MB 단위로 그룹이 나누어 저장된다.

전체 구조의 의미

디스크 기술은 점점 빨라지고 있지만, 디스크의 물리적인 움직임(헤드 이동, 회전)은 여전히 느리다. 그래서 시간이 지나도 “탐색 비용”은 중요한 성능 병목으로 남는다.

따라서 시스템은 점점 더 이런 방향으로 간다.

- 한 번 위치를 잡으면 더 많은 데이터를 처리한다.
- 이동 횟수를 줄인다.
- 대신 공간 배치는 더 복잡해진다.

[7] FFS 에 대한 기타 사항

FFS 는 당시 기준으로 매우 혁신적인 파일 시스템이었다. 단순히 디스크 성능만 개선하는 것이 아니라, 실제 사용에서 발생하는 여러 비효율과 불편함까지 함께 해결하려고 했다. 특히 중요한 목표 중 하나는 “작은 파일을 어떻게 효율적으로 저장할 것인가”였다.

당시 일반적인 파일 시스템은 4KB 블록 단위로 저장했다. 그런데 실제로는 2KB 정도 되는 작은 파일이 매우 많이 존재했다.

이 경우 4KB 블록 하나를 사용하면 나머지 절반이 사용되지 못하고 버려지게 된다. 이런 현상을 내부 단편화라고 한다. 결과적으로 평균적으로 절반 정도의 디스크 공간이 낭비되는 문제가 발생했다.

FFS 는 이 문제를 해결하기 위해 새로운 개념을 도입했다. 바로 512 바이트 단위의 서브블록(sub-block)이다.

작은 파일을 저장할 때는 전체 4KB 블록을 할당하지 않고, 필요한 만큼만 서브블록을 할당한다. 예를 들어, 1KB 파일이라면 512 바이트 서브블록 2 개만 사용해서 저장한다. 이렇게 하면 작은 파일 때문에 큰 블록 전체가 낭비되는 문제를 줄일 수 있다.

파일이 점점 커지면 상황이 달라진다. 서브 블록 여러 개를 사용하다가 파일 크기가 4KB 에 도달하면, FFS 는 이제 일반 4KB 블록을 할당하고 기존 서브블록에 있던 데이터를 그 블록으로 복한다. 이후부터는 다시 큰 블록 단위로 관리된다.

이 방식은 추가적인 복사 작업 때문에 비효율적으로 보일 수 있다.

실제로 디스크 I/O 도 더 발생할 수 있다. 하지만 FFS 설계자들도 이 문제를 알고 있었기 때문에 이를 완화하는 방법을 사용했다. 예를 들어, libc 라이브러리를 수정하여, 애초에 4KB 단위로 쓰기 요청이 모이도록 버퍼링했다. 즉, 작은 서브 블록이 계속 생성되는 상황 자체를 줄이도록 만든 것이다.

디스크 배치 최적화 문제

FFS가 해결할 또 다른 중요한 문제는 디스크의 물리적 특성과 관련된 성능 문제였다. 당시 디스크는 오늘날처럼 스마트하지 않았고, CPU가 디스크 동작을 더 직접적으로 제어하는 구조였다.

문제는 순차적으로 블록을 읽을 때 발생한다. 예를 들어, 블록 0을 읽고 바로 블록 1을 읽으려고 하면, 디스크 헤드가 이미 블록 1을 지나쳐 버리는 경우가 있었다.

이 경우 블록 1을 0리키 위해 디스크를 한 바퀴 더 돌아야 했다. 이것은 매우 큰 성능 손실이었다.

FFS는 이를 해결하기 위해 블록을 “약간 바껴서 배치하는 방법”을 사용했다.

예를 들어, 연속된 블록을 실제 디스크에 하나씩 건너뛰어 배치하는 방식이다. 이렇게 하면 블록 0을 읽은 뒤 블록 1을 요청했을 때, 디스크가 다시 돌아오는 시간 동안 블록 1이 헤드 아래로 도착하게 된다.

즉, 물리적 회전을 고려해서 논리적 순서와 물리적 배치를 일부러 어긋나게 만든 것이다.

이 기법은 디스크의 성능 특성을 측정해서 파라미터로 설정하는 방식으로 구현되었고, 이를 FFS에서는 매개화(parameterization)라고 불렀다.

이 방식은 단순해 보이지만 단점도 있었다. 잘못 설계하면 최대 성능의 절반 정도만 사용하는 상황이 발생할 수 있었다. 왜냐하면 블록을 읽을 때마다 추가 회전이 발생할 수 있기 때문이다.

하지만 현대 디스크는 이 문제를 내부적으로 해결한다. 디스크 자체에 트랙 캐시(트랙 버퍼)가 있어서, 한 트랙 전체를 미리 읽어 저장해 둔다. 그래서 연속된 읽기는 디스크 내부 캐시에서 처리된다. 이로 인해 파일 시스템이 이런 물리적 세부 사항을 신경 쓸 필요가 줄어들었다.

추가 기능: 사용성 개선

FFS는 성능뿐 아니라 사용자 편의성도 크게 개선했다.

먼저 긴 파일 이름을 지원했다. 이전 파일 시스템은 보통 8자리 같은 고정 길이 이름만 허용했지만, FFS는 더 긴 이름을 사용할 수 있게 했다. 이는 사람이 이해하기 쉬운 파일 구조를 만드는 데 큰 도움이 되었다.

또한 심볼릭 링크(symbolic link)를 도입한 최초의 파일 시스템 중 하나였다.

기존 하드 링크는 몇 가지 제약이 있었다.

반면 심볼릭 링크는 파일이나 디렉터리를 “이름으로 참조하는 별명”을 만들 수 있어서 훨씬 유연하다.

추가로 rename() 시스템 콜도 제공했다. 이는 파일 이름 변경을 원자적으로 수행할 수 있게 해주는 기능으로, 중간 상태 없이 안전하게 이름을 변경할 수 있게 한다.

여담: 심볼릭 링크

심볼릭 링크는 특정 파일이나 디렉터리의 “경로”를 저장해서, 해당 위치로 연결해주는 파일이다.

즉, 실제 데이터를 가지지 않고 다른 대상을 가리키는 참조 파일이다.

동작 방식

- 링크 파일 안에는 실제 데이터가 아니라 대상 경로가 저장
- 사용자가 링크에 접근하면, 운영체제가 해당 경로로 이동해서 원본 파일을 열어줌
- 예: link.txt -> /hom/user/a.txt 를 가리킴

사용 목적

(1) 편의성

- 긴 경로를 짧게 줄여서 사용
- 자주 쓰는 파일/프로그램 접근을 쉽게 함

(2) 버전 관리

- current 같은 링크를 만들어 최신 버전 폴더를 가리키게함
- 실제 경로를 바꾸지 않고 링크만 변경하면 됨

(3) 공유 및 재사용

- 하나의 파일을 여러 위치에서 동일하게 참조 가능
- 파일 복사 없이 재사용 가능

(4) 구조 정리

- 복잡한 디렉터리 구조를 단순한 경로로 추상화

특징

- 원본 파일이 삭제되면 링크는 깨짐
- 다른 파일 시스템도 참조 가능
- 디렉터리도 링크 가능
- 실제 데이터가 아닌 경로 기반 참조

하드링크와 차이

- 심볼릭 링크: 경로를 저장(원본 의존)
- 하드링크: inode 를 공유(동일 파일처럼 동작)
- 심볼릭 링크는 원본이 없어지면 깨짐
- 하드링크는 원본이 삭제돼도 유지됨

[1-4] 크래시 일관성: FSCK 와 저널링

핵심 질문: 크래시에도 불구하고 디스크 갱신하기

두 쓰기 동작 사이에 시스템은 크래시 되거나 끊어질 수도 있기 때문에 디스크 상의 상태는 부분적으로만 갱신이 될 수도 있다.

크래시 이후에 시스템이 재구동되면 파일 시스템을 다시 마운트하려고 할 것이다. (파일 접근 등의 동작을 위해서)

임의의 시간에 크래시가 발생할 수 있다고 하면 파일 시스템이 어떻게 해야 디스크 상의 자료를 올바른 상태로 유지할 수 있을까?

이제까지 본 바와 같이, 파일 시스템은 그 기본 개념들을 구현하는 데 필요한 각종 자료 구조들을 관리한다.

이 자료 구조에는 파일, 디렉터리, 그 외에 각종 메타데이터들이 있다.

여타 자료 구조와는 다르게(예를 들어, 실행 중인 프로그램들이 사용하는 메모리 상의 자료 구조) 파일 시스템의 자료 구조는 안전하게 저장되어야 한다.

즉, 장시간 사용 후에도 유지되어야 하며 전력 손실에도 하드 디스크나 플래시 기반 SSD 장치의 데이터는 손상없이 유지되어야 한다.

파일 시스템이 가진 가장 큰 어려움은 전력 손실이나 시스템 크래시가 발생하는 상황에서도, 어떻게 안전하게 디스크 상의 내용을 갱신하는 가에 대한 문제이다.

만약 디스크 자료 구조를 갱신하는 도중에 누군가 전원 선에 걸려 넘어져서 기계가 꺼졌다면 어떻게 되는가? 운영체제가 버그 때문에 멈춘 경우는 어떻게 하는가? 전력 손실이나 크래시 때문에 디스크 상의 자료 구조를 안전하게 갱신하는 것은 상당히 까다로운 작업이 된다.

파일 시스템은 크래시 일관성(crash-consistency)이라고 새롭고 흥미로운 문제에 직면하게 된다.

문제를 이해하는 것은 어렵지 않다.

어떤 특정 작업을 위해 디스크 상에서 두 개의 자료 구조 A 와 B 를 갱신해야 한다고

해보자. 디스크는 한 번에 하나의 요청만 처리할 수 있기 때문에 두 요청 중 하나의 요청이 먼저 디스크에 도달할 것이다.

하나의 쓰기 작업만 완료한 상태에서 시스템 전원이 나간 경우, 디스크 상의 자료 구조는 일관성이 깨지게 된다. 이러한 특성 때문에, 파일 시스템에서는 이제까지는 없었던 새로운 문제를 직면한다.

크래시 일관성 문제이다.

이번 장에서는 이 문제에 대해서 좀 더 자세히 다룰 것이다.

이에 대한 해결책들을 살펴보기로 한다. 먼저 과거 파일 시스템이 사용했던 fsck 또는 파일 시스템 검사기라는 방법을 살펴보도록 하겠다. 그 다음, 저널링(journaling) 또는 write-ahead logging(WAL)이라는 다른 기법을 살펴보겠다.

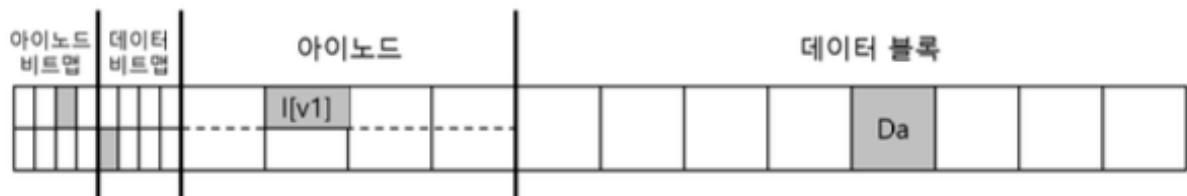
저널링이나 WAL 기법은 쓰기 오버헤드가 추가되기는 하지만 전력 손실이나 크래시 상황으로부터 좀 더 빠르게 복구할 수 있다. Linux ext3 가 구현하고 있는 몇 가지 기본적인 저널링 기법들을 살펴볼 것이다.

[1] 예제 1

이 예제는 “기존 파일에 데이터를 하나 더 추가하는 상황”을 다룬다.

구체적으로는 파일을 열고, 끝으로 이동한 다음, 4KB 데이터를 하나 더 쓰는 작업이다. 즉, 파일 크기가 1 블록에서 2 블록으로 늘어나는 상황이다.

파일 시스템 내부 상태에서 보면, 현재 파일은 하나의 데이터 블록만 사용하고 있다. 이 파일은 아이노드 2 번에 해당하며, 데이터 블록 4 번을 가리키고 있다. 아이노드에는 파일 크기가 1로 기록되어 있고, 첫 번째 포인터만 유효하며 나머지는 비어 있다.



이 상태에서 파일 끝에 데이터를 추가하면 어떤 일이 발생할까?

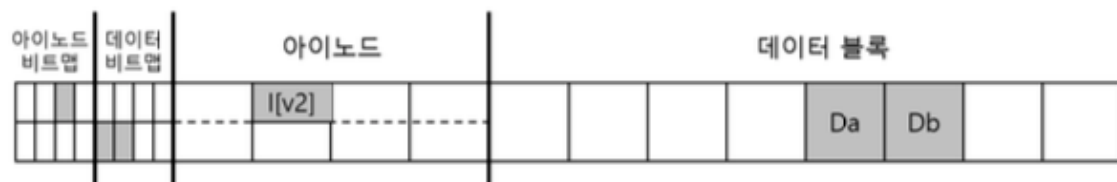
단순히 데이터를 쓰는 것처럼 보이지만, 실제로는 파일 시스템 내부의 여러 구조를 동시에 바꿔야 한다.

먼저 새로운 데이터 블록이 필요하다. 예를 들어 5 번 블록을 새로 할당한다고 하면, 이 블록에는 사용자가 쓴 데이터가 저장된다. 동시에 데이터 비트맵도 수정해야 한다. 5 번 블록이 이제 “사용 중”이라는 것을 표시해야 하기 때문이다.

그리고 아이노드도 수정된다. 파일 크기가 1 에서 2 로 증가해야 하고, 두 번째 포인터가 새로 할당된 5 번 블록을 가리키도록 변경되어야 한다.

정리하면, 이 단순한 write 작업 하나를 처리하기 위해 실제로는 세 가지가 바뀐다.

- 데이터 블록 하나(새로운 내용 저장)
- 데이터 비트맵(블록 사용 표시)
- 아이노드(크기 증가 + 포인터 추가)



이제 중요한 부분이 나온다.

이 세가지는 각각 따로 디스크에 기록된다. 즉, 한 번에 동시에 저장되는 것이 아니라, 순차적으로 각각 write 가 발생한다.

그런데 이 write 들은 즉시 디스크에 기록되지 않는다. 운영체제는 성능을 위해 이 변경들을 메모리에 잠시 저장해 두고, 나중에 한꺼번에 디스크에 반영한다. 보통 몇 초 뒤에 기록된다.

문제는 여기서 발생한다.

만약 디스크에 기록하는 도중에 시스템이 크래시되면 어떻게 될까?

세 개 중 일부만 기록되고 나머지는 기록되지 않을 수 있다. 이 경우 파일 시스템 상태가 깨진다.

크래시 시나리오

먼저 전제부터 다시 정리하자. 파일에 블록 하나를 추가하는 작업에서는 다음 세가지가 바뀐다.

- 새로운 데이터 블록(Db)
- 갱신된 아이노드(I[v2])
- 갱신된 데이터 비트맵(B[v2])

이 세개가 모두 디스크에 기록되어야만 파일 시스템은 정상 상태가 된다. 그런데 크래시가 발생하면 일부만 기록될 수 있다. 이때 어떤 문제가 생기는지를 하나씩 보자.

1. 하나만 쓰기만 성공한 경우

(1) 데이터 블록(Db)만 기록된 경우

이 경우 디스크에 새로운 데이터가 존재한다. 하지만 아이노드는 여전히 이전 상태이기 때문에 이 블록을 가리키지 않는다. 또한 비트맵도 이 블록이 사용 중이라는 사실을 반영하지 않는다.

결과적으로 이 블록은 “어느 파일에도 속하지 않는 데이터”가 된다. 파일 시스템 입장에서 보면 이 블록은 존재하지만 접근할 방법이 없다. 이런 상황은 공간이 낭비되는 문제로 이어진다.

중요한 점은, 이 경우에는 구조 간 충돌은 없다. 즉, 아이노드와 비트맵은 서로 모순되지 않는다. 다만 “쓸 수 없는 데이터”가 남는 것이 문제다.

(2) 아이노드(I[v2])만 기록된 경우

이 경우가 훨씬 위험하다.

아이노드는 이미 “파일 크기가 증가했고, 새로운 블록(예: 5 번)을 사용한다”고 기록되어 있다. 즉 파일을 읽으면 두 번째 블록으로 5 번을 참조하게 된다.

하지만 실제로는 데이터 블록 5 번에 새로운 데이터(Db)가 기록되지 않았다. 따라서 그 위치에는 이전에 있던 쓰레기 값이나 다른 데이터가 남아 있을 수 있다.

결과적으로 파일을 읽으면 잘못된 데이터를 읽게 된다.

여기에 더 큰 문제가 하나 더 있다.

비트맵은 아직 5 번 블록이 “사용 중이 아니다”라고 표시하고 있다. 반면 아이노드는 “사용 중이다”고 말한다.

즉,

- 아이노드: 이 블록을 사용 중
- 비트맵: 이 블록은 비어 있다.

이 두 구조가 서로 충돌한다. 이것이 바로 파일 시스템의 일관성 손상(inconsistency)이다.

(3) 비트맵(B[v2])만 기록된 경우

이 경우 비트맵은 5 번 블록이 “사용 중”이라고 표시한다. 하지만 아이노드는 여전히 그 블록을 가리키지 않는다.

즉,

- 비트맵: 이 블록은 사용 중
- 아이노드: 나는 이 블록을 사용하지 않는다

결과적으로 이 블록은 어떤 파일에도 연결되지 않지만, 시스템은 이미 사용 중이라고 생각한다.

이 상태에서는

- 해당 블록은 다시 할당되지 않는다.
- 실제로는 비어 있는데도 계속 못 쓰는 상태가 된다.

즉, 공간이 점점 낭비되는 문제가 발생한다.

2. 두 개의 쓰기만 성공한 경우

이번에는 세 개 중 두 개만 기록된 경우다.

(1) 아이노드 + 비트맵 기록, 데이터(Db)는 없음

이 겨웅 메타데이터는 서로 일치한다.

- 아이노드: 블록 5 사용
- 비트맵: 블록 5 사용 중

겉보기에는 완벽하다. 파일 시스템 입장에서는 문제가 없어 보인다. 하지만 실제 데이터 블록에는 우리가 쓰려고 했던 내용이 없다. 이전 데이터나 쓰레기 값이 들어 있다.

결과

- 파일을 읽으면 잘못된 데이터가 나온다.

즉, 구조적으로는 정상인데 내용이 깨진 상태다.

(2) 아이노드 + 데이터(Db) 기록, 비트맵은 없음

이 경우 아이노드는 5 번 블록을 가리키고 있고, 데이터도 실제로 존재한다. 하지만 비트맵은 여전히 그 블록을 “비어 있음”으로 표시하고 있다.

즉,

- 아이노드: 이 블록 사용 중
- 비트맵: 이 블록 사용 가능

이 상태에서는 심각한 문제가 발생할 수 있다.

파일 시스템은 나중에 다른 파일을 만들 때 이 블록을 “빈 블록”이라고 생각하고 다시 할당할 수 있다. 그러면 하나의 블록을 두 파일이 동시에 사용하게 된다.

이것은 데이터 덮어쓰기, 파일 손상 등 매우 심각한 오류로 이어진다.

(3) 비트맵 + 데이터(Db) 기록, 아이노드는 없음

이 경우 데이터는 존재하고, 비트맵도 해당 블록을 사용 중이라고 표시한다. 하지만 아이노드는 그 블록을 가리키지 않는다.

결과적으로

- 블록은 사용 중인데
- 어떤 파일에도 연결되지 않음

이 경우는 일종의 “유실된 데이터 블록”이다. 공간은 차지하고 있지만 접근할 수 없다.

이게 바로 저널링이 등장하는 이유다.

파일 시스템은 단순히 데이터를 저장하는 것이 아니라, 여러 구조를 “일관성 있게” 유지해야 한다. 그런데 디스크는 원자적으로 여러 블록을 한 번에 쓰지 못한다. 따라서 중간에 실패하면 상태가 깨질 수 밖에 없다. 그래서 저널링 파일 시스템은 이런 문제를 해결하기 위해 “먼저 변경 내용을 기록해 두고, 그 다음 실제 데이터를 반영하는 방식”을 사용한다.

크래시 일관성 문제

앞에서 살펴본 것처럼, 파일 시스템은 하나의 작업을 수행할 때 실제로는 여러 개의 디스크 구조를 함께 갱신한다. 예를 들어, 파일에 데이터를 추가하는 경우에도 데이터 블록, 아이노드, 비트맵이 동시에 변경되어야 한다.

문제는 디스크가 이런 여러 변경을 “한 번에” 처리하지 못하는데 있다. 디스크는 기본적으로 하나의 블록씩 순차적으로 기록한다.

따라서 세 개를 바꿔야 하면 세 번의 쓰기가 따로 발생한다.

이 과정에서 시스템이 정상적으로 끝까지 실행되면 문제가 없다. 하지만 중간에 크래시가 발생하면 일부만 기록되는 상태로 멈추게 된다. 그러면 파일 시스템은 완전한 상태(이전 상태 또는 이후 상태)가 아니라, 그 사이에 있는 어중간한 상태에 놓이게 된다.

이 중간 상태를 transient state(과도 상태)라고 한다. 원래는 아주 짧은 시간 동안만 존재해야 하는 상태인데, 크래시 때문에 디스크에 그대로 남아버리는 것이다.

이렇게 되면 여러 문제가 발생한다.

- 아이노드와 비트맵이 서로 다른 내용을 가리키는 불일치
- 실제로는 사용되지 않는 블록이 계속 점유되는 공간 누수
- 파일을 읽었을 때 잘못된 데이터가 반환되는 문제

이처럼 파일 시스템 내부 구조들이 서로 맞지 않는 상태가 되면, 전체 파일 시스템의 신뢰성이 깨진다.

그래서 파일 시스템이 달성하려는 목표는 명확하다.

어떤 연산을 수행할 때,

- 아예 반영되지 않은 상태(이전 상태)이거나
- 완전히 반영된 상태(이후 상태)

이 둘 중 하나만 존재하도록 만드는 것이다.

즉, 중간 상태가 디스크에 남지 않도록 해야 한다.

하지만 이 목표를 달성하기 어려운 이유는 디스크의 특성 때문이다.

디스크는

- 여러 블록을 동시에 기록하지 못하고
- 각 쓰기 사이에 시간이 존재하며
- 그 사이에 언제든지 크래시가 발생할 수 있다.

따라서 “항상 완전한 상태만 유지한다”는 것은 자연스럽게 보장되지 않는다.

이러한 문제를 일반적으로 크래시 일관성 문제(crash consistency problem)라고 부른다.
다르게 말하면, 여러 구조를 함께 업데이트해야 하는데, 일부만 반영되는 상황을 어떻게 막을 것인가에 대한 문제이다.

[2] 해법 1: 파일 시스템 검사기(fsck)

초기의 파일 시스템은 크래시 일관성 문제를 “미리 막는 방식”이 아니라, 문제가 생긴 뒤에 고치는 방식으로 접근했다.

즉, 시스템이 크래시되더라도 일단 그대로 두고, 다시 부팅할 때 파일 시스템 전체를 검사해서 틀어진 부분을 찾아 수정하는 전략이다.

이때 사용하는 대표적인 도구가 fsck(file system check)이다.

기본 아이디어

fsck의 핵심 개념은 단순하다.

- 디스크에 있는 모든 파일 시스템 구조를 전부 읽는다.
- 서로 맞지 않는 부분을 찾아낸다.
- 가능한 범위에서 “일관성 있는 상태”로 복구한다.

중요한 점은 fsck는 파일 내용이 아니라 구조의 일관성을 맞춘다는 것이다.

즉, 데이터가 맞는지까지는 보장하지 못하고 “파일 시스템이 깨지지 않도록” 만드는 것이 목표이다.

fsck가 실행되는 시점

fsck는 파일 시스템이 마운트되기 전에 실행된다. 이때는 파일 시스템이 아무 작업도 하지 않는 상태라고 가정한다.

검사가 끝나야만 파일 시스템이 정상적으로 사용 가능해진다.

fsck 가 하는 일들

fsck 는 파일 시스템 전체를 훑으면서 여러 단계를 거쳐 검사하고 수정한다.

(1) 슈퍼블럭 검사

먼저 파일 시스템의 가장 기본 정보인 슈퍼블럭을 검사한다.

예를 들어,

- 블록 개수가 실제 디스크 크기보다 큰지
- 구조 정보가 이상하지 않은지

이상이 발견되면

- 백업된 슈퍼블럭으로 교체할지 결정한다.

(2) 프리 블럭(비트맵) 재구성

다음으로 중요한 작업은 “어떤 블록이 사용 중인지”를 다시 계산하는 것이다.

방법은 이렇다.

- 모든 아이노드를 읽는다.
- 각 아이노드가 가리키는 데이터 블록을 추적한다.
- 그 결과를 기반으로 “실제 사용 중인 블록 목록”을 만든다.

그리고

- 기존 비트맵과 비교한다.
- 틀리면 아이노드 기준으로 비트맵을 다시 만든다.

아이노드 비트맵도 같은 방식으로 검사하고 필요하면 재구성한다.

(3) 아이노드 상태 검사

각 아이노드가 정상인지 확인한다.

예를 들어,

- 파일인지 디렉터리인지 타입이 정상인지
- 값이 깨져 있지 않은지

문제가 심각하면

- 해당 아이노드를 포기하고 초기화한다.

이 경우 파일은 사실상 사라진다.

(4) 링크 개수 검사

아이노드에는 “링크 개수”가 기록되어 있다.

이 값은 해당 파일을 참조하는 디렉터리 엔트리 수를 의미한다.

fsck 는 이를 검증하기 위해

- 루트부터 시작해서 모든 디렉터를 순회한다.
- 실제 링크 개수를 직접 계산한다.
- 아이노드에 기록된 값과 비교한다.

다르면:

- 아이노드의 값을 수정한다.

만약:

- 어떤 아이노드가 존재하는데
- 아무 디렉터리에서도 참조하지 않는다면

그 파일은 lost + found 디렉터리로 이동된다.

(5) 중복 블록 검사

두 개 이상의 아이노드가 같은 데이터 블록을 가리키는 경우를 검사한다.

이 상황은 매우 위험하다.

서로 다른 파일이 같은 데이터를 공유하게 되기 때문이다.

해결 방법은 두 가지다.

- 문제가 있는 아이노드를 제거하거나
- 블록을 복사해서 각 파일이 따로 사용하게 만든다.

(6) 배드 블록 검사

포인터가 이상한 위치를 가리키는 경우를 검사한다.

예:

- 디스크 범위를 벗어난 주소

이 경우:

- 해당 포인터를 제거한다.

즉, 데이터는 일부 손실될 수 있다.

(7) 디렉터리 구조 검사

디렉터리는 구조가 정해져 있기 때문에 추가 검사가 가능하다.

fsck 는 다음을 확인한다.

- “.”, “..” 항목이 있는지
- 디렉터리가 정상적인 아이노드를 가리키는지
- 디렉터리 구조에 이상한 순환이 없는지

근본적인 한계

여기까지 보면 fsck 는 굉장히 강력해보이지만, 중요한 한계가 있다.

(1) 완벽하지 않다.

예를 들어, 아이노드와 비트맵이 서로 맞고 구조적으로 문제없어 보이지만 실제 데이터가 잘못된 경우는 fsck 도 잡지 못한다.

즉, “일관성”은 맞추지만 “정확성”은 보장하지 못한다.

(2) 매우 느리다.

갖아 큰 문제는 성능이다.

fsck 는 디스크 전체를 다 읽어야 하고 모든 아이노드와 디렉터리를 검사해야 한다.

그래서 작은 디스크에서는 몇 분, 큰 디스크에서는 몇 시간까지 걸릴 수 있다.

디스크 용량이 커질수록 이 문제는 더 심각해진다.

(3) 비효율적인 방식

이 방식은 본질적으로 비효율적이다.

문제 상황을 보면:

- 단지 몇 개의 블록만 잘못 기록되었는데 이를 고치기 위해 디스크 전체를 검사한다.

비유하면

- 방에서 열쇠를 잃어버렸는데 집 전체를 뒤지는 것과 같다.

가능하긴 하지만 비효율적이다.

[3] 해법 2: 저널링(또는 Write-Ahead Logging)

저널링은 파일 시스템이 크래시 상황에서도 일관성을 유지하기 위해 사용하는 핵심 기법이다. 핵심 아이디어는 단순하다.

실제 데이터를 디스크의 “원래 위치”에 바로 쓰지 않고, 먼저 “로그(저널)”에 무엇을 쓸지 기록한 뒤, 그 다음에 실제 데이터를 반영하는 방식이다.

이렇게하면 도중에 시스템이 죽어도, 로그를 다시 복구할 수 있다.

1. 기본 개념: 왜 “먼저 기록”하는 가

파일 시스템에서 하나의 작업은 보통 여러 구조를 동시에 바꾼다.

예를 들어, 파일에 데이터를 추가하면

- 아이노드(파일 크기, 포인터 변경)
- 데이터 비트맵(블록 사용 표시)
- 실제 데이터 블록

이 세 가지가 모두 바뀐다. 문제는 디스크 한 번에 하나씩만 기록하기 때문에, 중간에 크래시가 나면 상태가 꼬인다.

그래서 해결책은

- 첫째, “이 작업에서 바꿀 내용”을 로그에 먼저 기록
- 둘째, 로그가 안전하게 기록되면
- 셋째, 실제 위치에 데이터를 반영

이 순서를 반드시 지키는 것이 핵심이다.

2. 저널(로그)의 구조

하나의 작업(트랜잭션)은 보통 다음과 같은 형태로 기록된다.

- TxB(Transaction Begin): 작업 시작, 무엇을 바꿀지 정보 포함
- 데이터/메타데이터 블록들: 실제로 바뀔 내용들
- TxE(Transaction End): 작업 완료(커밋)

즉, 로그는 “이 작업은 이런 내용으로 끝까지 정상 수행됐다”는 것을 보장하는 기록이다.

3. 정상 동작 흐름

파일에 블록 하나를 추가하는 예를 기준으로 보면:

- 첫째, 저널 쓰기 단계
 - TxB + 변경될 블록들(아이노드, 비트맵, 데이터)을 로그에 기록
 - 이게 디스크에 완전히 기록될 때까지 기다림
- 둘째, 커밋 단계
 - TxE(트랜잭션 종료 블록)를 기록
 - 이 시점에서 “이 작업은 안전하게 완료됨”이라고 간주
- 셋째, 체크포인트 단계
 - 실제 디스크의 원래 위치(아이노드 영역, 데이터 영역 등)에 반영

4. 크래시가 나면 어떻게 되는가

재부팅 후 파일 시스템은 저널만 확인하면 된다.

- TxB 는 있는데 TxE 가 없으면 -> 미완료 작업 -> 무시
- TxB 와 TxE 가 모두 있으면 -> 완료된 작업 -> 다시 반영(redo)

즉, 디스크 전체를 검사할 필요 없이 “로그에 있는 작업만 처리하면 된다”는 것이 핵심 장점이다.

5. 중요한 문제: 순서 꼬임

디스크는 내부적으로 요청 순서를 바꾸서 처리할 수 있다.

그래서 다음과 같은 위험이 생긴다.

- TxB, 메타데이터, TxE 는 기록됨
- 하지만 데이터 블록(Db)은 아직 안 써짐
- 그런데 TxE 가 있으므로 “완료된 작업”으로 오해됨

이 상태에서 복구하면 잘못된 데이터가 정상 데이터처럼 복구된다.

6. 해결 방법: 2 단계 쓰기

이 문제를 막기 위해 저널은 두 단계로 나눠서 기록한다.

- 첫째, TxE 를 제외한 모든 블록 먼저 기록
- 둘째, 그 다음에 TxE 를 따로 기록

이렇게 하면

- TxE 가 있다 -> 모든 데이터가 이미 기록된 상태
- TxE 가 없다 -> 불완전한 작업 -> 무시

7. 핵심 조건: TxE 의 원자성

TxE(트랙잭션 종료 블록)는 반드시 원자적으로 기록되어야 한다.

- 디스크는 보통 512 바이트 단위 쓰기 원자성 보장
- 따라서 TxE 는 반드시 한 블록 안에 들어가야한다.

그래야 “절만만 써진 TxE”같은 상황이 발생하지 않는다.

크래시 시 복구의 기본 원리

크래시는 언제든 발생할 수 있다. 크게 두 경우로 나눠서 생각하면 이해가 쉽다.

(1) 트랜잭션이 완전히 기록되기 전에 크래시

즉, 저널에 기록한 과정(특히 TxE 까지)이 끝나기 전에 시스템이 죽은 경우다.

- 이 경우 해당 트랜잭션은 불완전한 상태
- 복구 시에는 아무것도 하지 않고 무시

왜냐하면 “완료되지 않은 작업”이기 때문이다.

(2) 트랜잭션은 커밋됐지만 체크포인트 전 크래시

즉,

- 저널에는 TxB ~ TxE 까지 다 기록됨(커밋 완료)
- 하지만 실제 디스크의 원래 위치에는 아직 반영 안됨

이 경우 복구는 다음과 같이 한다.

- 저널을 스캔해서 커밋된 트랜잭션을 찾는다.
- 해당 블록들을 원래 위치에 다시 써준다(replay)

이 과정을 redo logging 이라고 한다.

(3) 체크포인트 중 크래시가 나도 괜찮은 이유

체크포인트는 “저널 -> 실제 위치로 복사”하는 단계다.

이때 크래시가 나면:

- 일부 블록만 반영된 상태일 수 있음

하지만 문제 없다.

- 복구 시 다시 replay 하면 됨

즉, 여러 번 써도 결과는 동일한다.(이 성질을 idempotent 하다고 본다)

(4) 로그를 묶어서 처리하는 이유(Batching)

문제는 저널링이 쓰기 비용이 크다는 점이다.

예를 들어, 같은 디렉터리에 파일 2 개를 생성하면

- 아이노드 비트맵
- 디렉터리 블록
- 디렉터리 아이노드

같은 블록이 반복해서 수정된다.

이걸 매번 따로 저널에 쓰면 비효율적이다. 그래서 사용하는 방법이:

트랜잭션 버퍼

- 메모리에 변경 사항들을 모아둔다.
- 일정 시간 후 한 번에 저널에 기록(예: 5 초)

또는

- fsync() 호출 시 즉시 기록

이렇게 하면:

- 같은 블록을 여러 번 쓰는 낭비 감소
- 디스크 I/O 감소

(5) 로그 공간 관리(Circular Log)

저널 영역은 크기가 제한되어 있다.

계속 트랜잭션을 기록하면 결국 공간이 꽉 찬다.

이때 발생하는 문제:

- 첫째, 로그가 길어지면 복구 시간이 증가
- 둘째, 더 이상 새로운 작업을 기록 못함 → 시스템 정지

해결 방법: 환형 로그

- 로그를 원형 버퍼처럼 사용
- 끝까지 쓰면 다시 처음으로 돌아감

단 조건이 있다.

- 이미 체크포인트 완료된 트랜잭션만 덮어쓰기 가능

그래서 파일 시스템은

- 가장 오래된 트랜잭션 위치
- 가장 최신 트랜잭션 위치

를 따로 관리한다.

(6) 최종 저널링 프로토콜 정리

전체 흐름은 다음 4 단계다.

1. 저널 쓰기: TxB + 데이터/메타데이터 기록
2. 저널 커밋: TxE 기록 → “안전”
3. 체크포인트: 실제 디스크 위치에 반영
4. 해제: 저널 공간을 다시 사용 가능하게 표시

(7) 데이터 저널링의 한계

문제는 성능이다.

- 모든 데이터를 두 번 씬(저널에 한 번, 실제 위치에 한 번)

특히 대용량 데이터에서는

- 디스크 대역폭 절반 손실
- 탐색 비용 증가

(8) 해결책: 메타데이터 저널링(Ordered journaling)

- 데이터는 저널에 안 쓴다
- 메타데이터만 저널링한다

그런데 새로운 문제가 생긴다.

아이노드(I[v2])는 데이터 블록 Db 를 가리킨다.

만약:

- 아이노드는 기록됨
- 데이터(Db)는 아직 디스크에 없음

이 상태에서 크래시 발생하면 아이노드가 “쓰레기 데이터”를 가리키게 된다.

(9) 해결 원칙: 쓰기 순서 보장

- 포인터보다 대상 데이터를 먼저 써라
- 즉,

1. 데이터 블록 먼저 디스크에 기록
2. 그 다음 메타데이터를 저널에 기록
3. 마지막으로 커밋

Ordered journaling 의 실제 순서

1. 데이터 블록(Db) 디스크에 쓰기
2. 메타데이터를 저널에 기록
3. TxE 기록(커밋)
4. 메타데이터를 실제 위치에 반영
5. 로그해제

[1-5] 로그 기반 파일 시스템

핵심 질문: 모든 쓰기를 어떻게 순차 쓰기로 변형시킬까

- 어떻게 하면 모든 쓰기를 순차 쓰기로 변형시킬 수 있을까?
- 읽기 대상이 디스크 상에 흩어져 있을 가능성이 있기 때문에, 읽기 동작을 순차적으로 변형하는 것은 원천적으로 불가능하다. 하지만, 쓰기의 경우 파일 시스템이 쓰기 위치를 선택할 수 있다. 선택의 여지를 최대한 활용해 보자.

로그 기반 파일 시스템(LFS)이 왜 등장했고 어떤 방식으로 동작하는지를 제대로 이해하려면, 먼저 기존 파일 시스템이 어떤 한계를 가지고 있었는지를 차분히 짚고 넘어가는 것이 중요하다.

1990년대 초반, Berkeley 대학의 John Ousterhout와 Mendel Rosenblum은 기존 파일 시스템의 성능 문제를 분석하면서 몇 가지 중요한 관찰을 하게 된다.

첫 번째는 메모리의 변화이다.

당시 컴퓨터의 메모리 용량이 점점 커지고 있었고, 그 결과 많은 파일 데이터가 디스크가 아니라 메모리에 캐시되기 시작했다.

이는 파일 시스템의 사용 패턴 자체를 바꾸었다. 예전에는 읽기와 쓰기가 모두 디스크 중심이었다면, 이제는 대부분의 읽기 요청이 메모리에서 처리되고, 디스크는 주로 쓰기 작업을 담당하게 되었다.

즉, 파일 시스템의 성능은 점점 “쓰기 성능”에 의해 결정되기 시작한 것이다.

두 번째는 디스크의 물리적 특성과 관련된 문제다.

디스크의 데이터 전송 속도는 매년 빠르게 증가했지만, 탐색 시간(seek time)과 회전 지연(rotational latency)은 그에 비해 매우 느리게 개선되었다.

이 차이는 시간이 갈수록 더 커졌다.

그 결과, 디스크를 순차적으로 접근할 때와 임의(random)로 접근할 때의 성능 차이가 매우 크게 벌어졌다.

순차 접근은 매우 빠른 반면, 랜덤 접근은 여전히 느린 상태로 남게 된 것이다.

세 번째는 기존 파일 시스템 자체의 구조적인 비효율이다.

예를 들어, FFS와 같은 파일 시스템에서는 아주 작은 작업 하나를 처리하기 위해서도 여러 번의 디스크 쓰기가 필요하다.

파일 하나를 생성하는 경우를 생각해 보면, 아이노드 비트맵 갱신, 아이노드 생성, 디렉터리 데이터 블록 수정, 디렉터리 아이노드 갱신, 실제 데이터 블록 기록, 데이터 비트맵 갱신 등 다양한 구조를 각각 따로 수정해야 한다.

이 작업들은 물리적으로 서로 떨어진 위치에 있기 때문에, 디스크 헤드는 계속 이동해야 하고 매번 회전 지역이 발생한다.

결국 전체 작업은 “짧은 랜덤 쓰기들의 연속”이 되고, 이는 디스크가 낼 수 있는 순차 성능에 비해 매우 비효율적이다.

네 번째는 RAID 환경과의 부조화이다.

RAID-4 나 RAID-5 같은 구조에서는 작은 쓰기 하나를 수행하더라도 실제로는 여러 번의 디스크 I/O 가 발생한다.

기존 파일 시스템은 이러한 구조를 고려하지 않고 설계되었기 때문에, 작은 단위의 쓰기가 많은 워크로드에서는 성능이 크게 떨어질 수 밖에 없었다.

이러한 문제들을 종합하면, 이상적인 파일 시스템은 다음과 같은 방향을 가져야 한다는 결론에 도달한다.

첫째, 쓰기 성능을 최우선으로 고려해야 한다.

둘째, 디스크의 순차 대역폭을 최대한 활용해야 한다.

셋째, 메타데이터와 같은 빈번한 갱신 작업도 효율적으로 처리해야 한다.

넷째, RAID 환경에서도 잘 동작해야 한다.

이러한 요구를 바탕으로 제안된 것이 로그 기반 파일 시스템, 즉, LFS 이다.

LFS 의 핵심 아이디어는 매우 단순하지만 기존 방식과는 근본적으로 다르다.

기존 파일 시스템은 데이터를 “제자리에서 갱신(in-place update)”하는 방식을 사용한다.

즉, 기존 블록을 읽고 수정한 뒤 다시 같은 위치에 덮어쓴다.

반면, LFS 는 이 방식을 완전히 버린다.

대신 모든 변경 사항을 “로그 처럼”취급한다.

즉, 변경된 데이터와 메타데이터를 모아서 디스크의 새로운 빈 공간에 순차적으로 계속 써 내려간다.

구체적인 동작을 보면, 파일 시스템에서는 발생하는 모든 갱신 작업은 먼저 메모리 상의 “세그먼트”라는 구조에 모인다.

이 세그먼트 안에는 단순한 데이터뿐만 아니라 아이노드, 비트맵과 같은 메타데이터까지 함께 포함된다.

일정량이 모여서 세그먼트가 충분히 커지면, 파일 시스템은 디스크 상의 빈 공간을 찾아서 이 세그먼트를 한 번에 기록한다.

여기서 중요한 점은 두가지다.

하나는 “한 번에 큰 단위로 쓴다”는 것이고, 다른 하나는 “항상 새로운 위치에 쓴다”는 것이다.

기존 방식처럼 작은 단위로 여러 번 나누어 쓰지 않고, 큰 덩어리로 묶어서 쓰기 때문에 디스크는 순차 쓰기 성능을 최대한 활용할 수 있다. 또한 기존 데이터를 덮어쓰지 않기 때문에 디스크 헤드가 여기저기 이동할 필요가 줄어든다.

이러한 구조 덕분에 LFS 는 디스크를 거의 이상적인 순차 장치처럼 사용할 수 있게 된다.

결과적으로 랜덤 쓰기를 순차 쓰기로 변환하는 효과를 얻고, 이는 특히 쓰기 중심 워크로드에서 큰 성능 향상을 가져온다.

정리하면, LFS는 “데이터를 어디에 어떻게 배치할 것인가”라는 기존 파일 시스템의 관점으로부터 바뀐 설계다.

기존 시스템이 위치 기반 갱신에 초점을 맞췄다면, LFS는 시간 순서대로 기록하는 로그 구조를 기반으로 한다. 이 차이가 디스크의 물리적 특성과 맞물리면서 성능 측면에서 큰 차이를 만들어낸다.

[1] 디스크에 순차적으로 쓰기

이 부분의 핵심은 LFS(Log-Structured File System)가 기존 파일 시스템처럼 “데이터를 여기저기 업데이트 하는 방식”을 버리고, 모든 변경 내용을 디스크에 순차적으로 써 내려가는 방식으로 바꾸려 한다는 점이다.

기존 파일 시스템에서는 파일에 데이터를 쓰면 데이터 블록을 한 위치에 기록하고, 동시에 그 데이터를 가리키는 아이노드도 다른 위치에 따로 갱신하고, 또 비트맵이나 디렉터리 같은 메타데이터도 각각 다른 곳에 흩어져서 수정된다.

이 방식은 구조적으로 랜덤 I/O를 계속 유발하기 때문에 디스크 헤드가 계속 움직이게 되고 성능이 떨어질 수 밖에 없다.

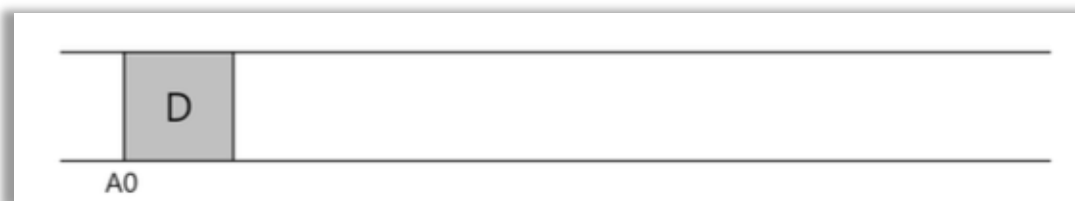
LSF는 여기서 접근 자체를 바꾼다.

예를 들어, 파일에 데이터 블록 D를 쓰는 상황을 보면, 단순히 D만 디스크 어딘가에 쓰는 게 아니라 그 데이터를 가리키게 될 아이노드 변경까지 포함해서 “이 변경에 필요한 모든 것”을 하나의 묶음으로 만든다.

그리고 이 묶음을 디스크의 빈 공간에 그냥 이어서 순서대로 기록한다.

그래서 어떤 경우에는 A0에 데이터 블록 D를 쓰고 바로 다음 위치 A1에 갱신된 아이노드를 같이 기록하는 식으로 진행된다.

중요한 것은 데이터와 메타데이터를 분리해서 각각 다른 위치에 덮어쓰는 게 아니라, 변경된 내용을 통째로 모아서 디스크 끝에 계속 append 하는 방식이라는 점이다.



이 방식의 본질은 “파일 시스템을 업데이트하는 모든 작업을 로그처럼 취급한다”는 것이다. 즉, 디스크를 임의 접근 가능한 저장 공간으로 보는 것이 아니라, 계속 이어붙이는 로그 구조로 바꾸는 것이다.

이렇게 하면 쓰기 작업이 대부분 순차 I/O로 바뀌기 때문에 디스크의 최대 순차 대역폭을 거의 그대로 활용할 수 있다.

기존 방식에서 문제가 되었던 짧은 탐색과 회전 지역이 크게 줄어드는 이유가 여기에 있다.

다만 여기서 중요한 포인트는 단순히 빠르게 쓰는 것만 아니라, “모든 변경을 순서대로 기록한다”는 구조 자체가 핵심이라는 점이다.

데이터 블록 하나를 쓰는 작업도 사실은 데이터 블록만의 문제가 아니라, 그 블록을 가리키는 아이노드 갱신, 디렉터리 갱신 같은 여러 메타데이터 변경이 함께 발생하는데, LFS 는 이 모든 것을 하나의 연속된 기록으로 만들어버린다.

그래서 결과적으로 디스크에는 항상 최신 변경들이 시간 순서대로 쌓이게 된다.

이 방식의 의미는 단순히 성능 개선에만 있는게 아니라, 파일 시스템 구조 자체를 “덮어쓰기 모델”에서 각존 데이터를 덮어쓰지 않고 항상 새로운 위치에 새로운 버전을 추가하는 형태가 된다. 이게 LFS 의 기존 철학이다.

팁: 세부적인 것들이 의미가 있다.

모든 흥미로운 시스템들은 몇 개의 일반적인 개념과 여러 개의 세부 정보들로 이루어져있다. 때로는 이러한 시스템들을 배울 때면 “대충 감이 잡히는데, 그 이상 세부 정보일 뿐이야”라고 생각할 수 있다.

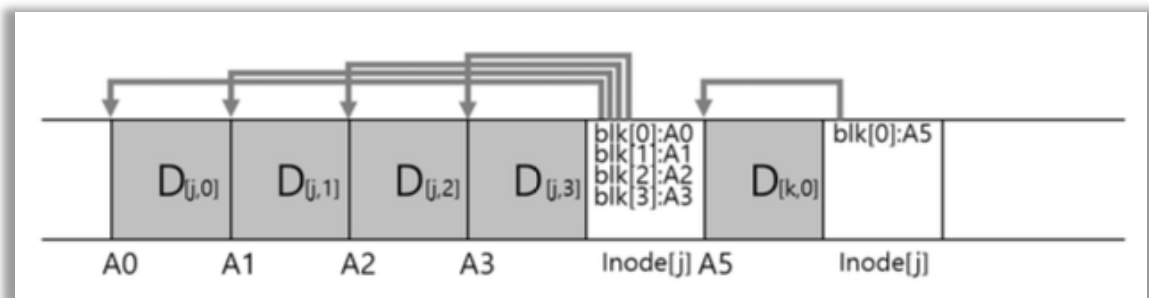
그러면 당신은 실제 어떻게 동작하는지에 대해서 반만큼만 배운 것이다.

대부분 세부적인 정보가 핵심이다.

LSF 에서도 보게 되겠지만 기본 개념은 이해하기 너무 간단하다.

하지만, 실제 동작하는 시스템을 만들려면 모든 종류의 까다로운 경우들을 고려해야 한다.

[2] 순차적이면서 효율적으로 쓰기



이부분의 핵심은 “순차 쓰기만으로는 충분하지 않고, 그걸 ‘큰 단위로 묶어서’ 써야 진짜 빠르다”는 점이다.

LFS 는 모든 쓰기를 순차적으로 한다고 했지만, 실제 디스크에서는 조금씩 쓰는 것만으로는 성능이 잘 나오지 않는다.

예를 들어, 시간 T 에 디스크 주소 A 에 블록 하나를 쓰고, 조금 뒤 T + Q 에 A + 1 위치에 또 쓰는 상황을 생각하면 겉보기에는 순차 쓰기처럼 보이지만 실제 디스크 입장에서는 문제가 생긴다. 디스크는 계속 회전하고 있기 때문에 두 번째 쓰기 시점에는 이미 해당 위치가 헤드를 지나가 버릴 수 있다.

그러면 다시 회전을 기다려야 하고, 결국 순차 쓰기인데도 불구하고 회전 지연 때문에 비효율이 생긴다. 즉 “조금씩 순차로 쓰는 것”은 디스크를 최대 속도로 활용하지 못한다.

그래서 LFS 가 도입하는 핵심 기술이 쓰기 버퍼링이다.

LFS 는 디스크에 바로바로 쓰지 않고, 먼저 메모리에서 변경 내용을 모아둔다.

데이터 블록 변경, 메타데이터 변경 같은 것들을 계속 쌓아두다가 어느 정도 충분히 모이면 그때 한 번에 디스크로 내려보낸다.

이렇게 하면 여러 개의 작은 쓰기들이 하나의 큰 순차 쓰기로 합쳐지기 때문에 디스크가 가장 잘 동작하는 형태, 즉 긴 연속 쓰기 형태가 된다.

이때 LFS 가 디스크에 한 번에 기록하는 단위를 세그먼트라고 부른다.

중요한 점은 세그먼트는 “작은 쓰기 하나”가 아니라 “여러 개의 파일 시스템 변경을 묶은 큰 덩어리”라는 것이다.

메모리에 세그먼트 버퍼를 두고, 여기에 변경 사항을 계속 모은 뒤 버퍼가 차면 디스크에 한 번의 큰 쓰기가 기록한다.

이렇게 하면 디스크는 짧은 탐색을 반복하지 않고 거의 순차 전송만 수행하게 된다.

예를 들어, 어떤 순간에 파일 j 에 블록 4 개가 추가되고, 동시에 파일 k 에 블록 1 개가 추가되는 변경이 발생했다고 하자.

LFS 는 이 두 개의 변경을 따로따로 디스크에 보내는게 아니라 하나의 세그먼트로 묶는다.

즉 총 7 개의 블록 (데이터 + 메타데이터 포함 가능)을 하나의 큰 순차 쓰기로 디스크에 기록한다. 그러면 디스크 입장에서는 “7 블록짜리 연속 쓰기”가 되기 때문에 효율이 훨씬 좋아진다.

결국 이 절의 핵심은 LFS 의 성능이 단순히 “순차 쓰기”에서 나오는데 아니라, “여러 작은 변경을 모아서 큰 순차 쓰기로 만드는 구조”에서 나온다는 점이다.

순차 쓰기라는 아이디어만으로는 부족하고, 그것을 극대화하기 위해 세그먼트 단위 버퍼링이 필수라는 의미다.

[3] 적절한 버퍼의 크기는?

이 부분은 LFS 에서 “세그먼트(버퍼)를 얼마나 크게 잡아야 디스크 성능을 최대한 뽑아낼 수 있는가”를 수식으로 정리하는 내용이다.

핵심은 디스크 성능이 단순히 전송 속도만으로 결정되는게 아니라, 항상 앞에 붙는 “탐색 + 회전 지연(= 위치 잡기 비용)” 때문에 실제 효율이 떨어진다는 점이다.

디스크에 어떤 크기 D 의 데이터를 쓰는 상황을 생각하면 전체 시간은 두 부분으로 나뉜다.

먼저 디스크 헤드를 원하는 위치로 옮기고 회전 위치를 맞추는 데 드는 고정 비용

T_{position} 이 있고, 그 다음에 실제로 데이터를 전송하는 시간이 D / R_{peak} 로 들어간다.

그래서 쓰기 시간은

$$T_{\text{write}} = T_{\text{position}} + D / R_{\text{peak}}$$

이 된다.

여기서 중요한 점은 Tposition 은 데이터 크기와 상관없이 항상 한 번 발생하는 “고정 비용”이라는 것이다. 즉, 작은 쓰기를 여러 번 하면 이 고정 비용이 계속 반복되면서 성능이 크게 떨어진다.

그래서 LFS 의 전략은 이 고정 비용을 최대한 “나눠서 부담”시키는 것이다.

방법은 간단하다. 한 번 디스크에 접근했을 때 가능한 한 많은 데이터를 한꺼번에 써버리는 것이다. 그러면 Tposition 이라는 비용이 큰 데이터 D 전체에 걸쳐 분산되기 때문에 전체 효율이 올라간다.

이걸 수식으로 표현하면 실제 유효 전송 속도는

$$\text{Reffective} = D / (T\text{position} + D / R\text{peak})$$

이 된다.

여기서 목표는 Reffective 를 최대 속도 Rpeak 에 최대한 가깝게 만드는 것이다.

하지만 완전히 같아질 수는 없고, 보통 “최대 속도의 일정 비율 F(예: 0.9, 0.95, 0.99)”를 목표로 잡는다.

즉,

$$\text{Reffective} = F \cdot R\text{peak}$$

이 되되록 D 를 결정하는 문제로 바뀐다.

이 식을 정리하면 결국 결론은 하나로 떨어진다.

$$D = (F / (1 - F)) \cdot R\text{peak} \cdot T\text{position}$$

이 식이 의미하는 바는 단순하다. 목표 성능 F 가 1 에 가까워질수록(즉, 99%, 99.9%에 가까울수록) 필요한 버퍼 크기 D 는 급격하게 커진다는 것이다.

왜냐하면 탐색 비용 Tposition 을 “무시할 수 있을 정도로 희석”시키려면 한 번에 써야 하는 데이터 양이 기하급수적으로 늘어나야 하기 때문이다.

예시에서처럼 Tposition 이 10ms 이고 Rpeak 가 100MB/s 이면, 90%성능을 내려면, 약 9MB 정도만 모아도 된다.

하지만 95%로 올리면 필요한 버퍼는 그보다 훨씬 커지고, 99%에 가까워지면 수십 MB 단위로 커진다. 즉 성능을 조금 더 끌어올릴수록 필요한 세그먼트 크기는 비선형적으로 커진다.

이게 LFS 설계에서 중요한 이유는 현실적인 시스템에서는 “무한히 큰 버퍼”를 쓸 수 없기 때문이다. 그래서 LFS 는 이 수식이 보여주는 이상적인 값과 실제 메모리 제약 사이에서 적당한 세그먼트 크기를 선택해야 한다는 결론으로 이어진다.

[4] 문제 1: 아이노드 찾기

이 절의 핵심은 “LFS에서는 아이노드가 고정된 위치에 있지 않아서, 기존처럼 단순한 계산으로 찾을 수 없다”는 문제다.

기존 Unix 파일 시스템에서는 아이노드 구조가 매우 단순하다.

아이노드 테이블이라는 배열이 디스크에 고정된 위치에 있고, 아이노드 번호만 알면 그 위치를 바로 계산할 수 있다. 즉, 아이노드 번호에 아이노드 크기를 곱해서 시작 주소에 더하면 끝이다. 그래서 아이노드 접근은 사실상 배열 인덱싱처럼 매우 빠르고 예측 가능하다.

FFS로 가면 구조가 조금 바뀌긴 하지만 본질은 크게 다르지 않다.

아이노드가 실린더 그룹 단위로 나뉘어 저장되기 때문에 “어느 그룹에 있는지”만 추가로 계산하면 되고, 그 이후에는 그룹 내부에서 배열처럼 접근할 수 있다.

즉, 복잡해졌지만 여전히 “고정된 위치 기반 구조”라는 점은 유지된다.

하지만 LFS에서는 이 전제가 깨진다.

LFS는 아이노드를 특정 위치에 고정해서 저장하지 않는다.

모든 변경을 세그먼트 형태로 계속 뒤에 추가하기 때문에, 아이노드는 디스크의 여기저기 새로운 위치에 계속 생성되고 이동한다.

더 중요한 점은 기존 데이터를 덮어쓰지 않기 때문에 아이노드 하나가 갱신될 때마다 새로운 버전이 새 위치에 기록된다는 것이다.

결과적으로 아이노드의 위치는 고정되어 있지 않고 계속 변한다.

같은 아이노드 번호를 가지고 있어도 “지금 최신 아이노드가 디스크 어디에 있는지”를 바로 계산할 방법이 없다. 예전처럼 배열 인덱스로 바로 접근하는 방식이 완전히 불가능해진다.

그래서 이 문제가 LFS에서 매우 중요한 설계 이슈가 된다.

단순히 “아이노드가 있다”가 아니라, “최신 아이노드를 어떻게 빠르게 찾을 것인가”가 핵심 문제가 된다. LFS는 이 문제를 해결하기 위해 이후에 inode map 같은 별도의 구조를 도입하게 된다.

[5] 간접 계층을 이용한 해법: 아이노드 맵

이 부분은 LFS 에서 가장 중요한 아이디어 중 하나인 “아이노드를 어떻게 추적할 것인가”를 해결하는 방식이다.

LFS 에서는 아이노드가 고정된 위치에 있지 않고 계속 새 위치로 이동하면서 기록되기 때문에, 기존처럼 “아이노드 번호로 바로 위치 계산”하는 방식이 완전히 깨진다. 그래서 필요한 것이 바로 아이노드 맵(inode map, imap)이다.

아이노드 맵은 말 그대로 “아이노드 번호 → 최신 아이노드 위치”를 알려주는 변환 테이블이다.

구조 자체는 매우 단순해서, 아이노드 번호를 인덱스로 쓰고 각 항목에는 해당 아이노드가 디스크의 어디에 있는지를 가리키는 포인터를 저장한다.

즉 $imap[i] = \text{디스크 주소 형태로, 아이노드 번호만 알면 최신 위치를 바로 찾을 수 있게 만든 간접 계층이다.}$

문제는 이 imap 자체도 계속 변한다는 점이다. LFS 에서는 아이노드가 갱신될 때마다 새로운 위치에 다시 쓰이고, 그때마다 imap 도 “이 아이노드는 이제 여기 있다”라고 갱신되어야 한다.

따라서 imap 은 단순한 보조 구조가 아니라 파일 시스템의 핵심 메타데이터 중 하나가 된다.

그래서 또 하나의 문제가 생긴다. 이 중요한 imap 을 디스크 어디에 저장할 것인가이다. 만약 imap 을 디스크의 고정된 한 위치에 둔다면, 아이노드가 바뀔 때마다 imap 도 계속 갱신해야 하고, 그때마다 디스크의 같은 위치를 반복적으로 수정하게 된다. 그러면 LFS 가 의도한 “순차 쓰기 중심 구조”가 깨지고 랜덤 I/O 가 다시 증가한다.

LFS 는 이 문제를 피하기 위해 imap 을 따로 고정된 장소에 두지 않는다.

대신 파일 데이터와 아이노드가 기록될 때 그 옆에 같이 기록하는 방식으로 해결한다.

즉, 어떤 파일 k 에 대해 데이터 블록과 아이노드가 새로 생성되면, 그 정보와 함께 “imap 의 일부 갱신 내용”도 같은 세그먼트에 포함시켜야 한 번에 순차적으로 디스크에 쓴다.

예를 들어, 파일 k 에 대해 새로운 데이터 블록이 A0 에 기록되고, 아이노드 I[k]가 A1 에 저장된다면, imap 에는 “k 번 아이노드는 A1 에 있다”라는 정보가 들어간다.

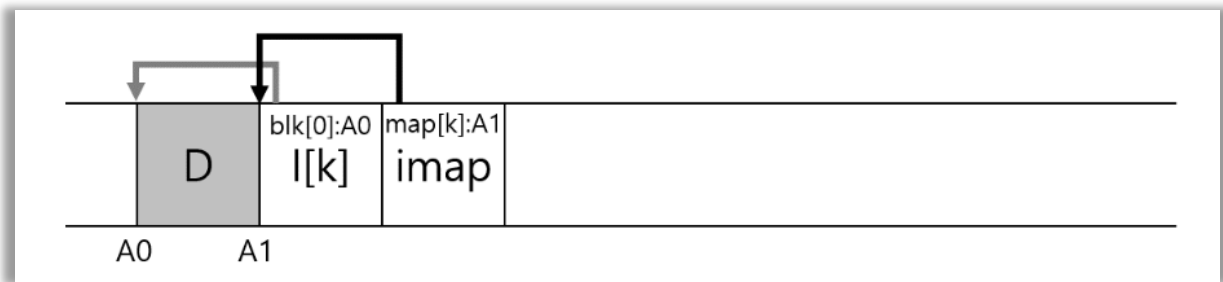
이 imap 갱신도 같은 로그세그먼트에 같이 기록된다.

이렇게 하면 아이노드 위치와 그를 추적하는 구조까지 모두 하나의 순차 쓰기 흐름 안에 포함될 수 있다.

결국 이 구조의 핵심은 단순하다.

아이노드 자체도 이동하고, 그 위치를 추적하는 imap 도 함께 로그에 기록되면서 계속 갱신되는 방식이다.

그래서 LFS 는 “아이노드 → imap → 최신 위치”라는 간접 계층을 통해 고정 위치 없이도 최신 메타데이터를 찾아갈 수 있게 만든다.



[6] 최종 완성: 체크포인트 영역

여기까지 오면 LFS의 “찾기 구조”가 완성된다.

핵심은 아이노드 맵(imap)을 도입했지만, 그 imap조차 디스크 전체에 흩어져 있기 때문에, “imap 자체를 어떻게 찾을 것인가”라는 새로운 문제가 남는다는 점이다.

앞에서 본 것처럼 LFS에서는 아이노드가 고정된 위치에 있지 않고 계속 새로운 위치로 이동하면서 기록된다. 그래서 아이노드를 찾기 위해서는 imap이 필요하고, imap이 있어야 아이노드 위치를 알 수 있다.

그런데 imap 역시 하나의 연속된 구조가 아니라 여러 블록으로 나뉘어 디스크 여러 곳에 흩어져 저장된다.

그러면 또 문제가 생긴다. “imap 블록들이 어디에 있는지 어떻게 아느냐”이다.

이 문제를 해결하기 위해 LFS는 단 하나의 고정된 기준점을 남긴다.

바로 체크포인트 영역(checkpoint region, CR)이다.

CR은 디스크 상에서 항상 약속된 특정 위치에 존재하며, 이 위치만은 절대 고정이다. 이곳에는 “현재 유효한 imap 블록들이 디스크 어디에 있는지”를 가리키는 포인터들이 저장된다.

즉 구조를 따라가면 다음과 같은 계층이 된다.

CR이 먼저 있고, CR은 imap 블록들의 위치를 가리키고, imap은 각 아이노드의 위치를 가리키고, 아이노드는 실제 데이터 블록을 가리킨다.

결국 CR은 전체 파일 시스템의 시작할 수 있는 유일한 진입점 역할을 한다.

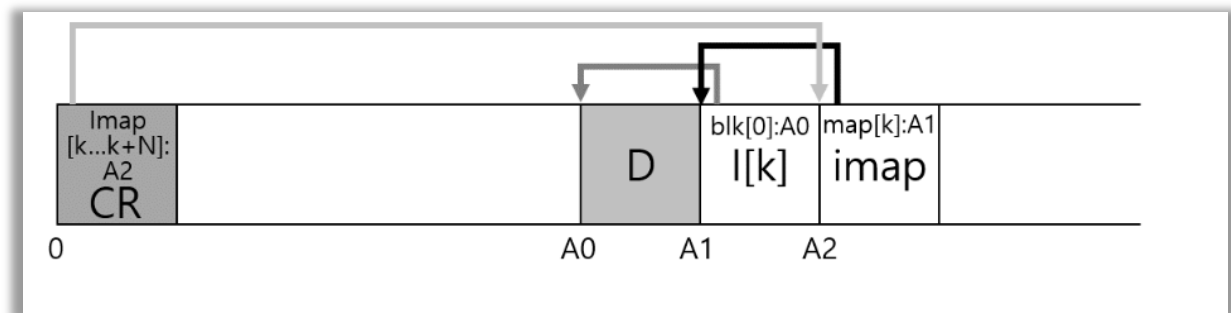
중요한 점은 CR 자체는 자주 바뀌지 않는다는 것이다. 파일 시스템이 변경될 때마다 매번 CR을 갱신하는 것이 아니라, 일정 시간 간격(예: 30초 정도)으로만 최신 imap 상태를 반영해서 업데이트한다. 그래서 LFS의 대부분의 쓰기는 여전히 순차 로그 구조로 처리되지만, “시작점”만은 고정된 위치에 유지된다.

이 구조 덕분에 LFS는 다음과 같이 동작한다. 시스템이 시작되면 먼저 디스크의 고정 위치에 있는 CR을 읽는다.

그러면 최신 imap 블록들이 어디에 있는지 알 수 있고, 그 imap 을 따라가면 최신 아이노드 위치를 찾을 수 있다. 아이노드를 찾으면 그 아이노드가 가리키는 데이터 블록으로 이동할 수 있다.

즉, 전체 파일 시스템 탐색이 “CR → imap → inode → data”라는 경로로 완성된다.

결국 이 절의 핵심은 LFS 가 아무리 로그 구조로 바뀌어도 완전히 무질서하게 흩어지는 것이 아니라, CR 이라는 최소한의 고정 앵커를 뒤편 전체 구조를 다시 추적할 수 있게 만든다는 점이다.



[7] 디스크에서 읽기: 요약

LFS 의 핵심은 모든 갱신을 순차적으로 세그먼트에 기록한다는 점이다. 그런데 이 구조는 시간이 지나면서 새로운 문제가 생긴다. 디스크에는 항상 “현재 사용 중인 데이터”만 존재하는 것이 아니라, 과거 버전의 데이터와 더 이상 참조되지 않는 블록들도 함께 남게 된다.

즉, 파일을 수정하거나 삭제할수록 디스크 전체에 “죽은 공간”이 늘어난다.

예를 들어, 파일의 한 블록을 수정한다고 해보자.

LFS 는 기존 블록을 덮어쓰지 않고 새로운 위치에 수정된 블록을 기록한다.

동시에 아이노드와 아이노드 맵도 갱신되어 새로운 블록을 가리킨다. 결과적으로 예전 블록은 어떤 아이노드에서도 참조되지 않게 되지만, 디스크에는 그대로 남아 있게 된다.

이런 블록이 계속 쌓이면 디스크 공간은 빠르게 파편화된다.

이 문제를 해결하기 위해 LFS 는 가비지 컬렉션 또는 세그먼트 정리 과정을 수행한다. 이 과정은 디스크의 일부 세그먼트를 선택해서 그 안에 들어 있는 블록 들 중 “여전히 유효한 데이터”만 골라내고, 이를 새로운 세그먼트로 복사한 뒤, 기존 세그먼트를 비우는 방식으로 진행된다.

여기서 핵심은 “유효한 블록인지 아닌지”를 판단하는 것이다.

이를 위해 LFS 는 각 세그먼트에 포함된 블록들이 어떤 아이노드에 의해 참조되는지를 추적한다.

최신 imap 과 아이노드를 기준으로 더 이상 참조되지 않는 블록은 버리고, 살아있는 블록만 새로운 위치로 이동시킨다.

이 과정은 단순히 공간 회복만 하는 것이 아니라 디스크 배치도 함께 정리하는 효과를 가진다. 자주 함께 접근되는 블록들을 새로운 세그먼트에 다시 모아두기 때문에, 시간이 지나면서 디스크가 다시 “순차적인 구조”에 가까워지도록 만든다.

LFS는 처음부터 순차 쓰기를 극대화하지만, 동시에 시간이 흐르면서 발생하는 파편화를 정리해서 성능을 유지하려는 구조를 가진다.

결국 LFS는 쓰기 시에는 순차성을 극단적으로 추구하고, 이후에는 백그라운드에서 정리 작업을 수행하여 공간과 성능을 균형 있게 유지하는 방식으로 동작한다.

[8] 디렉터리 관리 방법은?

디렉터리 관리도 결국 LFS에서는 “파일처럼 취급되는 데이터 구조”라는 점에서 출발한다. 디렉터리는 본질적으로 이름과 아이노드 번호를 매핑하는 테이블이고, Unix 계열 파일 시스템에서의 디렉터리와 구조적으로 크게 다르지 않다.

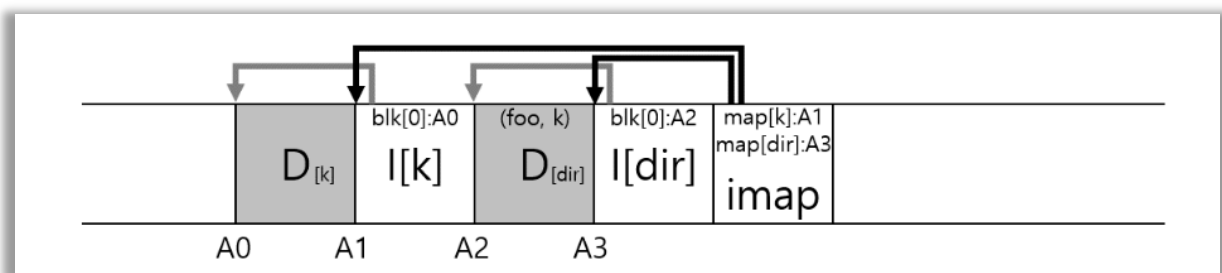
예를 들어, foo 라는 파일을 /home/remzi 같은 디렉터리 안에 생성한다고 하면, LFS는 단순히 데이터 블록 하나만 추가하는 것이 아니라 여러 종류의 정보를 함께 생성한다. 새 파일 foo를 위한 아이노드가 하나 생성되고, 그 아이노드는 데이터 블록을 가리키게 된다.

동시에 부모 디렉터리 dir의 데이터 블록도 수정되어 (foo, k)같은 형태로 “이름과 아이노드 번호의 쌍”이 추가된다. 그리고 이 디렉터리의 아이노드 역시 크기 변화나 타임스탬프 변경 때문에 갱신된다.

중요한 점은 LFS가 이 모든 변경을 덮어쓰기로 처리하지 않는다는 것이다.

기존 위치를 수정하지 않고, 새로운 세그먼트에 디렉터리 데이터, 아이노드, 데이터 블록, 그리고 관련된 아이노드 맵 변경까지 모두 순차적으로 기록한다.

따라서 디렉터리 변경 역시 결국 “로그에 추가되는 이벤트”처럼 처리된다.



이 구조에서 디렉터를 읽는 과정은 여러 단계의 간접 참조를 거친다.

먼저 imap을 통해 디렉터리 dir의 아이노드 위치를 얻고, 그 아이노드를 통해 디렉터리 데이터 블록 위치를 찾는다. 디렉터리 데이터 안에는 (foo, k) 형태의 항목들이 들어 있고, 여기서 foo의 아이노드 번호 k를 얻는다. 다시 imap을 조회해서 k에 해당하는 최신 아이노드 위치를 찾고, 마지막으로 그 아이노드가 가리키는 데이터 블록을 읽는다.

이 구조가 중요한 이유는 “재귀 갱신 문제”를 피하기 위해서다. 만약 디렉터리가 아이노드의 실제 디스크 주소를 직접 저장하는 구조라면, 아이노드 위치가 변경될 때마다 디렉터리 데이터도 수정해야 하고, 그 디렉터를 포함하는 상위 디렉터리도 계속 갱신해야 한다. 이런 식이면 루트까지 연쇄적으로 업데이트가 발생한다.

LFS 는 이 문제를 imap 이라는 간접 계층으로 끊어버린다.

디렉터리는 아이노드 “번호”만 알고 있고, 실제 위치는 imap 이 관리하므로 아이노드 위치가 바뀌어도 디렉터리 자체는 수정할 필요가 없다.

[9] 새로운 문제: 가비지 컬렉션

LFS 에서 가비지 컬렉션 문제가 생기는 이유는 구조 자체가 “덮어쓰기 없이 항상 새로운 위치에 기록하는 방식”이기 때문이다. 이 방식은 쓰기 성능을 높이기 위해 설계된 것이지만, 그 대가로 과거 데이터가 디스크에 그대로 남는 상황을 만든다.

파일의 데이터를 수정하거나 추가할 때마다 LFS 는 새로운 아이노드와 새로운 데이터 블록을 생성하고, 이를 새로운 세그먼트에 순차적으로 기록한다.

동시에 아이노드 맵도 갱신되어 최신 버전을 가리키게 된다.

문제는 이전에 사용되던 아이노드와 데이터 블록이 더 이상 참조되지 않더라도 디스크에 그대로 남아 있다는 점이다. 이런 블록들은 현재 파일 시스템 관점에서는 필요 없는 “쓰레기”가 된다.

예를 들어, 아이노드 k 가 데이터 블록 $D0$ 을 가리키고 있었고, 이후에 해당 파일이 수정되었다고 가정하자. 그러면 새로운 아이노드 k' 와 새로운 데이터 블록 $D1$ 이 생성되고, $imap$ 도 k' 를 가리키도록 갱신된다.

이 순간 $D0$ 과 이전 아이노드는 더 이상 어떤 경로에서도 접근되지 않지만 디스크에는 그대로 남아 있다. 파일을 계속 수정하거나 확장할수록 이런 과거 버전의 블록이 쌓이게 된다.

이 문제를 단순하게 보면 과거 데이터를 보관하는 “버전 파일 시스템”처럼 동작할 수도 있다. 실제로 이런 방식은 파일의 이전 상태 복원이 가능하다는 장점이 있다.

하지만 LFS 의 목표는 버전 관리가 아니라 최신 상태의 파일 시스템을 효율적으로 유지하는 것이다. 따라서 LFS 는 주기적으로 백그라운드 작업을 통해 오래된 블록들을 정리한다.

이 정리 작업이 바로 가비지 컬렉션이다. 가비지 컬렉션은 디스크 전체를 개별 블록 단위로 하나씩 처리하는 방식이 아니라 세그먼트 단위로 수행된다.

만약 블록 단위로 조금씩 해제한다면 디스크에는 여기저기 작은 빈 공간이 생기고, 이는 다시 순차 쓰기를 방해하게 된다. LFS 는 이런 상황을 피하기 위해 큰 단위로 공간을 정리한다.

가비지 컬렉터는 특정 세그먼트를 선택한 뒤 그 안에 있는 블록들을 검사한다. 각 블록이 현재의 아이노드나 imap 을 통해 여전히 참조되고 있는지 확인하고, 살아 있는 블록만 새로운 세그먼트로 복사한다. 복사가 끝나면 원래 세그먼트는 더 이상 유효한 데이터가 없는 빈 공간으로 간주된다. 이 과정을 통해 여러 개의 오래된 세그먼트를 하나의 새 세그먼트로 압축하게 된다.

이 방식은 단순히 공간을 회수하는 것 이상의 의미를 가진다. 유효한 블록들만 다시 모아서 새로운 세그먼트에 연속적으로 배치하기 때문에 디스크의 순차성을 다시 회복시키는 효과가 있다. 즉, LFS 는 쓰기 성능을 위해 순차 기록을 선택했고, 시간이 지나면서 발생하는 파편화는 가비지 컬렉션으로 다시 정리하는 구조를 가진다.

남아 있는 핵심 문제는 두 가지다. 하나는 세그먼트 내부에서 어떤 블록이 살아 있는지 정확히 어떻게 판단할 것인가이고, 다른 하나는 이 가비지 컬렉션을 언제, 얼마나 자주, 어떤 세그먼트에 대해 수행할 것인가라는 정책 문제다.

[10] 블록의 최신 여부 판단

LFS 에서 가비지 컬렉션을 수행하려면 핵심적으로 “이 블록이 아직 유효한가”를 판단할 수 있어야 한다. 왜냐하면 세그먼트 안에는 현재 사용 중인 데이터와 과거 버전의 데이터가 섞여있기 때문이다.

이를 위해 LFS 는 각 데이터 블록마다 “어느 파일에 속하는지”와 “파일 내부에서 몇 번째 블록인지”를 함께 기록한다.

구체적으로는 아이노드 번호와 파일 내 오프셋 정보를 저장한다. 이 정보는 개별 블록에 따로 붙어 있는 것이 아니라, 세그먼트의 앞부분에 있는 세그먼트 요약 블록에 기록된다.

이제 세그먼트 안에 있는 특정 블록 D 가 유효한지 판단하는 과정을 보면 다음과 같다. 먼저 해당 블록이 속한 세그먼트의 요약 블록에서 D 의 아이노드 번호 N 과 오프셋 T 를 읽는다.

그 다음 imap 을 이용해서 아이노드 N 의 현재 위치를 찾고, 해당 아이노드를 디스크에서 읽는다. 그리고 아이노드 안에 있는 포인터 구조를 따라가며 오프셋 T 에 해당하는 블록이 현재 어디를 가리키는지를 확인한다.

만약 그 위치가 현재 검사 중인 블록 D 의 주소 A 와 같다면, D 는 최신 블록이다. 반대로 다르다면 이미 새로운 버전이 존재하므로 D 는 더이상 사용되지 않는 오래된 블록이다.

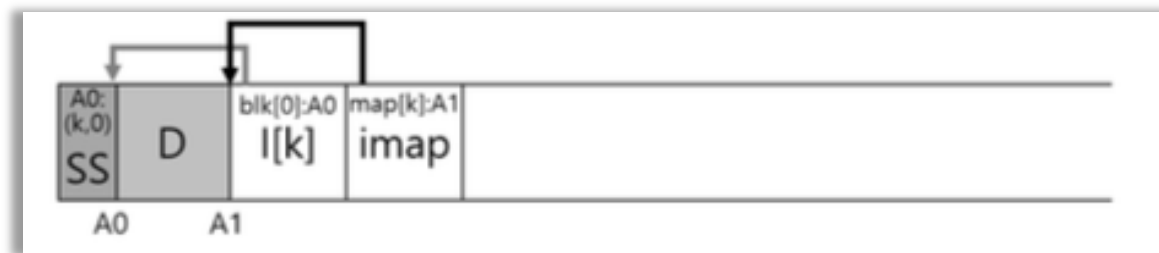
이 방식은 개념적으로는 정확하지만, 실제로는 비용이 꽤 크다. 각 블록마다 imap 조회, 아이노드 읽기, 포인터 추적이 필요할 수 있기 때문이다. 그래서 LFS 는 이 과정을 줄이기 위한 추가적인 최적화를 사용한다.

대표적인 방법이 버전 번호를 활용하는 것이다. 파일이 삭제되거나 truncate 될 때마다 해당 파일의 버전 번호를 증가시키고, 이 값을 imap 에 저장한다.

그리고 세그먼트에 기록된 블록에도 해당 버전 번호를 함께 저장한다.

가비지 컬렉션 시에는 현재 imap 에 있는 버전 번호와 세그먼트에 기록된 버전 번호만 비교하면 된다.

값이 다르면 이미 새로운 버전이 존재하는 것이므로 해당 블록은 즉시 무효로 판단할 수 있다. 이렇게 하면 앞에서 설명한 복잡한 아이노드 추적 과정을 생략할 수 있어서 유효성 검사 비용을 크게 줄일 수 있다.



[11] 정책: 어떤 블록을 언제 정리하는가

가비지 컬렉션에서 핵심은 “언제 실행할 것인가”와 “어떤 세그먼트를 선택할 것인가”이다. LFS 는 이 두 가지 모두 정책 문제로 다룬다.

먼저 언제 가비지 컬렉션을 수행할 것인지는 비교적 단순하다.

시스템이 주기적으로 실행할 수도 있고, 디스크가 유헴 상태일 때 수행할 수도 있으며, 또는 디스크 공간이 부족해질 때 강제로 수행할 수도 있다. 실제 시스템에서는 이 조건들이 혼합되어 사용된다. 즉, 항상 일정한 주기만으로 동작하는 것이 아니라 시스템 상태에 따라 유동적으로 실행된다.

더 어려운 문제는 어떤 세그먼트를 선택할 것인가이다.

LFS 의 초기 연구에서는 세그먼트를 “hot”과 “cold”로 구분하는 방법을 제안했다.

hot 세그먼트는 최근에 자주 수정되는 데이터가 많이 포함된 세그먼트이다. 이런 세그먼트는 아직 유효한 블록이 많이 남아 있을 가능성이 높기 때문에, 바로 정리하지 않고 더 기다리는 것이 유리하다.

시간이 지나면서 해당 블록들이 다시 수정되거나 덮어써지면, 결과적으로 남아 있는 유헴 블록의 비율이 줄어든다. 그렇게 되면 가비지 컬렉션 시에 복사해야 할 데이터가 거의 없거나 매우 적어지므로 효율이 좋아진다.

반대로 cold 세그먼트는 거의 수정되지 않는 데이터를 포함하는 세그먼트이다. 예를 들어, 음악 파일이나 이미지 파일처럼 한 번 기록된 이후 거의 변경되지 않는 데이터가 여기에 해당한다.

이런 세그먼트는 시간이 지나도 대부분의 블록이 여전히 유효 상태로 남아 있을 가능성이 높다. 따라서 가비지 컬렉션을 해도 없는 이득이 크지 않다.

대신 오래 보존되면서 거의 이동이 없기 때문에 상대적으로 안정적인 저장 영역이 된다.

초기 LFS 연구에서는 hot 세그먼트는 가비지 컬렉션을 늦게 수행하고, cold 세그먼트는 더 자주 정리하는 것이 전체 시스템 효율에 유리하다고 보았다.

하지만 실제로는 데이터의 접근 패턴이 단순히 hot/cold 로 나뉘지 않고 변화하기 때문에 이 방식만으로는 최적의 성능을 얻기 어렵다.

이후 연구에서는 더 세밀한 통계 기반 방법이나 비용 모델을 이용해 세그먼트를 선택하는 방식들이 제안되었다.

[12] 크래시로부터의 복구와 로그

LFS 에서도 크래시 문제는 매우 중요하다.

구조 자체가 모든 갱신을 로그 형태로 순차 기록하는 방식이기 때문에, 쓰기 도중에 시스템이 중단되면 파일 시스템 상태가 중단 단계에 머무를 수 있다.

따라서, LFS 는 이를 복구하기 위한 메커니즘을 별도로 설계한다.

먼저 LFS 의 기본 구조를 보면, 메모리에 세그먼트 버퍼를 두고 갱신 내용을 모은 뒤 일정 시점(세그먼트가 가득 찼거나 시간이 지나면)에 디스크에 한 번에 기록한다.

이렇게 기록된 세그먼트들은 단순한 블록 집합이 아니라 하나의 “로그”처럼 연결된다.

각 세그먼트는 다음 세그먼트를 가리키는 포인터를 가지고 있고, 체크포인트 영역은 이 로그의 시작과 끝에 대한 정보를 가지고 있다.

결과적으로 전체는 연결 리스트 형태의 세그먼트 로그 구조를 갖게 된다.

크래시는 크게 두 가지 상황에서 문제가 된다.

하나는 체크포인트 영역을 갱신하는 도중에 발생하는 경우이고, 다른 하나는 세그먼트를 쓰는 도중 또는 체크포인트가 오래된 상태에서 발생하는 경우이다.

체크포인트 영역 갱신 중 크래시 문제로부터 보면, 이 영역은 파일 시스템 전체 상태를 요약하는 매우 중요한 구조이기 때문에 반드시 일관성이 유지되어야 한다.

이를 위해 LFS 는 체크포인트 영역을 두 개 유지한다.

디스크의 서로 다른 위치(보통 양 끝)에 두고 번갈아 가며 갱신한다.

갱신 과정도 단순히 덮어쓰는 것이 아니라, 먼저 헤더를 쓰고 그 다음 실제 내용을 기록한 뒤 마지막 블록을 갱신하는 순서를 따른다.

헤더와 마지막 블록에는 시간 정보가 들어 있다. 정상적으로 완료되었다면 두 값이 일관된 관계를 가진다. 만약 갱신 도중 크래시가 발생하면 이 값들이 어긋나게 되고, 복구 시 어떤

체크포인트가 유효한지 판단할 수 있다. 두 개 중에서 가장 최근이면서 일관된 체크포인트를 선택하여 사용한다.

다음으로 더 일반적인 문제는 체크포인트가 최신 상태를 반영하지 못하는 경우이다. 체크포인트는 일정 주기(예를 들어 약 30 초)로만 갱신되기 때문에, 크래시가 발생하면 마지막 몇 초 동안 변경 내용은 체크포인트에 반영되지 않을 수 있다. 단순히 체크포인트만 사용하면 이 기간 동안의 모든 변경이 사라진다.

이 문제를 줄이기 위해 LFS 는 롤 포워드(roll forward) 기법을 사용한다.

복구 과정은 다음과 같이 진행된다.

먼저 체크포인트 영역을 읽어서 파일 시스템의 기준 상태를 복원한다.

그 다음 체크포인트가 가리키는 “마지막 세그먼트” 위치를 기준으로 로그를 따라간다.

각 세그먼트는 다음 세그먼트를 가리키는 포인터를 가지고 있기 때문에, 이를 따라가며 체크포인트 이후에 기록된 세그먼트들을 순차적으로 탐색할 수 있다.

이 과정에서 발견되는 최신 데이터와 메타데이터를 반영하여 파일 시스템 상태를 앞으로 진행시킨다.

결국 LFS 의 복구는 “마지막으로 안정적으로 저장된 스냅샷(체크포인트)”에서 시작해서, 그 이후의 로그를 재사용하는 방식이다. 이 구조는 데이터베이스의 로그 기반 복구와 매우 유사하며, 전체 디스크를 검사하는 방식보다 훨씬 효율적으로 최신 상태에 가까운 파일 시스템을 복원할 수 있게 해준다.

[1-6] 데이터 무결성과 보호

핵심 질문: 데이터 무결성을 어떻게 보장하는가

- 저장 장치에 쓴 데이터가 보호되고 있다는 것을 시스템이 어떻게 보장할까?
- 어떤 기술들이 필요할까?
- 어떻게 하면 그 기술들을 공간과 시간 오버헤드가 적으면서 효율적으로 만들 수 있을까?

우리가 지금까지 살펴본 파일 시스템의 기본적인 기술들 이외에도 몇 가지 더 다루어야 할 주제들이 있다.

이번 장에서는 신뢰성을 다시 한 번 다루도록 하겠다(RAID 를 설명한 장에서 저장 시스템의 신뢰성에 대해서 학습했었다)

이 장에서는 구체적으로 저장 장치가 기본적으로 신뢰할 수 없다는 전제 하에 파일 시스템이 데이터의 안정성을 보장하는 방법에 대해 살펴볼 것이다.

이 주제가 다루는 부분을 데이터 무결성 또는 데이터 보호라고 부른다. 이번장에서 살펴볼 기술은 저장 시스템에서 데이터를 읽었을 때 그 데이터가 처음에 썼던 것과 동일하다는 것을 보장하는 기술이다.

[1] 디스크 오류 모델

이 장에서 말하고 싶은 핵심은 하나다. “디스크는 더 이상 단순히 ‘죽거나 살아있거나’하지 않는다”

과거에는 디스크 오류 모델이 굉장히 단순했다. 디스크가 정상적으로 동작하거나, 아니면 완전히 고장나서 아예 응답을 안하거나 둘 중 하나였다.

이걸 fail-stop 모델이라고 한다. RAID 가 비교적 단순하게 설계될 수 있었던 이유도 이 가정 때문이다.

디스크 하나가 죽으면 그냥 그 디스크 전체를 잃어버린 것으로 보고 복구하면 됐다.

그런데 현대 디스크에서는 이 가정이 깨졌다. 디스크 전체는 멀쩡하게 돌아가는데, 일부 블럭만 문제를 일으키는 문제가 생긴다.

즉, 디스크는 “겉으로는 정상처럼 보이지만 내부적으로는 부분적으로 망가져 있는 상태”가 될 수 있다. 이걸 부분 실패(fail-partial)모델이라고 부른다.

이 모델에서 중요한 오류는 두 가지다.

첫 번째는 LSE(Latent Sector Error, 잠재적 섹터 오류)다.

이건 쉽게 말하면 “특정 블럭이 물리적으로 망가져서 읽을 수 없는 상태”다.

예를 들어

- 디스크 헤드가 표면에 살짝 닿아서 긁힘이 생기거나
- 방사선이나 전기적 문제로 비트가 깨지거나

이런 일이 발생하면 해당 섹터의 데이터가 손상된다.

디스크에는 보통 ECC(에러 정정 코드)가 있어서 약간의 오류는 자동으로 고칠 수 있다.
하지만 손상이 심하면 복구가 불가능하다.
그 상태에서 해당 블록을 읽으려고 하면, 디스크는 “이건 못 읽겠다”라고 에러를 반환한다.

즉, LSE는 특징이 명확하다.

- 문제가 있는 걸 디스크가 “알고 있음”
- 읽기 시도하면 에러를 리턴함
- 데이터는 못 얻는다

이건 그래도 다루기 쉬운 편이다. 왜냐하면 “문제가 발생했다”는 걸 시스템이 인지할 수 있기 때문이다.

두 번째는 훨씬 더 위험한 블록 손상(block corruption)이다.
이건 상황이 다르다. 디스크는 문제가 있는지조차 모른다.

예를 들어:

- 펌웨어 버그로 잘못된 위치에 데이터가 써짐
- 전송 과정에서 데이터가 깨졌는데 그대로 저장됨

이 경우 ECC는 “이 데이터는 정상이다”라고 판단할 수 있다.
왜냐하면 내부적으로는 일관성이 맞기 때문이다. 그런데 실제로는 완전히 틀린 데이터가 들어 있다.

그래서 이 경우

- 읽기는 정상적으로 성공한다.
- 에러도 안 난다
- 하지만 잘못된 데이터가 반환된다.

이걸 silent fault(조용한 오류)라고 부른다.

이게 왜 위험하냐면, 시스템 입장에서는 “정상 데이터”라고 믿고 계속 사용하기 때문이다.
즉, 오류를 감지할 기회조차 없다.

여기 까지 정리하면 차이가 명확해진다.

- LSE: 읽기 실패 -> “문제 발생”을 알 수 있음
- 블록 손상: 읽기 성공 -> 하지만 데이터는 틀림 -> “문제를 모름”

후자가 훨씬 위험하다.

이 두가지를 합쳐서 현대 디스크의 특징을 설명하는 모델이 바로 fail-partial 모델이다.

이 모델에서는 디스크가

- 대부분은 정상처럼 동작하지만
- 일부 블록은 읽기 실패를 일으키거나
- 일부 블록은 조용히 잘못된 데이터를 반환한다.

즉, 디스크를 더 이상 “신뢰 가능한 저장장치”로 가정할 수 없다.

그 다음 중요한 건 “이게 얼마나 자주 발생하느냐”다

연구 결과를 보면 생각보다 드물지 않다.

3 년 동안 150 만 개 이상의 디스크를 조사했을 때

- LSE 는 꽤 높은 비율의 디스크에서 발생
- 블록 손상은 더 드물지만 여전히 존재

그리고 몇 가지 중요한 특징이 있다.

LSE 의 경우

- 한 번 발생한 디스크는 추가 오류가 생길 가능성이 높다.
- 시간이 지나면서(특히 2 년차) 오류율이 증가한다.
- 디스크 용량이 클수록 더 많이 발생한다.
- 특정 위치에 몰려서 발생하는 경향이 있다.

블록 손상의 경우:

- 디스크 모델에 따라 발생 확률이 크게 다르다
- 워크로드 영향을 많이 받는다
- RAID 환경에서도 독립적이지 않을 수 있다.

이걸 왜 배우는지 핵심을 짚자

앞에서 RAID 를 배울 때는 이런 가정을 했다. “디스크 하나가 통째로 고장난다 -> 나머지로 복구하면 된다” 하지만 이제는 상황이 바뀐다.

- 디스크는 살아있지만 일부 블록만 망가질 수 있다.
- 더 나쁜 경우, 틀린 데이터를 조용히 반환할 수도 있다.

그래서 저장 시스템은 이제 단순히 “디스크 고장 대비”만 하면 안된다.

반드시

- 데이터가 맞는지 검증해야 하고
- 손상된 블록을 복구할 수 있어야 한다.

즉, 신뢰성을 위해서는 검출 + 복구 메커니즘이 필수다.

[2] 숨어있는 섹터 에러(Latent Sector Error)

핵심 질문: 숨어 있는 섹터 에러를 어떻게 처리할까

- 저장 시스템이 숨어있는 섹터 에러들을 어떻게 처리해야 할까?
- 이런 형태의 부분 오류를 처리하기 위해서는 어떤 추가적인 기술을 필요로 할까?

먼저 LSE 라는 건 디스크가 “이 블록 못 읽겠다”라고 에러를 명확하게 알려주는 경우다. 즉, 데이터가 망가졌지만 그 사실을 디스크가 인지하고 있다는 점이 핵심이다. 이게 중요한 이유는 오류를 “알려주기 때문에” 복구 전략을 적용할 수 있기 때문이다.

이 상황에서 저장 시스템이 해야 할 일은 간단하다.

문제가 있는 블록을 그냥 포기하는 게 아니라, 다른 곳에 저장된 중복 정보로부터 원래 데이터를 복원해서 사용자에게 정상 데이터를 돌려주는 것이다.

이걸 가능하게 해주는 게 바로 RAID 같은 중복 저장 구조다.

미러링 RAID(예: RAID-1)의 경우는 가장 직관적이다.

같은 데이터를 여러 디스크에 복사해두기 때문에, 하나의 디스크에서 LSE 가 발생하면 다른 디스크의 복사본을 읽어서 그대로 반환하면 된다.

구조가 단순한 대신, 디스크를 두 배로 써야 한다는 비용이 있다.

반면 RAID-4 나 RAID-5 같은 패리티 기반 RAID 는 방식이 조금 다르다.

여기서는 데이터를 단순히 복사하지 않고, 여러 블록을 조합해서 만든 패리티 정보를 함께 저장한다. 그래서 특정 블록 하나가 읽기 실패(LSE)를 일으키면, 같은 패리티 그룹에 속한 나머지 블록들과 패리티를 이용해서 수학적으로 해당 블록을 다시 계산해서 복원한다.

즉, LSE 처럼 “어디가 망가졌는지 아는 경우”에는 이미 오래전부터 알려진 중복 기반 복구 기법으로 충분히 대응 가능하다는 게 핵심이다.

그런데 문제가 하나 더 있다. 실제 시스템에서는 오류가 하나만 깔끔하게 발생하지 않는다. 특히 RAID-4/5 에서 심각한 상황은 다음과 같다.

어떤 디스크가 완전히 고장난 상태(전체 디스크 실패)에서, 나머지 디스크들로 데이터를 재구성하려고 할 때를 생각해보자.

이때 RAID 는 고장난 디스크의 데이터를 복구하기 위해 다른 디스크들에 있는 모든 관련 블록을 읽어서 재구성(rebuild)을 수행한다.

문제는 이 과정에서 다른 디스크 중 하나에서 LSE 가 발생하는 경우다. 상황을 정리하면

- 디스크 A: 완전히 고장(데이터 없음)
- 디스크 B, C, D: 살아 있음
- 그런데 재구성 중에 B 에서 LSE 발생(특정 블록 못 읽음)

이렇게 되면, 패리티를 계산할 때 필요한 입력값이 하나 부족해진다.
즉, 복구에 필요한 정보 자체가 깨진 상태가 되어버린다.
결과적으로 RAID-4/5 는 이상황에서 복구 실패가 발생할 수 있다.

이게 현실에서 꽤 위험한 이유는, 디스크 하나가 고장난 뒤 재구성하는 동안이 시스템에서 가장 취약한 시점인데, 그 순간에 LSE 가 하나라도 겹치면 전체 데이터가 날아갈 수 있기 때문이다.

이 문제를 해결하기 위해 등장한 방법 중 하나가 이중 패리티다.

대표적인 예가 NetApp 의 RAID-DP 인데 이 방식은 패리티를 하나가 아니라 두 개를 둔다.
그 결과

- 디스크 하나가 완전히 죽고
- 동시에 다른 디스크에서 LSE 가 발생하더라도

추가적인 패리티 정보를 이용해서 여전히 데이터를 복구할 수 있다.

즉, RAID-5 가 “한 개 장애까지 버틴다”면 RAID-DP 같은 구조는 “두 개 문제까지 동시에 대응”할 수 있는 셈이다.

물론 이건 공짜가 아니다. 대가로 치르는 비용은 명확하다.

- 패리티를 하나 더 저장해야 하므로 디스크 공간이 더 필요하다.
- 각 스트라이프마다 추가 연산이 필요하다

하지만 대규모 저장 시스템에서는 데이터 안정성이 훨씬 중요하기 때문에 이 정도 비용을 감수하고라도 신뢰성을 높이는 방향을 선택하는 경우가 많다.

정리하면 이 부분의 핵심 흐름은 이렇게 이어진다.

- LSE 는 “에러를 알려주는” 오류라서 복구 자체는 어렵지 않다.
- RAID 의 중복 구조(미러링, 패리티)를 이용하면 정상 데이터 복원이 가능하다.
- 하지만 디스크 전체 장애 + LSE 가 동시에 발생하면 기존 RAID-4/5 는 취약하다
- 이를 해결하기 위해 이중 패리티 같은 구조가 등장했다.

[3] 손상 검출: 체크섬

핵심질문: 어떻게 데이터가 손상이 되어도 무결성을 유지할까

- 오류의 침묵적인 특성을 고려한다면 손상이 일어났다는 것을 저장 시스템이 검출하기 위해서는 무엇을 해야할까?
- 어떤 기술들이 필요한가?
- 어떻게 하면 기술들을 효율적으로 구현할 수 있을까?

여기서 다루는 문제는 “블럭이 망가졌는지 시스템이 어떻게 알아내느냐”이다.

앞에서 본 LSE는 디스크가 스스로 “이 블럭 못읽는다”라고 에러를 주기 때문에 단순하다. 하지만 여기서는 훨씬 까다로운 상황이다.

디스크 입장에서는 정상적으로 데이터를 썼다고 생각하지만, 실제로는 데이터가 틀린 값으로 저장되는 경우가 있다. 이걸 블럭 손상(block corruption)이라고 한다.

더 큰 문제는, 이 경우 디스크가 아무 에러도 주지 않는다는 점이다. 즉, 사용자는 “정상 데이터”라고 믿고 잘못된 데이터를 읽게 된다.

그래서 필요한 것이 손상 검출 메커니즘, 즉 체크섬(checksum)이다.

체크섬의 핵심 아이디어는 단순하다.

어떤 데이터 블럭이 있을 때, 그 내용을 바탕으로 작은 요약값을 계산해서 저장해둔다.

예를 들어 보자.

데이터 블럭 D가 다음과 같다고 하자(간단히 8비트만 사용)

$D = [10, 20, 30, 40, 50, 60, 70, 80]$

이 데이터에 대해 체크섬을 하나 계산해서 같이 저장한다.

이제 데이터를 읽을 때 과정을 보자

1. 디스크에서 데이터 D와 체크섬 C(D)를 함께 읽는다.
2. 읽어온 데이터로 다시 체크섬을 계산한다.
3. 기존 체크섬과 비교한다.

예를 들어:

- 저장된 체크섬: 100
- 새로 계산한 체크섬: 100 → 정상
- 새로 계산한 체크섬: 87 → 손상됨

이렇게 해서 데이터가 바뀌었는지 감지할 수 있다.

1. XOR 체크섬(가장 단순)

예시를 직접 계산해보자.

4 바이트씩 나눠서 XOR 한다고 하면

$$D = [10, 20, 30, 40]$$

$$10 \oplus 20 = 30$$

$$30 \oplus 30 = 0$$

$$0 \oplus 40 = 40$$

이제 데이터가 아래와 같이 변했다고 하자

$$D' = [11, 21, 30, 40]$$

계산:

$$11 \oplus 21 = 30$$

$$30 \oplus 30 = 0$$

$$0 \oplus 40 = 40$$

즉, 데이터가 바뀌었는데도 체크섬은 동일하다. 이게 XOR 의 한계다.

2. 덧셈 체크섬

같은 데이터를 더해보자.

$$10 + 20 + 30 + 40 = 100$$

데이터가 바뀌면

$$11 + 21 + 30 + 40 = 102$$

체크섬이 바뀜 -> 오류 검출 가능

하지만 문제는

$$[10, 20, 30, 40]$$

$$\rightarrow [20, 10, 30, 40]$$

이렇게 위치만 바뀌면 체크섬이 동일하기 때문에 오류를 못잡는다.

3. Fletcher 체크섬(조금 더 강력)

같은 데이터:

$$D = [10, 20, 30]$$

계산:

$$- s1 = 10 + 20 + 30 = 60$$

$$- s2 = (10) + (10 + 20) + (10 + 20 + 30) = 10 + 30 + 60 = 100$$

$$\rightarrow \text{체크섬} = (s1, s2) = (60, 100)$$

이 방식은 순서 정보까지 반영하기 때문에 단순 합보다 훨씬 다양한 오류를 잡는다.

4. CRC(실제 시스템에서 많이 사용)

개념적으로 보면: 데이터를 하나의 큰 이진수로 보고 어떤 정해진 값으로 나눈 “나머지”를 체크섬으로 사용한다.

예를 들어:

데이터: 110101
나눌 값: 1011
→ 나머지: 011

이 방식은 매우 다양한 오류를 잡을 수 있고 네트워크, 파일 시스템에서 널리 사용된다.

중요한 사실: 체크섬은 완벽하지 않다.

서로 다른 데이터가 같은 체크섬을 가질 수 있다.

데이터 A → checksum = 123
데이터 B → checksum = 123

이걸 충돌(collision)이라고 한다.

왜냐하면:

- 큰 데이터(예: 4KB)
 - 작은 값(예: 4 바이트)
- 으로 압축하기 때문이다.

체크섬을 어디에 저장할까?

방법 1: 데이터 옆에 붙이기

[데이터][체크섬]
[데이터][체크섬]

장점: 빠름(쓰기 1 번이면 끝)

단점: 디스크가 520 바이트 같은 특수 구조를 지원해야 함

방법 2: 체크섬 따로 모아서 저장

[체크섬 블록]
[데이터 1]
[데이터 2]
[데이터 3]

예를 들어:

C1, C2, C3 → 하나의 블록에 저장
D1, D2, D3 → 각각 데이터 블록

이제 D1 을 수정한다고 해보자.

필요한 작업:

1. 체크섬 블록 읽기
2. C1 다시 계산
3. 체크섬 블록 쓰기
4. D1 데이터 블록 쓰기

-> 총: 1 번 읽기 + 2 번 쓰기

반면 첫 번째 방식은 1 번 쓰기 때문에 성능 차이가 발생한다.

전체 흐름을 다시보면

- 디스크는 조용히 틀린 데이터를 줄 수 있다(block corruption)
- 그래서 체크섬이 필요하다
- 체크섬은 데이터의 “변화 여부”를 검출한다
- 다양한 알고리즘이 있고 각각 장단점이 있다
- 완벽한 체크섬은 없다(충돌 존재)
- 저장 방식에 따라 성능 비용이 달라진다.

RAID 는 “디스크가 고장 났을 때 복구”는 잘하지만, “조용히 틀린 데이터”는 못잡는다.

- RAID(복구)
- 체크섬(검출)

이 두 가지를 같이 써야 한다.

[4] 체크섬의 활용

체크섬을 계산해서 저장하는 것 까지 했다고 하자. 이제 중요한 건 읽을 때 어떻게 검증하느냐다.

블록 D 를 읽는 상황을 생각해보자. 디스크에는 두 가지가 같이 저장되어 있다.

- 데이터 블록 D
- 그때 계산해서 저장해둔 체크섬 -> Cs(D) (stored checksum)

이 상태에서 사용자가 데이터를 읽으면 다음 순서로 진행된다.

1. 디스크에서 데이터 D 를 읽는다.
2. 동시에 저장되어 있던 체크섬 Cs(D)도 읽는다.
3. 읽어온 데이터 D 를 가지고 다시 체크섬을 계산한다. -> Cc(D) (computed checksum)
4. 두 값을 비교한다

여기서 경우가 두 가지로 나뉜다.

경우 1: 정상

$C_s(D) == C_c(D)$

- 저장 당시 데이터와 지금 읽은 데이터가 동일하다는 의미
- 즉, 중간에 손상이 없다고 판단
- > 데이터를 그대로 사용자에게 반환

경우 2: 손상 발생

$C_s(D) != C_c(D)$

- 저장 이후에 데이터가 바뀌었다는 의미
- 디스크가 조용히 틀린 데이터를 준 상황
- > 이때 “손상 검출 성공”

손상된 걸 알았다면, 그 다음은

예시로 상황을 만들어보자.

디스크에 이런 데이터가 저장되어 있다고 가정한다.

$D = [10, 20, 30, 40]$

체크섬 $C_s(D) = 100$ (덧셈 체크섬)

시간이 지나서 디스크 내부 문제로 데이터가 아래와 같이 바뀌었다고 하자.

$D' = [10, 20, 99, 40]$

이제 읽을 때

- 저장된 체크섬: $C_s(D) = 100$
 - 새로 계산: $C_c(D') = 10 + 20 + 99 + 40 = 169$
 - > 값이 다름 -> 손상 검출
- 여기까지는 완벽하게 동작한다.

이 다음 진짜 문제는 “데이터가 망가진 건 알겠는데 정상 데이터는 어디서 가져오는가”이다.

경우 1: 복사본(중복)이 있는 경우

예를 들어, RAID 를 쓰고 있다고 하자.

- RAID 1(미러링): 같은 데이터가 다른 디스크에 있음
- RAID 5/6: 패리티로 복구 가능

이 경우 흐름은 이렇게 된다.

1. 체크섬 불일치 발견
2. 다른 디스크에서 동일 블록 읽음
3. 정상 데이터 확보
4. 필요하면 손상된 블록을 덮어써서 복구

즉, 검출 -> 복구 까지 가능

경우 2: 복사본이 없는 경우

이게 현실적으로 더 무서운 상황이다.

예:

- 단일 디스크,
- 백업 없음
- RAID 없음

이 경우:

1. 체크섬 불일치 발견
2. 하지만 대체 데이터 없음

-> 시스템이 할 수 있는건: 에러 반환, “이 데이터는 신뢰할 수 없음”이라고 알림
즉, 검출은 했지만, 복구는 못함

[디스크]

